



TrainHub

guida completa per l'esame

Il progetto spiegato da zero, in italiano semplice: tecnologie, struttura, codice, flussi dell'applicazione, database, sicurezza, modalità offline e preparazione all'orale.

Nessuna conoscenza iniziale richiesta.

Ogni termine tecnico viene introdotto prima di essere usato e ogni flusso viene collegato ai file reali del progetto.

Indice

00 Fondamenta assolute — Prima ancora di leggere il codice

01 Imparare a leggere i simboli del codice

02 Come parte davvero TrainHub

03 Componenti e Material UI

04 Router, layout e controllo dei ruoli

05 Autenticazione completa

06 Dati, cache e offline

07 Tutto il codice workout

08 Builder workout e nutrizione

09 Clienti, abbonamento, calendario e appuntamenti

10 Chat e aggiornamenti in tempo reale

11 Profilo, badge, scanner, premi, notifiche e progressi

12 Il database, spiegato da zero

13 Sicurezza: chi può fare cosa e perché

14 PWA, modalità offline, tema e avvio del progetto

15 Ripasso guidato per l'esame

A Appendice — Mappa di tutti i file dell'applicazione

Come usare questo manuale

Il PDF è autosufficiente e può essere letto dall'inizio alla fine. Quando una sezione indica un file, puoi affiancare VS Code per vedere le righe reali: premi `Ctrl+P`, scrivi il percorso e premi Invio. Con `Ctrl+` dividi l'editor e con `Ctrl+G` raggiungi una riga.

Non cercare di memorizzare immediatamente ogni nome. Per prima cosa ricostruisci sempre quattro domande: **quali dati entrano, cosa fa il codice, quali dati escono e quale parte dell'app cambia.**

Percorso consigliato se parti da zero: leggi i capitoli in ordine. I primi tre insegnano il linguaggio necessario per capire tutti i successivi. Alla fine usa il capitolo di ripasso per simulare l'orale e l'appendice per trovare rapidamente qualunque file.

Fondamenta assolute — Prima ancora di leggere il codice

Questo capitolo presume che tu non abbia mai programmato. Non descrive ancora una singola schermata: costruisce le parole necessarie per capire tutto il resto.

1. Che cos'è un'applicazione?

Un'applicazione è un insieme di istruzioni che un computer esegue per ricevere dati, elaborarli e produrre un risultato.

Esempio TrainHub:

```
input: il membro preme "Complete set" e inserisce 10 ripetizioni
elaborazione: il codice controlla i valori e prepara una richiesta
salvataggio: il database registra la serie
output: la schermata mostra 1/3 serie completate
```

Il **codice sorgente** è il testo scritto dagli sviluppatori. Il computer non "capisce l'intenzione": esegue le regole espresse da parole, simboli e ordine delle istruzioni.

2. Hardware, sistema operativo, browser e applicazione

- **hardware**: telefono o computer fisico;
- **sistema operativo**: Windows, Android o iOS;
- **browser**: Chrome, Safari ecc.; legge HTML/CSS/JavaScript e offre funzioni come fotocamera e memoria locale;
- **TrainHub**: codice caricato ed eseguito dal browser;
- **server Supabase**: computer remoto che conserva account e dati condivisi.

Quando usi TrainHub, parte del lavoro avviene nel browser e parte sui server Supabase.

3. Frontend, backend e database

Frontend

È la parte eseguita nel browser e visibile all'utente. In TrainHub comprende:

- schermate;
- pulsanti e campi;
- navigazione;
- controlli immediati dei form;
- cache locale;
- comunicazione con Supabase.

La cartella principale è `src/`.

Backend

È la parte server che riceve richieste, verifica autorizzazioni e compie operazioni protette. TrainHub usa Supabase invece di un server scritto interamente a mano.

Database

È la memoria strutturata e persistente. Una variabile React sparisce chiudendo la pagina; una riga PostgreSQL rimane.

Esempio:

```
profiles: Daniel è member e ha Marco come assigned_pro_id
workout_runs: Daniel ha iniziato una determinata sessione alle 18:03
set_logs: nella run ha eseguito 10 ripetizioni con 40 kg
```

4. Locale e remoto

- **locale**: sul dispositivo attuale;
- **remoto**: sul server raggiunto attraverso Internet.

`useState` conserva un valore locale della schermata. PostgreSQL conserva dati remoti condivisi. IndexedDB conserva una copia locale duratura di dati remoti per lavorare offline.

5. File, cartella, percorso e modulo

Un **file** contiene testo o risorse. Una **cartella** raggruppa file. Il **percorso** dice dove si trova:

```
src/components/OfflineBanner.jsx
```

- `src` : codice frontend;
- `components` : elementi riutilizzabili;
- `OfflineBanner.jsx` : file specifico.

Un file JavaScript con `import` ed `export` è un **modulo**. Decide cosa usare da altri file e cosa rendere disponibile agli altri.

```
import { useState } from 'react'
```

significa: “dal pacchetto React prendi l’elemento esportato con nome `useState` e rendilo disponibile in questo file”. Non copia manualmente il suo codice e non esegue ancora `useState`.

```
export default function OfflineBanner() { ... }
```

significa: “l’elemento principale che questo file offre agli altri è la funzione `OfflineBanner`”.

6. Linguaggio, libreria, framework e strumento

Queste parole non sono sinonimi.

- **JavaScript**: linguaggio di programmazione del frontend;
- **JSX**: sintassi usata dentro JavaScript per descrivere elementi dell'interfaccia;
- **React**: libreria che trasforma componenti e dati in interfaccia aggiornata;
- **Material UI (MUI)**: libreria di componenti grafici React;
- **React Router**: libreria che associa URL a schermate;
- **TanStack Query**: libreria per leggere, scrivere e memorizzare dati server;
- **Supabase**: piattaforma backend;
- **PostgreSQL/SQL**: database e linguaggio con cui viene definito/interrogato;
- **Vite**: strumento che avvia lo sviluppo e costruisce la versione da pubblicare;
- **Node.js**: programma che esegue JavaScript fuori dal browser, usato da Vite e dai controlli;
- **TypeScript**: JavaScript con controlli di tipo; nel progetto appare nella Edge Function push.

Una libreria è codice già pronto che il progetto chiama. Non “fa tutto automaticamente”: TrainHub le passa parametri e combina i risultati.

7. Valore, variabile e tipo

Un **valore** è un dato:

```
160                // numero
'TrainHub'         // stringa, cioè testo
true               // booleano vero
false              // booleano falso
null               // assenza intenzionale
undefined          // valore non definito
```

Una **variabile** assegna un nome a un valore:

```
const width = 160
```

Da quel momento `width` indica 160. `const` impedisce di riassegnare quel nome a un altro valore. Non significa che un oggetto contenuto sia magicamente immutabile.

`let` si usa quando il nome dovrà essere riassegnato:

```
let stream = null
stream = await navigator.mediaDevices.getUserMedia(...)
```

8. Oggetto, proprietà, array e metodo

Un **oggetto** raggruppa valori con nomi:

```
const profile = {
  full_name: 'Daniel Aresta',
  role: 'member',
}
```

`profile.role` entra nell'oggetto e legge la proprietà `role`.

Un **array** è una sequenza ordinata:

```
const sessions = ['Upper Body', 'Leg Day']
```

`sessions[0]` è il primo elemento. In programmazione si comincia da zero.

Un **metodo** è una funzione raggiunta attraverso un oggetto:

```
sessions.filter(...)
```

Qui `filter` è un metodo dell'array.

9. Funzione, parametro, argomento e valore restituito

Una funzione è un gruppo di istruzioni riutilizzabile.

```
function double(number) {
  return number * 2
}
```

- `double` : nome;
- `number` : parametro, cioè nome del dato che entrerà;
- corpo tra `{}` : istruzioni;
- `return` : valore che esce.

Quando scriviamo:

```
double(5)
```

stiamo **chiamando** la funzione. `5` è l'argomento reale assegnato al parametro `number`. Il risultato è 10.

Differenza fondamentale:

```
double      // riferimento alla funzione: non la esegue
double(5)   // chiamata: la esegue subito
```

10. Funzione freccia e callback

Questa è una funzione normale scritta in forma compatta:

```
() => navigator.onLine
```

- `()` significa che non riceve parametri;
- `=>` introduce il corpo;
- non essendoci graffe, restituisce direttamente `navigator.onLine`.

Equivale a:

```
function leggiStatoRete() {  
  return navigator.onLine  
}
```

Una **callback** è una funzione passata a un'altra funzione affinché venga chiamata più tardi.

```
window.addEventListener('online', goOnline)
```

Non stiamo chiamando `goOnline()`. Passiamo `goOnline` al browser, che la chiamerà quando si verifica l'evento `online`.

11. Sincrono, asincrono, Promise e `await`

Il codice sincrono termina subito prima di proseguire. Una richiesta Internet richiede tempo e restituisce una **Promise**, cioè la promessa di un risultato futuro.

```
const result = await fetchSomething()
```

`await` mette in pausa quella funzione asincrona finché la Promise termina. Non blocca tutto il browser.

```
async function load() { ... }
```

`async` permette di usare `await` e fa sì che la funzione restituisca sempre una Promise.

`try/catch/finally` gestisce successo ed errore:

```
try {  
  const data = await request()  
} catch (error) {  
  // richiesta fallita  
} finally {  
  // eseguito in entrambi i casi  
}
```


12. Evento

Un evento è qualcosa che accade:

- click su un pulsante;
- invio di un form;
- rete diventata online;
- nuovo messaggio nel database;
- tempo trascorso.

Il codice registra una callback e reagisce quando l'evento arriva.

```
const goOnline = () => setOnline(true)
window.addEventListener('online', goOnline)
```

Il browser conserva `goOnline`. Quando rileva il ritorno della rete, la esegue.

13. Stato e render in React

Un **componente React** è una funzione che restituisce una descrizione dell'interfaccia.

```
function Greeting() {
  return <p>Hello</p>
}
```

Il **render** è l'esecuzione della funzione componente per capire cosa mostrare.

Lo **stato** è una memoria locale che React osserva:

```
const [online, setOnline] = useState(true)
```

`useState` restituisce un array con due elementi:

1. `online`: valore attuale;
2. `setOnline`: funzione fornita da React per cambiarlo.

```
setOnline(false)
```

non modifica soltanto una variabile: comunica a React il nuovo valore e richiede un nuovo render. Al render successivo, `online` vale `false` e il JSX può mostrare il banner offline.

14. Hook

Gli hook sono funzioni React con nome che inizia per `use`.

- `useState`: memoria locale osservata;
- `useEffect`: collega il componente a qualcosa di esterno dopo il render;

- `useRef` : conserva un valore o un riferimento senza richiedere render;
- `useContext` : legge un valore condiviso da un Provider;
- `useQuery` : hook TanStack che gestisce una lettura server;
- `useMutation` : hook TanStack che gestisce una scrittura server.

Non sono parole del linguaggio JavaScript: sono funzioni importate da librerie.

15. DOM, HTML e JSX

Il browser trasforma l'HTML in una struttura di oggetti chiamata DOM. React aggiorna il DOM.

JSX:

```
<Button disabled={saving}>Save</Button>
```

non è una stringa e non è HTML puro. Vite lo trasforma in chiamate JavaScript a React. Le graffe dentro JSX inseriscono un'espressione JavaScript: `disabled` riceve il booleano contenuto in `saving`.

Un nome minuscolo come `<div>` rappresenta un elemento del browser. Un nome maiuscolo come `<Button>` o `<OfflineBanner>` rappresenta un componente React.

16. Props e children

Le **props** sono gli input di un componente:

```
<BrandLogo width={80} />
```

React chiama concettualmente `BrandLogo({ width: 80 })`.

Il contenuto annidato arriva nella prop speciale `children`:

```
<ThemeProvider theme={theme}>
  <App />
</ThemeProvider>
```

Per `ThemeProvider`, `<App />` è `children`.

17. Provider e Context

Un Provider è un componente che rende un valore disponibile a tutti i discendenti.

Immagina una presa elettrica dell'edificio: ogni stanza interna può collegarsi senza far passare un cavo come prop attraverso tutte le stanze intermedie.

In TrainHub:

- `ThemeProvider` fornisce colori e font;
- `PersistQueryClientProvider` fornisce cache e gestione dati;

- `AuthProvider` fornisce sessione, utente e profilo;
- `RouterProvider` decide la schermata dall'URL.

“Fornire” non significa eseguire tutte le funzioni al posto loro: significa creare il contesto a cui i figli possono accedere.

18. URL, rotta e parametro

Un URL identifica una posizione dell'app:

```
/p/clients/123/progress
```

Una **rotta** associa quel modello a un componente. `:clientId` è una parte variabile; `useParams()` la legge.

React Router cambia componente senza chiedere al server un nuovo file HTML completo. TrainHub è quindi una **Single Page Application**: una pagina HTML, molte schermate React.

19. API e richiesta

API è un'interfaccia con cui due programmi comunicano. Il frontend non apre direttamente i file interni di PostgreSQL: invia richieste all'API Supabase.

```
supabase.from('profiles').select(...)
```

chiede dati a una tabella.

```
supabase.rpc('create_appointment_secure', parameters)
```

chiede al database di eseguire una funzione protetta.

La risposta può contenere `data` oppure `error`.

20. Query, mutation e cache

- **query**: lettura di dati, per esempio “dammi il piano di Daniel”;
- **mutation**: operazione che cambia dati, per esempio “registra una serie”;
- **cache**: copia di un dato già ottenuto, utile per non ripetere tutto e per mostrare contenuto offline;
- **invalidate**: segnala che una copia potrebbe essere vecchia e va riletta;
- **optimistic update**: mostra subito il risultato previsto, poi conferma o annulla quando risponde il server.

21. Memorie diverse usate da TrainHub

- stato React: dura finché il componente è montato;
- `localStorage`: piccole stringhe persistenti nel browser;
- IndexedDB: archivio locale più strutturato per cache e mutation;
- cache HTTP/Service Worker: file necessari ad avviare l'app;

- PostgreSQL: fonte remota e condivisa dei dati applicativi.

Dire “salvato” senza specificare dove è ambiguo. Nel manuale aggiornato verrà sempre indicata la memoria coinvolta.

22. Autenticazione e autorizzazione

- **autenticazione:** dimostrare chi sei, per esempio con email e password;
- **autorizzazione:** stabilire cosa quella persona può fare.

Supabase Auth produce una sessione. Il database usa l'identità della sessione per applicare RLS e RPC. Vedere un pulsante non concede il permesso; il server verifica sempre.

23. Esecuzione completa di TrainHub

1. Il browser legge index.html.
2. index.html carica main.jsx.
3. main.jsx chiede a React di renderizzare App.
4. App installa tema, dati/cache, autenticazione e router.
5. Il router osserva l'URL e sceglie una schermata.
6. La schermata esegue query per ottenere dati.
7. Il modulo data invia richieste a Supabase.
8. Supabase verifica sessione e permessi.
9. La risposta entra nella cache.
10. React esegue un nuovo render e aggiorna ciò che vedi.

Quando l'utente compie un'azione:

```
click → callback → mutation → funzione data → RPC/database  
← nuovo render ← cache aggiornata ← risposta
```

Questa è la mappa fondamentale dell'intero progetto.

Lezione 01 — Imparare a leggere i simboli del codice

Apri nel codice

In VS Code premi `Ctrl+P`, scrivi:

```
src/components/ScreenState.jsx
```

e premi Invio. Poi premi `Ctrl+\` per affiancarlo a questa lezione.

Non provare ancora a capire il file completo. Lo useremo soltanto per riconoscere i simboli.

1. Il codice è testo con regole precise

Il computer non indovina. Ogni simbolo ha una funzione.

Parentesi tonde `()`

Servono soprattutto per:

- passare dati a una funzione;
- dichiarare quali dati riceve una funzione;
- raggruppare un'espressione;
- racchiudere JSX su più righe.

Esempio reale, riga 23:

```
export function ErrorState({ error, onRetry }) {
```

`ErrorState` riceve due valori: `error` e `onRetry`.

Esempio reale, riga 38:

```
<Button variant="contained" onClick={onRetry}>
```

Qui non ci sono parentesi tonde perché stiamo descrivendo un componente JSX. `onClick={onRetry}` significa: quando il pulsante viene premuto, usa la funzione ricevuta in `onRetry`.

Graffe `{}`

Hanno più usi.

Nel codice JavaScript delimitano un blocco:

```
function LoadingState() {  
  // istruzioni della funzione  
}
```

Nel JSX permettono di inserire JavaScript:

```
<Box sx={centred}>
```

Senza graffe, `centred` sarebbe testo. Con le graffe, React usa il valore della variabile `centred`.

Nella lista dei parametri estraggono proprietà da un oggetto:

```
function ErrorState({ error, onRetry })
```

Questo è chiamato **destructuring**. Invece di ricevere `props` e scrivere `props.error`, la funzione estrae subito `error` e `onRetry`.

Parentesi quadre []

Rappresentano normalmente una lista, detta array.

```
[]
```

è una lista vuota.

Negli hook React possono anche fare destructuring di due valori:

```
const [online, setOnline] = useState(true)
```

`useState` restituisce una coppia:

1. valore corrente: `online`;
2. funzione per cambiarlo: `setOnline`.

Virgolette

Testo tra virgolette è una **stringa**.

```
'Loading'  
"contained"
```

Apici singoli e doppi rappresentano entrambi testo. Il progetto usa soprattutto apici singoli in JavaScript e doppi negli attributi JSX.

Backtick ` `

Permette di inserire valori dentro una stringa con `${...}`.

```
`Syncing ${waiting} changes`
```

Se `waiting` vale 3, il risultato è `Syncing 3 changes`.

Punto .

Accede a una proprietà.

```
error.message
```

significa: prendi la proprietà `message` dell'oggetto `error`.

```
navigator.onLine
```

legge la proprietà del browser che indica lo stato di rete.

Virgola ,

Separa elementi: parametri, proprietà o voci di una lista.

Due punti :

In un oggetto separano nome e valore:

```
{  
  display: 'flex',  
  textAlign: 'center',  
}
```

`display` è il nome della proprietà; `'flex'` è il valore.

Punto interrogativo ? e due punti :

Insieme formano una scelta, detta operatore ternario:

```
offline ? 'You are offline' : 'Something went wrong'
```

Si legge:

Se `offline` è vero, usa il primo testo; altrimenti usa il secondo.

? . accesso opzionale

```
error?.message
```

significa: leggi `message` soltanto se `error` esiste. Se manca, non generare un errore JavaScript.

?? valore di riserva

```
error?.message ?? 'Please try again.'
```

Se il messaggio manca, usa il testo di riserva.

! negazione

```
!navigator.onLine
```

Se `navigator.onLine` è vero, la negazione è falsa. Significa “non online”.

=> funzione freccia

```
() => setOnline(true)
```

È una funzione compatta. Non riceve parametri, e quando viene chiamata esegue `setOnline(true)`.

```
(event) => setValue(event.target.value)
```

Riceve `event` e usa il valore del campo che ha prodotto l'evento.

2. Le parole fondamentali

import

```
import { Box, Button } from '@mui/material'
```

Significa: usa in questo file `Box` e `Button`, esportati dalla libreria Material UI.

```
import PageHeader from '../components/PageHeader.jsx'
```

Significa: usa l'esportazione principale del file `PageHeader`.

Percorsi:

- `./` parte dalla cartella corrente;
- `../` sale di una cartella;
- `../../` sale di due;
- un nome come `react` cerca una libreria in `node_modules`.

const

```
const offline = !navigator.onLine
```

Crea un nome per un valore. Il collegamento `offline` non verrà riassegnato con `=` dentro quello stesso blocco.

`const` non significa che ogni parte interna di un oggetto sia magicamente immutabile. Significa che quella variabile non verrà fatta puntare a un altro valore.

let

È come `const`, ma permette di riassegnare il valore. Nel progetto si usa quando una variabile deve cambiare all'interno della stessa funzione.

function

```
function LoadingState() {
  return ...
}
```

Crea una funzione chiamata `LoadingState`. Una funzione è un gruppo riutilizzabile di istruzioni.

Parametri e argomenti

```
function somma(a, b) {
  return a + b
}

somma(2, 3)
```

`a` e `b` sono **parametri**, cioè nomi nella definizione. `2` e `3` sono **argomenti**, cioè valori usati nella chiamata.

Nel componente:

```
function ErrorState({ error, onRetry })
```

`error` e `onRetry` sono parametri estratti dalle props.

return

Indica il risultato della funzione.

In un componente React il risultato è normalmente il JSX da mostrare.

export

Rende una funzione o un valore utilizzabile da altri file.

```
export function LoadingState() { ... }
```

è un export con nome. Per importarlo servono le graffe:

```
import { LoadingState } from './ScreenState.jsx'
```

```
export default function PageHeader() { ... }
```

è l'export principale del file. Si importa senza graffe e il chiamante può scegliere il nome.

if

Esegue un blocco soltanto se una condizione è vera.

```
if (error) throw error
```

Se esiste un errore, interrompi il percorso normale e propagalo.

null e undefined

Entrambi esprimono assenza, ma in modo diverso.

- **undefined** : valore non definito/non ancora assegnato;
- **null** : assenza impostata intenzionalmente.

In JSX **null** significa spesso “non mostrare nulla”.

true e false

Sono valori booleani: vero e falso.

3. JSX riga per riga

Apri le righe 14–21 di `ScreenState.jsx`.

```
export function LoadingState() {  
  return (  
    <Box sx={centred} role="status" aria-label="Loading">  
      <CircularProgress />  
    </Box>  
  )  
}
```

```
)  
}
```

Spiegazione completa:

- `export` : altri file possono usare questa funzione.
- `function` : stiamo definendo un blocco eseguibile.
- `LoadingState` : nome, con iniziale maiuscola perché è un componente React.
- `()` : non riceve props.
- `{` : inizio del corpo della funzione.
- `return` : restituisce ciò che React deve mostrare.
- `(` : permette di scrivere il JSX su più righe.
- `<Box ...>` : apre un contenitore Material UI.
- `sx={centred}` : applica l'oggetto di stile dichiarato nelle righe 3–12.
- `role="status"` : comunica a tecnologie assistive che è uno stato aggiornato.
- `aria-label="Loading"` : fornisce un nome leggibile da screen reader.
- `<CircularProgress />` : indicatore circolare. `/>` significa che non contiene figli.
- `</Box>` : chiude il contenitore.
- `)` : chiude il valore restituito.
- `}` : chiude la funzione.

4. Oggetto di stile `centred`

Guarda le righe 3–12.

```
const centred = {  
  display: 'flex',  
  flexDirection: 'column',  
  alignItems: 'center',  
  justifyContent: 'center',  
  textAlign: 'center',  
  p: 4,  
  gap: 2,  
  minHeight: 240,  
}
```

- `display: 'flex'` : usa il sistema di disposizione Flexbox.
- `flexDirection: 'column'` : mette gli elementi dall'alto al basso.
- `alignItems: 'center'` : li centra lungo l'asse orizzontale.
- `justifyContent: 'center'` : li centra lungo l'asse verticale disponibile.
- `textAlign: 'center'` : centra il testo.
- `p: 4` : padding interno secondo la scala MUI, non necessariamente quattro pixel.
- `gap: 2` : distanza tra figli secondo la scala MUI.
- `minHeight: 240` : altezza minima 240 pixel CSS.

Questo oggetto viene riusato da loading, error ed empty state. È un esempio di eliminazione della duplicazione.

5. Esercizio senza modificare il codice

Indica a voce:

1. quale riga importa il pulsante;
2. quale funzione non riceve parametri;
3. quali sono i parametri di `ErrorState` ;
4. che cosa accade se `onRetry` manca;
5. perché `error?.message` non causa errore se `error` è assente;
6. quali parametri hanno valori predefiniti in `EmptyState` .

Risposte:

1. riga 1;
2. `LoadingState` ;
3. `error` e `onRetry` ;
4. il ternario restituisce `null` , quindi il pulsante non appare;
5. `?` , interrompe l'accesso in modo sicuro;
6. `action = null` , `minHeight = 240` , `padding = 4` .

Devi saper dire

Un componente React è una funzione con iniziale maiuscola che restituisce JSX. Le props sono parametri. Le graffe nel JSX inseriscono valori JavaScript. Gli import recuperano funzioni e componenti da librerie o file locali; gli export li rendono disponibili agli altri moduli.

Lezione 02 — Come parte davvero TrainHub

In questa lezione aprirai tre file nell'ordine in cui entra in gioco l'app:

1. `index.html`;
2. `src/main.jsx`;
3. `src/App.jsx`.

Passo 1 — Apri `index.html`

Premi `Ctrl+P`, scrivi `index.html`, Invio.

Riga 1

```
<!doctype html>
```

Dice al browser che il documento usa HTML moderno. Non produce un elemento visibile.

Riga 2

```
<html lang="en">
```

Apri il documento HTML. `lang="en"` dichiara che il testo dell'app è inglese, utile a browser e screen reader.

Un elemento HTML normalmente ha:

```
<nome attributo="valore">contenuto</nome>
```

`<html>` è il tag di apertura; `</html>` alla fine è quello di chiusura.

Riga 3: `head`

`<head>` contiene informazioni sulla pagina, non il contenuto principale visibile.

Riga 4

```
<meta charset="UTF-8" />
```

`meta` descrive il documento. `charset` indica la codifica dei caratteri. UTF-8 permette accenti e simboli internazionali.

Il tag finisce con `</>`: non contiene testo o figli.

Righe 5–7: icone

```
<link rel="icon" href="/favicon.ico" sizes="48x48" />
<link rel="icon" type="image/svg+xml" href="/favicon.svg" sizes="any" />
<link rel="apple-touch-icon" href="/apple-touch-icon-180x180.png" />
```

`link` collega risorse esterne al documento.

- `rel`: tipo di relazione;
- `href`: percorso del file;
- `type`: formato MIME dell'SVG;
- `sizes`: dimensione supportata.

La apple touch icon viene usata quando il sito viene aggiunto alla Home Screen Apple.

Riga 8: viewport

```
<meta name="viewport" content="width=device-width, initial-scale=1.0, viewport-fit=cover" />
```

Dice al browser mobile:

- usa la larghezza reale del dispositivo;
- non partire con zoom diverso da 1;
- permetti al layout di coprire l'intero schermo, gestendo poi le safe area.

Senza questa riga un sito mobile può apparire rimpicciolito.

Riga 9: colore del browser

```
<meta name="theme-color" content="#FE6363" />
```

Suggerisce il colore brand ad alcune parti dell'interfaccia browser/PWA.

Righe 10–12: commento

```
<!-- ... -->
```

È un commento per sviluppatori. Il browser non lo mostra. Spiega che la descrizione deve restare uguale a quella del manifest PWA.

Righe 13–16: descrizione

La meta description riassume TrainHub per motori di ricerca e strumenti di analisi.

Riga 17

```
<title>TrainHub</title>
```

È il titolo della scheda del browser.

Riga 19: `body`

`body` contiene ciò che appartiene alla pagina visibile/eseguibile.

Riga 20: il punto di montaggio

```
<div id="root"></div>
```

È un contenitore vuoto con ID `root`. React inserirà qui tutta l'app.

`id` è un identificatore della pagina HTML. Deve poter essere trovato dal JavaScript.

Riga 21: avvia il JavaScript

```
<script type="module" src="/src/main.jsx"></script>
```

- `script` : esegui codice;
- `type="module"` : abilita import/export JavaScript;
- `src` : il primo file da caricare è `main.jsx`.

Questa riga collega HTML e React.

Passo 2 — Apri `src/main.jsx`

Premi `Ctrl+P`, scrivi `src/main.jsx`, Invio.

Riga 1

```
import { StrictMode } from 'react'
```

Importa un export con nome dalla libreria React. `StrictMode` abilita controlli aggiuntivi in sviluppo. Può eseguire nuovamente certi passaggi per evidenziare effetti scritti male; non duplica normalmente il comportamento in produzione.

Riga 2

```
import { createRoot } from 'react-dom/client'
```

React descrive i componenti; React DOM sa montarli nella pagina del browser. `createRoot` crea la radice React nel nodo HTML.

Riga 3

```
import '@fontsource-variable/inter'
```

È un import eseguito per il suo effetto: include il font Inter. Non assegna un nome a una variabile.

Riga 4

```
import App from './App.jsx'
```

Importa l'export default di `App.jsx`. `./` significa “nella stessa cartella di main”.

Righe 6–7

```
import { registerSW } from 'virtual:pwa-register'  
registerSW({ immediate: true })
```

`virtual:pwa-register` non è un normale file visibile: viene fornito dal plugin PWA durante build/dev.

`registerSW` registra il service worker. Il parametro è un oggetto:

- `immediate: true`: prova a registrarlo senza aspettare un ulteriore evento della pagina.

Nel server di sviluppo la configurazione disattiva il vero service worker; viene usato nel build di produzione.

Riga 10

```
createRoot(document.getElementById('root')).render(  

```

Dividiamola:

1. `document` rappresenta la pagina HTML;
2. `getElementById('root')` cerca il div della riga 20;
3. `createRoot(...)` lo trasforma nella radice controllata da React;
4. `.render(...)` dice quale albero di componenti inserire.

Righe 11–13

```
<StrictMode>  
  <App />  
</StrictMode>
```


`App` è figlio di `StrictMode`. In JSX un componente senza contenuto interno si scrive `<App />`.

Riga 14: virgola finale

La virgola è ammessa nelle chiamate multilinea. Facilita modifiche e formattazione; non aggiunge un argomento vuoto.

Passo 3 — Apri `src/App.jsx`

Premi `Ctrl+P`, scrivi `src/App.jsx`, Invio.

Prima domanda: che cos'è `App`?

`App` è il **componente radice**, cioè il primo componente TrainHub che React esegue. Il nome non indica un programma separato e il file non contiene tutte le schermate.

Il suo compito è preparare l'ambiente comune prima di mostrare una pagina:

```
tema grafico
  ↓
gestione e memoria dei dati
  ↓
identità dell'utente
  ↓
scelta della schermata dall'URL
```

Una pagina come `WorkoutPlanScreen` rappresenta una funzione visibile. Un `Provider` non è normalmente una pagina: è un componente infrastrutturale che avvolge i figli e rende disponibile un servizio.

Analogia concreta:

- `ThemeProvider`: impianto grafico, stabilisce colori e font;
- `PersistQueryClientProvider`: archivio e corriere dei dati;
- `AuthProvider`: controllo identità all'ingresso;
- `RouterProvider`: cartello che decide in quale stanza andare dall'indirizzo;
- schermata: la stanza realmente visibile.

Import 1: tema MUI

```
import { CssBaseline, ThemeProvider } from '@mui/material'
```

- `ThemeProvider`: rende il tema disponibile ai componenti figli;
- `CssBaseline`: normalizza gli stili di base del browser.

La parola **Provider** indica spesso un componente che fornisce qualcosa a tutti i discendenti senza passarlo manualmente in ogni prop.

Import 2: router

```
import { RouterProvider } from 'react-router'
```

Rende attivo l'albero di rotte.

Import 3: cache persistita

```
import { PersistQueryClientProvider } from '@tanstack/react-query-persist-client'
```

Fornisce TanStack Query e collega il suo stato alla persistenza locale.

Import locali

```
import theme from './theme/index.js'
import router from './routes/index.jsx'
import { AuthProvider } from './features/auth/AuthProvider.jsx'
```

- `theme` : oggetto con colori, font e componenti;
- `router` : definizione degli URL;
- `AuthProvider` : sessione e profilo.

Cache

```
import { CACHE_MAX_AGE, persister, queryClient } from './lib/queryClient.js'
import { QUERY_CACHE_BUSTER } from './lib/cacheMigration.js'
```

- `queryClient` : il gestore centrale di query e mutation;
- `persister` : salva/rilegge la cache in IndexedDB;
- `CACHE_MAX_AGE` : tempo massimo della cache salvata;
- `QUERY_CACHE_BUSTER` : etichetta di versione. Se cambia, una cache incompatibile non viene riusata ciecamente.

Mutation defaults

```
import { registerMutationDefaults } from './data/mutations.js'
registerMutationDefaults(queryClient)
```

La seconda riga viene eseguita appena il modulo è caricato, prima del componente.

`registerMutationDefaults` è la funzione importata. `queryClient` è l'argomento che le viene passato. Le parentesi finali significano che la funzione viene eseguita immediatamente. Scrivere soltanto `registerMutationDefaults` sarebbe un riferimento alla funzione e non la eseguirebbe.

Perché? IndexedDB può salvare:

```
operazione = logSet
parametri = {...}
```

ma non può salvare la funzione JavaScript. `registerMutationDefaults` ricollega il nome dell'operazione alla funzione reale prima che la cache venga ripristinata.

Se fosse dentro `useEffect`, sarebbe troppo tardi: il provider potrebbe provare a riprendere la mutation prima della registrazione.

Esempio completo:

1. Daniel registra una serie mentre è offline.
2. TanStack salva in IndexedDB il nome "logSet" e i parametri della serie.
3. Daniel chiude TrainHub.
4. Più tardi il browser ricarica App.jsx.
5. `registerMutationDefaults` collega "logSet" alla funzione JavaScript `logSet`.
6. Il provider rilegge l'operazione da IndexedDB.
7. Quando torna la rete, `queryClient` può eseguire la funzione corretta.

IndexedDB conserva dati serializzabili, ma non il codice eseguibile della funzione. Per questo il collegamento nome → funzione viene ricostruito a ogni avvio.

Definizione di App

```
export default function App() {
  return (
```

`App` non riceve props. Restituisce provider annidati.

Quando React incontra `<App />` in `main.jsx`, chiama `App()`. La funzione restituisce JSX, cioè la descrizione dell'albero da costruire. Non restituisce un file HTML e non salva dati nel database.

ThemeProvider

```
<ThemeProvider theme={theme}>
```

Parametro/prop:

- `theme={theme}`: passa l'oggetto importato. Le graffe indicano un valore JavaScript, non il testo "theme".

Tutto ciò che si trova dentro può usare quel tema.

`ThemeProvider` riceve la prop `theme` e anche la prop implicita `children`, cioè tutto ciò che è scritto tra apertura e chiusura. Conserva il tema in un Context. Quando `BrandLogo` chiama `useTheme()`, React risale fino a questo Provider e gli restituisce lo stesso oggetto.

CssBaseline

```
<CssBaseline />
```

Non riceve props nel nostro uso. Applica una base CSS coerente.

`CssBaseline` non è una pagina. Inserisce regole CSS globali che uniformano il comportamento predefinito dei browser. È autochiuso con `/>` perché non contiene figli.

PersistQueryClientProvider

```
<PersistQueryClientProvider
  client={queryClient}
  persistOptions={{ ... }}
  onSuccess={() => queryClient.resumePausedMutations()}
>
```

Props:

- `client` : quale `QueryClient` usare;
- `persistOptions` : oggetto di configurazione della persistenza;
- `onSuccess` : funzione eseguita dopo il ripristino della cache.

Nota le doppie graffe `{{ ... }}`:

- quelle esterne inseriscono JavaScript nel JSX;
- quelle interne creano un oggetto JavaScript.

persist, maxAge, buster

- `persist` : oggetto con funzioni per leggere, scrivere e rimuovere la cache da IndexedDB;
- `maxAge` : numero di millisecondi; una copia più vecchia del limite non viene riusata;
- `buster` : stringa di versione; se è diversa dalla versione salvata, la vecchia cache viene migrata o scartata in sicurezza.

Persistenza significa che il dato sopravvive al ricaricamento. **Hydration** significa leggere il dato serializzato da IndexedDB e rimetterlo nella memoria viva del `QueryClient`.

dehydrateOptions

“Dehydrate” significa trasformare lo stato in una forma serializzabile da salvare.

L'oggetto vivo contiene anche funzioni e riferimenti non salvabili. La `dehydrate` conserva solo dati; la `hydrate` li ricostruisce all'avvio.

```
shouldDehydrateMutation: (mutation) => mutation.state.status === 'pending'
```

Riceve una mutation. Restituisce `true` se lo stato è pending. Quindi salva ogni scrittura non terminata, anche se il browser non l'ha ancora classificata formalmente come paused.

`mutation` è il parametro della funzione freccia. TanStack chiama questa funzione per ogni mutation. `mutation.state.status` legge la proprietà `status`; il confronto `=== 'pending'` produce `true` oppure `false`.

```
shouldDehydrateQuery: (query) => query.state.data !== undefined
```

Salva una query se possiede dati. Anche una query che ha fallito un aggiornamento offline può mantenere dati vecchi utili.

`undefined` significa che non esiste ancora un valore. `[]`, invece, è un dato valido che significa “lista vuota”; `null` può significare validamente “nessun piano”. Entrambi vengono conservati.

`!==` significa “diverso anche come tipo/valore”.

onSuccess

```
() => queryClient.resumePausedMutations()
```

È una funzione senza parametri. Quando la persistenza è stata ripristinata, chiede al client di riprendere le mutation in pausa.

Il Provider conserva questa callback e la chiama più tardi: non viene eseguita mentre React legge il JSX.

AuthProvider

```
<AuthProvider>
```

Tutto ciò che sta dentro può usare `useAuth()` per leggere sessione, utente e profilo.

`AuthProvider` riceve `RouterProvider` come `children`. Prima determina la sessione e carica il profilo. Le schermate possono poi chiamare `useAuth()` senza passare manualmente l'utente attraverso ogni componente intermedio.

RouterProvider

```
<RouterProvider router={router} />
```

Prop:

- `router`: albero delle rotte importato.

È il punto in cui comparirà la schermata scelta dall'URL.

`router` non è un indirizzo: è l'oggetto creato in `routes/index.jsx`, contenente l'albero path → componente. `RouterProvider` osserva l'URL, trova il ramo corrispondente e lo renderizza.

Ordine temporale reale

1. Il browser importa App.jsx.
2. Vengono risolti gli import.
3. registerMutationDefaults(queryClient) viene eseguita subito.
4. React chiama App().
5. ThemeProvider rende disponibile il tema.
6. PersistQueryClientProvider ripristina IndexedDB.
7. AuthProvider controlla la sessione Supabase.
8. RouterProvider osserva l'URL.
9. AppLayout aspetta che identità e ruolo siano conosciuti.
10. Viene mostrata la schermata corretta.
11. Finito il ripristino, onSuccess riprende le scritture sospese.

Cosa App.jsx non fa

- non contiene il database;
- non decide direttamente quale pagina mostrare: lo fa il router;
- non esegue il login: lo fanno AuthProvider e Supabase Auth;
- non registra direttamente set o appuntamenti;
- prepara i servizi comuni che permettono alle schermate interne di farlo.

Albero finale

```
ThemeProvider
├─ CssBaseline
├─ PersistQueryClientProvider
│   └─ AuthProvider
│       └─ RouterProvider
│           └─ schermata scelta dall'URL
```

I provider esterni sono disponibili a quelli interni. Una schermata scelta dal router può quindi usare tema, query e autenticazione.

Esercizio in VS Code

1. In index.html, metti il cursore su root.
2. In main.jsx, trova il secondo root: deve essere lo stesso ID.
3. Su App premi F12: VS Code dovrebbe portarti a App.jsx.
4. In App.jsx, premi Ctrl+F e cerca Provider: elenca i quattro provider.
5. Spiega perché registerMutationDefaults è fuori dalla funzione App.

Devi saper dire

Il browser carica index.html, che contiene il nodo root e importa main.jsx. Main crea la radice React e renderizza App. App annida tema, cache persistita, autenticazione e router. I mutation defaults vengono

registrati prima della hydration perché IndexedDB conserva parametri e chiavi, non le funzioni JavaScript.

Lezione 03 — Componenti e Material UI

Hai già analizzato `ScreenState.jsx`. Ora imparerai a distinguere logica, props, JSX e stile.

Apri `src/components/BrandLogo.jsx`

Import

- `Box`: contenitore MUI capace di rendere anche un elemento diverso da `div`.
- `useTheme`: hook che restituisce il tema corrente.

Commento JSDoc

Il blocco `/** ... */` documenta il componente. Le righe con `@param` descrivono i parametri.

```
props          oggetto completo delle proprietà
props.width    larghezza opzionale, stringa o numero
[props.width]  le parentesi quadre nella documentazione significano opzionale
```

Firma

```
function BrandLogo({ width = 160 })
```

Riceve le props, estrae `width` e usa 160 se il chiamante non lo fornisce.

Variabili

```
const theme = useTheme()
const gap = theme.palette.background.default
const t = 'M26 ...'
```

- `useTheme()` è una chiamata di funzione: le parentesi vuote significano che non le passiamo argomenti. MUI restituisce l'oggetto `theme` creato in `src/theme/index.js`.
- `theme.palette.background.default` si legge da sinistra a destra: entra nell'oggetto `theme`, poi nella proprietà `palette`, poi in `background`, infine prende `default`. Il risultato è il colore dello sfondo generale dell'app.
- quel colore viene salvato nella variabile `gap`, cioè “spazio/separazione”. Viene usato come una gomma: alcune forme sono colorate come lo sfondo e sembrano ritagliare il logo.
- `t` è una stringa. Il nome corto indica la sagoma della lettera **T** di TrainHub. La stringa non viene mostrata all'utente: viene consegnata all'attributo SVG `d` più sotto.

Prima di continuare: che cos'è SVG?

SVG significa **Scalable Vector Graphics**. Invece di conservare una fotografia composta da pixel, descrive forme geometriche: rettangoli, linee, curve e colori. Il browser ridisegna quelle forme alla dimensione richiesta senza sgranarle.

Qui il logo non viene caricato da un file PNG. Il codice costruisce direttamente la **T** scura e la **H** color corallo, con una forma simile a un bilanciere.

Box `component="svg"`

Il componente Box normalmente produce un `div`. La prop `component="svg"` gli dice di produrre un SVG.

Props:

- `viewBox="0 0 269 192"`: crea un foglio virtuale che parte dal punto `(0, 0)`, è largo 269 unità ed è alto 192. Il punto `(0, 0)` è in alto a sinistra; aumentando `x` ci si sposta a destra, aumentando `y` si scende;
- `role="img"`: comunica che l'SVG è un'immagine;
- `aria-label="TrainHub"`: nome per screen reader;
- `sx={{ ... }}`: stile MUI scritto come oggetto JavaScript;
- `width`: è una scorciatoia per `width: width`; usa il parametro ricevuto dalla funzione, 160 per default;
- `height: 'auto'`: calcola automaticamente l'altezza mantenendo il rapporto 269:192, così il logo non si deforma;
- `display: 'block'`: tratta l'SVG come un blocco ed elimina il piccolo spazio che gli elementi inline possono lasciare sotto la linea di testo.

Il `viewBox` e `width` non sono la stessa cosa: il primo stabilisce le coordinate usate per disegnare; il secondo stabilisce quanto grande appare il risultato. Se si usa `<BrandLogo width={80} />`, tutte le 269×192 unità interne vengono scalate proporzionalmente dentro una larghezza visibile di 80 pixel.

`<g>`, `<rect>` e `<path>`

- `<g>` significa **group**. Non disegna niente da solo: raggruppa i figli. Il suo `fill` viene ereditato da tutte le forme interne che non specificano un altro colore;
- `<rect>` disegna un rettangolo. `x` e `y` indicano il punto in alto a sinistra, `width` e `height` le dimensioni, `rx` arrotonda gli angoli;
- `<path>` disegna una sagoma seguendo il piccolo linguaggio di comandi contenuto nell'attributo `d`;
- `fill` è il colore interno;
- `stroke` è il contorno;
- `strokeWidth` è lo spessore;
- `strokeLinejoin="round"` arrotonda le giunzioni.

Primo gruppo: costruzione della H corallo

```
<g fill={theme.palette.primary.main}>
```

Le graffe fanno entrare JavaScript dentro JSX. `theme.palette.primary.main` è il colore principale corallo. Tutte le forme fino a `</g>` ricevono quel colore.

```
<rect x="128" y="0" width="32" height="192" />
```

Disegna la barra verticale sinistra della H: comincia a 128 unità da sinistra, parte dall'alto, è larga 32 e arriva per tutta l'altezza 192.

```
<path d="M224 0h32v160a32 32 0 0 1-32 32z" />
```

Disegna la barra destra con il fondo arrotondato. Qui **d** significa **path data**, cioè "dati del percorso", non la lettera d e non un comando generico "disegna". I comandi si leggono così:

- **M224 0** : **Move**, sposta la penna al punto assoluto x=224, y=0 senza tracciare;
- **h32** : traccia una linea orizzontale relativa di 32 unità verso destra, arrivando a x=256;
- **v160** : linea verticale relativa di 160 unità verso il basso;
- **a32 32 0 0 1 -32 32** : crea un arco con raggi 32×32 e termina 32 unità a sinistra e 32 più in basso; è ciò che arrotonda il fondo;
- **z** : chiude la sagoma tornando al punto iniziale.

Nei path, una lettera maiuscola usa generalmente coordinate assolute; una minuscola usa spostamenti relativi alla posizione corrente.

```
<rect x="128" y="80" width="128" height="32" />
```

È la barra orizzontale che unisce le due aste e completa la H.

```
<rect x="115" y="84" width="10" height="24" rx="5" />  
<rect x="259" y="84" width="10" height="24" rx="5" />
```

Sono due piccoli terminali arrotondati ai lati della barra. **rx="5"** usa un raggio di 5; essendo metà della larghezza 10, le estremità sembrano pillole. Visivamente richiamano i fermi di un bilanciere.

Secondo gruppo: i ritagli color sfondo

```
<g fill={gap}>
```

Questo gruppo usa il colore dello sfondo. Non rende davvero trasparenti le forme: le copre con lo stesso colore che sta dietro, producendo visivamente degli spazi.

I quattro rettangoli verticali con **rx** separano asta, pesi e terminali del bilanciere. Il rettangolo orizzontale:

```
<rect x="139" y="91" width="106" height="10" rx="5" />
```

crea una fessura chiara al centro della barra corallo. Queste cinque forme rendono leggibili i diversi pezzi invece di mostrare un unico blocco pieno.

La stringa `t`, comando per comando

La variabile completa è:

```
M26 0 h133 v6 a26 26 0 0 1 -26 26 H96 v160 H64 V32 H0 v-6 A26 26 0 0 1 26 0 z
```

Costruisce la T:

- `M26 0` : parte vicino all'angolo superiore sinistro;
- `h133` : traccia verso destra la parte superiore;
- `v6` : scende di 6;
- `a26 26 ... -26 26` : arrotonda l'estremità destra della barra superiore;
- `H96` : va orizzontalmente alla coordinata assoluta x=96;
- `v160` : scende e forma il lato destro del gambo;
- `H64` : va a sinistra fino a x=64, formando la base del gambo;
- `V32` : risale fino alla coordinata y=32;
- `H0` : va verso il bordo sinistro;
- `v-6` : sale di 6;
- `A26 26 ... 26 0` : arrotonda l'estremità sinistra e torna verso il punto iniziale;
- `z` : chiude la sagoma, rendendola riempibile.

Non è necessario saper inventare a memoria una stringa SVG. Devi però sapere che `d` contiene istruzioni geometriche e riconoscere `M` come spostamento, `H/h` come linea orizzontale, `V/v` come verticale, `A/a` come arco e `z` come chiusura.

Perché la T viene disegnata due volte?

```
<path d={t} fill={gap} stroke={gap} strokeWidth="7" strokeLinejoin="round" />
<path d={t} fill={theme.palette.text.primary} />
```

SVG dipinge nell'ordine del codice: ciò che viene dopo finisce sopra ciò che è venuto prima.

1. Il primo path usa la sagoma T, la riempie con il colore di sfondo e aggiunge un contorno spesso 7 dello stesso colore. Funziona come una zona di separazione attorno alla T e copre leggermente la H sottostante.
2. Il secondo path ridisegna la stessa identica T sopra il ritaglio, questa volta con `text.primary`, il colore scuro del testo.

Se ci fosse soltanto il secondo path, la T scura e la H corallo si toccherebbero e la sovrapposizione sarebbe meno chiara. Il primo crea il bordo vuoto; il secondo mette dentro la forma visibile.

Cosa produce davvero l'intera funzione

`BrandLogo` riceve una larghezza e restituisce un'immagine vettoriale accessibile. Disegna prima la H/bilanciere corallo, sovrappone ritagli dello stesso colore dello sfondo e infine costruisce la T scura con un margine di separazione. I colori non sono fissi nel componente: arrivano dal tema, quindi il logo rimane coerente con il resto dell'app.

Apri `src/components/OfflineBanner.jsx`

Che cosa fa realmente questo file

`OfflineBanner` è il controllore visivo della sincronizzazione. Viene inserito in `AppLayout`, quindi è presente nelle pagine autenticate. Osserva tre situazioni diverse:

1. il dispositivo non ha rete;
2. esistono scritture ancora in attesa di essere inviate;
3. una scrittura è fallita definitivamente oppure è stata scartata dopo un aggiornamento dell'app.

Il componente non salva direttamente set o appuntamenti. Legge lo stato globale delle mutation gestite da TanStack Query e decide quale messaggio mostrare.

Import

```
import { useEffect, useState } from 'react'
```

- `useState` è una funzione React che crea memoria locale osservata;
- `useEffect` è una funzione React che collega il componente a sistemi esterni dopo il render, qui gli eventi del browser.

```
import { Alert, Box, Snackbar, Typography } from '@mui/material'
```

Sono quattro componenti grafici:

- `Box`: contenitore generico;
- `Typography`: testo con stile del tema;
- `Snackbar`: messaggio sovrapposto alla pagina;
- `Alert`: contenuto colorato per errore o avviso.

```
import CloudOffIcon from '@mui/icons-material/CloudOff'
```

Importa il disegno dell'icona cloud offline. È un componente React, non una variabile booleana sulla rete.

```
import { useMutationState } from '@tanstack/react-query'
```

`useMutationState` è un hook che interroga il `QueryClient` globale. Non legge PostgreSQL: guarda le operazioni di scrittura che TanStack sta gestendo nel browser.

```
import {  
  CACHE_MIGRATION_NOTICE_EVENT,  
  clearCacheMigrationNotice,  
}
```

```
readCacheMigrationNotice,  
} from '../lib/cacheMigration.js'
```

Questi sono esattamente i tre elementi importati:

- `CACHE_MIGRATION_NOTICE_EVENT` : una stringa usata come nome di un evento browser, precisamente `trainhub:cache-migration-notice` ;
- `readCacheMigrationNotice()` : funzione che legge da `localStorage` se un aggiornamento di TrainHub ha dovuto scartare vecchie operazioni offline incompatibili;
- `clearCacheMigrationNotice()` : funzione che elimina quell'avviso da `localStorage` quando l'utente lo chiude.

“Aggiornamento incompatibile” significa questo: il browser aveva salvato, per esempio, una vecchia operazione con parametri adatti alla versione precedente del codice; dopo un aggiornamento quei parametri potrebbero non essere più validi. TrainHub preferisce non inviare una scrittura potenzialmente sbagliata e avvisa l'utente di ripeterla.

plural

```
const plural = (n) => (n === 1 ? '' : 's')
```

Questa è una funzione freccia chiamata `plural` :

- riceve il parametro `n` , cioè un numero;
- controlla `n === 1` ;
- se è esattamente 1 restituisce testo vuoto;
- altrimenti restituisce `'s'` .

Viene chiamata dentro una frase inglese:

```
`${waiting} change${plural(waiting)}`
```

Con `waiting = 1` produce `1 change` ; con `waiting = 2` produce `2 changes` . Non modifica nessun dato.

Inizio del componente

```
export default function OfflineBanner() {
```

È un componente React senza props: le parentesi sono vuote perché non riceve dati dal genitore. Ottiene ciò che gli serve dal browser, dalla cache e dagli hook.

Stato `online`

```
const [online, setOnline] = useState(() => navigator.onLine)
```

Leggiamola simbolo per simbolo:

1. `useState(...)` è la funzione React che stiamo chiamando;
2. l'argomento passato a `useState` è `() => navigator.onLine`;
3. `() => navigator.onLine` è **la funzione** a cui si riferisce la spiegazione: è una funzione freccia senza nome e senza parametri;
4. React chiama quella funzione una sola volta, durante il primo render del componente;
5. la funzione restituisce `navigator.onLine`;
6. `navigator` è un oggetto globale fornito dal browser;
7. la sua proprietà `onLine` è un booleano: `true` se il browser ritiene disponibile la rete, `false` altrimenti;
8. `useState` restituisce un array di due elementi;
9. la sintassi `[online, setOnline]` estrae quei due elementi.

I due elementi sono:

- `online`: il valore attuale conservato da React;
- `setOnline`: funzione fornita da React per sostituire quel valore e richiedere un nuovo render.

La riga equivale concettualmente a:

```
function leggiStatoInizialeDellaRete() {  
  return navigator.onLine  
}  
  
const risultatoDiUseState = useState(leggiStatoInizialeDellaRete)  
const online = risultatoDiUseState[0]  
const setOnline = risultatoDiUseState[1]
```

Perché si dice “valore iniziale”? Perché leggere `navigator.onLine` una volta non crea un collegamento continuo. Se il dispositivo perde rete dieci secondi dopo, la vecchia lettura non cambia da sola. Per aggiornare `online`, più sotto vengono registrati gli eventi `online` e `offline`, che chiamano `setOnline(true)` o `setOnline(false)`.

La funzione freccia passata a `useState` si chiama **lazy initializer**: React la esegue soltanto alla creazione iniziale del componente, invece di ricalcolarla a ogni render.

Stato dismissed

```
const [dismissed, setDismissed] = useState(() => new Set())
```

- `new Set()` crea una collezione inizialmente vuota;
- un `Set` conserva valori senza duplicati;
- qui conterrà gli ID numerici delle mutation fallite che l'utente ha già riconosciuto chiudendo il messaggio;
- `dismissed.has(id)` restituisce vero se quell'ID è presente;
- `setDismissed(...)` sostituisce il Set e provoca un nuovo render.

Non viene usato un solo booleano come `errorClosed` : chiudere un errore non deve nascondere per sempre tutti gli errori che potrebbero arrivare dopo.

Stato `migrationNotice`

```
const [migrationNotice, setMigrationNotice] = useState(() => readCacheMigrationNotice())
```

La funzione passata a `useState` chiama `readCacheMigrationNotice()` una sola volta. Questa funzione:

1. cerca in `localStorage` la chiave `trainhub-cache-migration-notice` ;
2. se non esiste restituisce `null` ;
3. se esiste converte il testo JSON in un oggetto;
4. controlla versione e numero di operazioni scartate;
5. restituisce, per esempio, `{ version: 'trainhub-security-v4', discardedCount: 2 }`.

Quindi `migrationNotice` è `null` quando non c'è niente da comunicare, oppure un oggetto con il numero di modifiche che l'utente deve ripetere.

`useEffect`

```
useEffect(() => {
```

`useEffect` riceve come primo argomento una funzione callback. React la esegue dopo che il componente è comparso nella pagina.

Dentro vengono create tre funzioni:

```
const goOnline = () => setOnline(true)
const goOffline = () => setOnline(false)
const showMigrationNotice = (event) => setMigrationNotice(event.detail)
```

- `goOnline` non riceve parametri utili e imposta lo stato a vero;
- `goOffline` imposta lo stato a falso;
- `showMigrationNotice` riceve l'oggetto evento; `event.detail` contiene l'oggetto inserito nel `CustomEvent` da `cacheMigration.js`.

Poi:

```
window.addEventListener('online', goOnline)
```

- `window` rappresenta la finestra del browser;
- `addEventListener` registra una funzione da eseguire in futuro;
- `'online'` è il nome dell'evento;
- `goOnline` è passata senza parentesi: non viene eseguita ora; il browser la eseguirà al ritorno della rete.

Le tre registrazioni ascoltano ritorno rete, perdita rete e avviso personalizzato di migrazione.

Il `return` dell'effetto restituisce una funzione di pulizia:

```
return () => {  
  window.removeEventListener('online', goOnline)  
  ...  
}
```

React la esegue quando `OfflineBanner` viene rimosso. Usa gli stessi riferimenti `goOnline`, `goOffline` e `showMigrationNotice`, perché il browser può rimuovere soltanto la callback esatta registrata prima. Senza pulizia, rimontare il componente accumulerebbe ascoltatori duplicati.

Il secondo argomento:

```
}, [])
```

è l'array delle dipendenze. Essendo vuoto, l'effetto viene collegato al montaggio e pulito allo smontaggio; non viene registrato di nuovo a ogni render.

Mutation in attesa

```
const waiting = useMutationState({  
  filters: { status: 'pending' },  
  select: (mutation) => mutation.state.isPaused,  
}).filter(Boolean).length
```

Una mutation è una scrittura, per esempio inviare un messaggio o registrare un set.

1. `useMutationState({...})` chiede al `QueryClient` le mutation globali;
2. `filters: { status: 'pending' }` conserva quelle non ancora concluse;
3. per ciascuna, `select` esegue la funzione `(mutation) => mutation.state.isPaused`;
4. quella funzione riceve una mutation e restituisce il booleano `isPaused`;
5. il risultato di `useMutationState` è quindi un array come `[true, false, true]`;
6. `.filter(Boolean)` rimuove i valori falsi e produce `[true, true]`;
7. `.length` vale 2.

`waiting` è dunque il numero di scritture realmente ferme in attesa della rete. Filtrare soltanto `pending` conterebbe anche normali richieste online che stanno impiegando pochi millisecondi, facendo lampeggiare inutilmente il banner.

Mutation fallite

```
const failedIds = useMutationState({  
  filters: { status: 'error' },
```



```
select: (mutation) => mutation.mutationId,
})
```

Questa volta seleziona mutation definitivamente fallite ed estrae da ciascuna il suo identificatore. Il risultato può essere `[12, 18]`.

```
const unacknowledged = failedIds.filter((id) => !dismissed.has(id))
```

`filter` esegue la callback per ogni ID:

- `dismissed.has(id)` controlla se è già stato chiuso;
- `!` nega il risultato;
- restano soltanto gli errori non ancora riconosciuti dall'utente.

Variabile `banner`

`banner` contiene JSX oppure `null`:

```
se non online OPPURE esistono mutation in attesa → mostra Box
altrimenti → null
```

```
!online || waiting > 0 ? ( ...Box... ) : null
```

- `!online`: vero quando `online` è falso;
- `||`: OR, basta che una delle due condizioni sia vera;
- `? valoreA : valoreB`: operatore ternario;
- `null`: React non disegna niente.

Il `Box` usa `role="status"`: uno screen reader può annunciare il cambiamento senza considerarlo un errore bloccante. `sx` definisce disposizione flex, centratura, spazi, colori e padding. `CloudOffIcon` `aria-hidden` è decorativa perché il testo comunica già il significato.

Il testo usa un ternario annidato:

- online ma sincronizzazione in corso;
- offline con modifiche da sincronizzare;
- offline senza modifiche in attesa.

Quindi il banner non afferma sempre la stessa cosa: distingue assenza rete da sincronizzazione in corso dopo il ritorno online.

Return

`<>...</>` è un **Fragment**: raggruppa più elementi senza aggiungere un `div`.

Contiene:

- `{banner}` : il Box di stato oppure niente;
- primo Snackbar: errori definitivi delle mutation;
- secondo Snackbar: operazioni offline scartate dopo aggiornamento dell'app.

Nel primo Snackbar:

```
open={unacknowledged.length > 0}
```

`open` riceve vero se esiste almeno un errore. Alla chiusura:

```
setDismissed(new Set(failedIds))
```

crea un nuovo Set contenente tutti gli ID falliti attuali. Non c'è chiusura automatica: una scrittura persa richiede conferma esplicita.

Nel secondo:

```
open={Boolean(migrationNotice)}
```

`Boolean(...)` converte `null` in falso e un oggetto in vero. Alla chiusura vengono eseguite due operazioni:

1. `clearCacheMigrationNotice()` rimuove l'avviso persistente da `localStorage`;
2. `setMigrationNotice(null)` aggiorna immediatamente React e nasconde lo Snackbar.

```
migrationNotice?.discardedCount ?? 0
```

- `?.` legge `discardedCount` soltanto se l'oggetto esiste;
- `?? 0` usa zero solo se il risultato è `null` o `undefined`.

I quattro risultati visibili

Situazione	Valori principali	Risultato
rete disponibile, nessuna scrittura	<code>online=true</code> , <code>waiting=0</code>	nessun banner
rete assente, nessuna scrittura	<code>online=false</code> , <code>waiting=0</code>	informa che l'app è offline
scritture sospese	<code>waiting>0</code>	indica quante modifiche verranno sincronizzate
scrittura fallita	<code>unacknowledged.length>0</code>	Snackbar rosso persistente
vecchia scrittura scartata dopo update	<code>migrationNotice</code> è un oggetto	Snackbar giallo persistente con il conteggio

Collegamento con `cacheMigration.js`

`OfflineBanner` mostra soltanto l'avviso; la decisione di scartare avviene in `cacheMigration.js` durante il ripristino della cache.

- `QUERY_CACHE_BUSTER` è il nome della versione corrente del formato cache;
- `migratePersistedClient(persisted)` confronta la versione salvata;
- conserva le vecchie mutation esplicitamente compatibili;
- calcola `discardedCount` per le altre;
- `saveMigrationNotice` scrive quel numero in `localStorage` e invia un `CustomEvent`;
- `OfflineBanner` può riceverlo tramite `showMigrationNotice` anche se è già visibile.

In una frase: **TanStack decide quali scritture sono in attesa o fallite; `cacheMigration.js` protegge dagli aggiornamenti incompatibili; `OfflineBanner` traduce questi stati tecnici in messaggi per l'utente.**

Props di tutti i componenti condivisi

Apri uno alla volta con `Ctrl+P`. Leggi prima la firma della funzione.

`AppointmentCard({ appointment, person, to })`

- `appointment`: riga con tipo, stato e orari;
- `person`: profilo da mostrare;
- `to`: URL aperto dalla card.

`BottomNav({ items })`

`items` è un array di voci con label, icona e destinazione. Confronta `location.pathname` per evidenziare la voce attiva.

`ClientCard({ client, to })`

- `client`: profilo membro;
- `to`: dossier da aprire.

`ConfirmDialog(...)`

Riceve `open`, `title`, `description`, testi dei pulsanti, `onCancel`, `onConfirm` e possibili opzioni di disabilitazione. È riutilizzato dove una scelta può perdere dati o eliminare contenuto.

`LiveSessionBar({ sessionName, sessionId, elapsed, paused })`

Mostra la run aperta e costruisce il link con `sessionId`. `elapsed` è il tempo già formattato; `paused` cambia testo/stato.

`MembershipBanner()`

Non riceve props: usa `useAuth` per il profilo e calcola lo stato dell'abbonamento.

`MonthGrid({ cells, selected, onSelect, markers = {} })`

- `cells`: giorni da disegnare, inclusi riempimenti del mese;

- `selected` : data selezionata;
- `onSelect` : callback al click;
- `markers` : informazioni su giorni con appuntamenti.

`PageHeader({ title, subtitle, backTo, backLabel, action })`

Titolo e sottotitolo; destinazione e nome accessibile del pulsante indietro; eventuale azione a destra.

`PasswordField({ visibilityLabel = 'password', ...props })`

`...props` raccoglie tutte le altre props e le passa a `TextField`. Il componente aggiunge lo stato mostra/nascondi e un pulsante icona.

`SessionCard({ session, to, status, run, onDelete = null })`

Dati sessione, destinazione, stato derivato, possibile run e callback eliminazione opzionale.

`TopHeader({ profileHref, notificationCount = 0, notificationHref, scanHref })`

Link profilo, conteggio/URL notifiche e URL scanner opzionale per il professionista.

`WeekStrip({ days, selected, onSelect, markers = {} })`

Array giorni, data selezionata, callback e marker facoltativi.

Material UI: come leggere `sx`

`sx` riceve un oggetto di stile. Abbreviazioni comuni:

- `p`, `px`, `py` : padding totale/orizzontale/verticale;
- `m`, `mt`, `mb` : margin;
- `bgcolor` : colore sfondo;
- `color` : colore testo;
- `display` e `flexDirection` : disposizione;
- `alignItems`, `justifyContent` : allineamento;
- `flexGrow: 1` : usa lo spazio libero;
- `minWidth: 0` : permette a un figlio flex di restringersi;
- `borderRadius` : arrotondamento;
- `position: fixed` : elemento legato allo schermo.

Valori come `primary.main` non sono colori CSS letterali: MUI li cerca nel tema.

Devi saper dire

I componenti comuni ricevono props e centralizzano elementi ripetuti. Material UI fornisce struttura e controlli, mentre `sx` e il tema definiscono lo stile. Gli hook collegano i componenti a stato, eventi e dati; ogni effetto con listener restituisce la propria pulizia.

Lezione 04 — Router, layout e controllo dei ruoli

Prima di aprire il codice: che problema risolve il router?

`index.html` contiene un solo punto `root`. TrainHub non possiede un file HTML separato per Home, Workout e Calendar. React Router osserva l'indirizzo e decide quale componente inserire nel punto `<Outlet />` del layout.

```
URL /m/workout
→ il router trova il ramo /m
→ AppLayout verifica che il profilo sia member
→ nel suo Outlet inserisce WorkoutPlanScreen
```

Il **path** è il modello dell'indirizzo; la **rotta** è l'oggetto che collega path e componente; la **navigazione** cambia URL; il **redirect** sostituisce una destinazione con un'altra; il **layout** è la cornice che rimane mentre cambia la pagina interna.

Apri `src/routes/navItems.js`

Il file esporta due array.

Ogni oggetto contiene:

- `label` : testo in navigazione;
- `to` : indirizzo;
- `icon` : componente icona.

`memberNav` contiene Home, Workout, Nutrition e My Trainer. `professionalNav` contiene Home, Clients, Calendar e Chat.

Un elemento dell'array ha concettualmente questa forma:

```
{ label: 'Workout', to: '/m/workout', icon: FitnessCenterIcon }
```

`icon` è un riferimento alla funzione componente dell'icona; non viene ancora eseguita. `BottomNav` percorre l'array con `map` e crea una voce visuale per ogni oggetto.

Quando `BottomNav` esegue `items.map`, crea una voce per ogni oggetto.

Apri `src/routes/index.jsx`

Import principali

- `createBrowserRouter` : crea un router che usa gli URL del browser;

- `Navigate` : componente che esegue un redirect;
- `lazy` : rimanda il caricamento di un componente;
- layout, nav e schermate pubbliche.

Perché `lazy`

```
const MemberHomeScreen = lazy(() => import('../features/home/MemberHomeScreen.jsx'))
```

Spiegazione:

- `lazy` riceve una funzione;
- la funzione usa `import(...)` dinamico;
- il file non entra nel primo blocco JavaScript obbligatorio;
- viene scaricato quando la rotta lo richiede.

`() => import(...)` è la funzione passata a `lazy`. React la conserva e la chiama quando quella schermata serve. L'import dinamico restituisce una Promise perché il file può dover essere caricato.

`createBrowserRouter([...])`

Riceve un array di oggetti rotta. Proprietà comuni:

- `path` : parte dell'indirizzo;
- `element` : componente da mostrare;
- `children` : rotte interne;
- `index: true` : rotta predefinita del genitore;
- `errorElement` : cosa mostrare se il routing fallisce.

Rotte pubbliche

Il genitore usa `PublicLayout`. I figli login/forgot/reset vengono mostrati nel suo `<Outlet />`.

`Outlet` significa: "qui inserisci il componente della rotta figlia".

Area membro

Il path `/m` usa:

```
<AppLayout
  navItems={memberNav}
  profileHref="/m/profile"
  requiredRole="member"
/>
```

Props:

- `navItems` : barra membro;
- `profileHref` : destinazione avatar;

- `requiredRole` : ruolo necessario.

I path figli sono relativi. Un figlio `workout` dentro `/m` diventa `/m/workout`.

Parametri

`session/:sessionId` accetta qualunque valore nella posizione di `:sessionId`. React Router lo rende disponibile a `useParams`.

Redirect

```
<Navigate to="/m/workout/builder" replace />
```

- `to` : destinazione;
- `replace` : sostituisce la voce corrente nella cronologia, così "indietro" non rimbalza sul redirect.

Area professionista

Usa lo stesso `AppLayout`, ma passa `professionalNav`, link profilo `/p/profile` e ruolo `professional`.

Questa è la prova che non sono due app: stesso componente, props diverse.

Fallback

La rotta `*` cattura indirizzi non riconosciuti e reindirizza. Le guardie del layout correggono poi l'area in base al ruolo.

Apri `src/layouts/PublicLayout.jsx`

È un componente semplice:

- Box esterno: pagina intera;
- contenitore con larghezza massima;
- BrandLogo;
- Outlet per login/reset.

Non gestisce autenticazione: è soltanto una cornice visiva pubblica.

Apri `src/layouts/AppLayout.jsx`

È il componente che tiene insieme le pagine autenticate.

Firma

```
AppLayout({ navItems, profileHref, requiredRole })
```

Conosci già le tre props dal router.

Hook

- `useAuth` : sessione, utente, profilo, errori e caricamento;
- `useLocation` : indirizzo corrente;
- `useQuery` : notifiche non lette e run membro aperta;
- `useEffect` : subscription e timer;
- `useRef /ResizeObserver`: misura altezza header.

Guardie in ordine

Il componente deve distinguere:

1. autenticazione ancora in caricamento;
2. nessuna sessione → login;
3. errore profilo senza copia disponibile;
4. profilo non ancora disponibile;
5. ruolo sbagliato → area corretta;
6. ruolo corretto → mostra shell.

L'ordine evita di fare redirect prima di sapere chi è l'utente.

Qui “guardia” significa una sequenza di `if` con return anticipato. Se una condizione restituisce Loading, Error o Navigate, la funzione termina e non costruisce la shell sottostante.

Notifiche

La query legge il numero non letto. `refetchInterval: 60_000` significa aggiornamento ogni 60.000 millisecondi, cioè un minuto.

Una subscription Realtime su INSERT invalida il conteggio quando arriva una nuova notifica.

Run aperta

La query è abilitata solo se il profilo è membro. Il professionista non ha mini-player workout.

Il tempo viene ricalcolato da timestamp. Un intervallo serve solo a ridisegnare il numero; non è la fonte della durata.

ResizeObserver

Osserva l'altezza reale dell'header. Il contenuto riceve padding sufficiente per non finire sotto elementi fixed, anche quando compaiono banner.

Gli elementi `position: fixed` non occupano spazio nel flusso normale. `ResizeObserver` è un oggetto del browser che chiama una funzione quando cambia la dimensione dell'header. La misura viene scritta in una variabile CSS usata come spazio superiore del contenuto.

Struttura visiva

```
header fixed
  TopHeader
  OfflineBanner
```


MembershipBanner membro

main

Outlet della schermata

LiveSessionBar membro, se run aperta

BottomNav fixed

Sicurezza UI e sicurezza reale

`requiredRole` impedisce di usare normalmente l'area sbagliata. Ma è codice nel browser. La protezione definitiva è ripetuta da RLS e RPC nel database.

Esercizio

1. In `routes/index.jsx` cerca `requiredRole` e conta i due usi.
2. Cerca `:clientId` e osserva tutte le pagine costruite attorno allo stesso cliente.
3. In `AppLayout` trova `<Outlet />`: è il punto in cui cambia la schermata.
4. Trova `enabled` nella query della run e spiega perché dipende dal ruolo.

Devi saper dire

Il router è un albero di oggetti path-element. I layout contengono Outlet, dove viene inserita la rotta figlia. `AppLayout` riceve navigazione, link profilo e ruolo; carica servizi comuni e applica guardie. La protezione di ruolo nel frontend migliora il flusso, mentre RLS/RPC proteggono davvero i dati.

Lezione 05 — Autenticazione completa

Prima di aprire il codice: quattro oggetti diversi

Non confondere:

- **credenziali**: email e password inviate al login;
- **sessione**: prova temporanea restituita da Supabase dopo il login;
- **user**: identità Auth, con UUID ed email;
- **profile**: riga `profiles`, con nome, ruolo, coach e abbonamento.

La password non viene salvata nello stato globale di TrainHub. Supabase Auth la verifica e restituisce una sessione. Le richieste successive includono un token della sessione; PostgreSQL ne ricava `auth.uid()`.

Autenticazione significa stabilire chi sei. Autorizzazione significa decidere cosa puoi fare. `AuthProvider` rende nota l'identità all'interfaccia; RLS e RPC applicano i permessi sul server.

1. Apri `src/features/auth/AuthContext.js`

Il file crea un context React. Un context è un canale attraverso l'albero dei componenti.

Senza context, `App` dovrebbe passare l'utente a ogni componente intermedio. Con il provider, qualunque discendente usa `useAuth`.

Il valore iniziale è normalmente `null`: fuori dal provider non esiste autenticazione disponibile.

`createContext(null)` crea il canale, non crea una sessione. Il valore reale viene inserito più avanti da `<AuthContext.Provider value={...}>`.

2. Apri `src/features/auth/useAuth.js`

La funzione chiama `useContext(AuthContext)`.

- se riceve il valore, lo restituisce;
- se il componente è fuori dal provider, genera un errore chiaro.

È un **custom hook**: una funzione del progetto che combina hook e ha nome `use...`.

3. Apri `src/features/auth/AuthProvider.jsx`

Props

`AuthProvider({ children })` riceve `children`: tutto ciò che `App` ha annidato tra apertura e chiusura. In questo caso è `RouterProvider`.

Stato

Il provider conserva:

- `session` : sessione Supabase oppure null;
- `profile` : riga profiles oppure copia locale;
- `profileError` : errore dell'ultima lettura;
- `loading` : non abbiamo ancora deciso lo stato iniziale.

`user` si ricava dalla sessione: non serve conservarlo due volte.

`loading=true` significa “il controllo iniziale non è finito”, non “esiste un utente”. Dopo il controllo, `session=null` significa “nessun utente autenticato”. Distinguere questi due casi evita un redirect prematuro al login.

Profilo locale

Una chiave `localStorage` associa una copia minima all'utente. Serve per routing e shell quando la rete manca. Non sostituisce RLS e non dà autorizzazioni al database.

Le funzioni interne leggono, scrivono e rimuovono questa copia dentro `try/catch`, perché `localStorage` può fallire in modalità privata.

`localStorage` conserva solo stringhe. `JSON.stringify` trasforma l'oggetto profilo in testo; `JSON.parse` lo ricostruisce. Questa copia può aiutare la UI offline, ma non è firmata e non dà alcun permesso al database.

`fetchProfile(userId)`

Parametro `userId` : UUID Auth.

Esegue una select su `profiles` con `.eq('id', userId).single()`. Aggiorna stato e `localStorage`. Se fallisce ma esiste una copia locale, la conserva per l'interfaccia; registra comunque l'errore.

`fetchProfile` è asincrona: `userId` è il parametro; `session.user.id` è l'argomento reale. `.single()` richiede una sola riga. `setProfile(data)` cambia lo stato e provoca il nuovo render di chi usa `useAuth()`.

Effetto iniziale

1. chiama `supabase.auth.getSession()`;
2. controlla se il componente è ancora montato;
3. salva la sessione;
4. se esiste utente, carica profilo;
5. conclude loading;
6. si iscrive a `onAuthStateChange`;
7. alla pulizia annulla subscription e ignora risposte tardive.

Il flag locale evita che una Promise terminata dopo lo smontaggio aggiorni un componente inesistente.

Eventi visibility e online

Quando la pagina torna visibile o la rete torna online, il provider può rileggere il profilo. Serve a riconciliare cambiamenti avvenuti altrove.

Evento profilo aggiornato

`PROFILE_UPDATED_EVENT` è un evento browser personalizzato. Dopo avatar o assegnazione, il provider sa di dover aggiornare la propria copia senza creare un legame diretto tra moduli.

`signOut`

Ordine concettuale:

1. prova a disabilitare la push locale;
2. chiede sign-out a Supabase;
3. rimuove profilo salvato;
4. cancella query e mutation dalla memoria;
5. cancella la cache persistita IndexedDB;
6. azzera lo stato Auth.

La pulizia evita una perdita tra due utenti sullo stesso dispositivo.

Valore del provider

L'oggetto fornito contiene session, user, profile, profileError, loading, signOut e funzioni necessarie. `children` viene renderizzato dentro `AuthContext.Provider`.

4. Apri `LoginScreen.jsx`

Stato del form

Email, password, loading e messaggio di errore sono state locali.

`useLocation`

Può contenere la pagina da cui una guardia ha mandato al login. Dopo l'accesso si prova a tornare lì se è coerente col ruolo.

Submit

L'evento submit viene ricevuto dal form. `event.preventDefault()` impedisce al browser di ricaricare la pagina.

Chiama:

```
supabase.auth.signInWithPassword({ email, password })
```

L'oggetto contiene i due argomenti richiesti da Supabase.

Poi legge il profilo e naviga:

- member → `/m`;
- professional → `/p`.

`navigate(path, { replace: true })` evita che il pulsante indietro torni al form già superato.

5. Apri `ForgotPasswordScreen.jsx`

Chiama `resetPasswordForEmail(email, { redirectTo })`.

- `email` : account;
- `redirectTo` : URL della schermata reset sulla stessa origine.

La risposta mostrata è volutamente generica. Rivelare “questa email non esiste” permetterebbe di enumerare account.

6. Apri `ResetPasswordScreen.jsx`

PKCE in parole semplici

Il link contiene un codice breve. L'app lo scambia con Supabase per ottenere la sessione di recupero, senza mettere una chiave segreta nel browser.

Protezione StrictMode

Una `ref` ricorda se il codice è già stato scambiato. In sviluppo StrictMode può rieseguire l'effetto; un codice monouso non deve essere consumato due volte.

Form password

Due state contengono password e conferma. Il client controlla corrispondenza e minimo 8 caratteri per feedback rapido. Il server applica le regole definitive.

La mutation Auth è:

```
supabase.auth.updateUser({ password })
```

Al successo naviga nell'area membro prevista dal flusso corrente.

7. Apri `src/lib/supabase.js`

Variabili

`import.meta.env.VITE_SUPABASE_URL` e `VITE_SUPABASE_ANON_KEY` sono sostituite da Vite nel build.

Se mancano, il file genera un errore subito: senza backend l'app non deve fallire in modo misterioso più avanti.

`createClient(url, anonKey, options)`

Parametri:

- `url` : progetto Supabase;
- `anonKey` : chiave pubblica;
- `options.auth.persistSession` : conserva sessione;
- `autoRefreshToken` : rinnova token;
- `flowType` : 'pkce' : flusso recupero;
- `detectSessionInUrl` : false : il reset scambia esplicitamente il codice.

Esercizio

1. Segui `AuthProvider` → `AuthContext.Provider` → `useAuth` → una schermata.
2. Trova ogni chiamata `supabase.auth` nei tre screen.
3. Spiega perché il profilo locale non può aggirare RLS.
4. Spiega perché la cache viene cancellata al logout.

Devi saper dire

Supabase Auth produce una sessione; AuthProvider la osserva e carica il profilo, che contiene il ruolo. Il context rende questi dati disponibili con useAuth. Il profilo locale aiuta soltanto l'interfaccia offline; il database continua a verificare JWT e RLS. Il logout elimina anche cache e push del dispositivo.

Lezione 06 — Dati, cache e offline

Prima di aprire il codice: lettura e scrittura

Una **query** chiede dati senza modificarli: “qual è il piano?”. Una **mutation** tenta una modifica: “registra questo set”. TanStack Query non è il database: coordina queste operazioni nel browser.

`useQuery` restituisce `data`, `isPending`, `isError`, `error` e `refetch()`. `useMutation` restituisce `mutate(variables)`, `mutateAsync(variables)`, `isPending`, `isPaused`, `isError` ed `error`. Le `variables` sono i dati della singola esecuzione, per esempio `{ id, runId, reps }`.

```
query/mutation nella schermata
→ funzione in src/data
→ richiesta Supabase
→ controllo server
→ risposta
→ aggiornamento cache
→ nuovo render React
```

Apri `src/lib/queryClient.js`

ONE_WEEK

È il numero di millisecondi in sette giorni. Viene usato sia per garbage collection sia per età massima della copia persistita.

`new QueryClient({ defaultOptions })`

Crea il gestore TanStack globale.

`new QueryClient(...)` costruisce un oggetto che conserva cache, mutation e configurazioni. Il progetto ne esporta una sola istanza e `App.jsx` la passa al Provider: tutte le schermate vedono quindi lo stesso archivio.

Query:

- `staleTime: 30_000`: per 30 secondi il dato è considerato fresco;
- `gcTime`: quanto può restare inutilizzato in cache;
- `networkMode: 'offlineFirst'`: usa dati/cache e gestisce assenza rete;
- `retry`: ritenta letture solo se il browser è online.

Mutation:

- `networkMode: 'offlineFirst'`;
- retry secondo la configurazione del progetto.

IndexedDB

Il persistor usa `idb-keyval` per scrivere un oggetto serializzato nel database locale del browser.

IndexedDB vive sul dispositivo e non è PostgreSQL. Conserva una copia per riapertura/offline; PostgreSQL rimane la fonte remota condivisa.

`createMigratingPersister` lo avvolge e controlla la versione di sicurezza.

Apri `queryKeys.js`

È un oggetto di funzioni. Esempio concettuale:

```
openRun: (memberId) => ['runs', 'open', memberId]
```

Parametro `memberId`: rende diversa la cache di Daniel da quella di Sofia.

`queryPrefixes` contiene inizi di chiavi utili per invalidare famiglie intere.

Apri `mutationKeys.js`

Contiene array stabili per ogni scrittura. Non sono funzioni server: sono nomi usati da TanStack per ritrovare la configurazione.

Apri `src/data/mutations.js`

È il registro centrale.

`registerMutationDefaults(queryClient)`

Parametro `queryClient`: il gestore su cui registrare i comportamenti.

Per ogni chiave chiama `setMutationDefaults` con:

- `mutationFn`: vera funzione data;
- eventuale `scope`: serializzazione;
- `onMutate`: prima della richiesta, spesso ottimismo;
- `onSuccess`: dopo risposta positiva;
- `onError`: rollback o gestione;
- `onSettled`: eseguito in ogni caso;
- invalidazioni della cache.

Ordine concreto:

```
mutate(variables)
→ onMutate modifica provvisoriamente la cache e conserva uno snapshot
→ mutationFn invia la richiesta
→ onSuccess conferma, oppure onError ripristina lo snapshot
→ onSettled viene eseguito comunque e invalida i dati collegati
```

Invalidare significa marcare una query come potenzialmente vecchia e richiederne una nuova versione quando possibile; non equivale necessariamente a svuotare subito la schermata.

Esempio log set

La mutation chiama `logSet`. Al successo/in ogni caso invalida i log e le run interessate.

`ExerciseDetailScreen` aggiunge anche un ottimismo specifico perché possiede il contesto visivo.

Esempio start run

Scrivi subito una run ottimistica nella query `openRun(memberId)`. Se il server rifiuta, ripristina lo snapshot precedente.

Esempio send message

Prima dell'invio:

1. annulla refetch concorrenti;
2. legge la vecchia lista;
3. aggiunge il messaggio con ID client;
4. restituisce lo snapshot.

All'errore rimette la vecchia lista. Al successo sostituisce/riconcilia e aggiorna inbox.

Scope

Le operazioni della stessa sequenza condividono uno scope. TanStack le esegue in ordine. Serve, per esempio, ad abbandonare una run prima di iniziarne una nuova.

Avatar

Upload e delete immagine usano `networkMode: 'always'`: devono fallire subito se non possono partire, perché un Blob non viene persistito come normale JSON.

Apri `cacheMigration.js`

`QUERY_CACHE_BUSTER = 'trainhub-security-v4'` è la versione della forma persistita.

`migratePersistedClient(persisted)`:

- riceve la cache salvata;
- controlla mutation legacy;
- conserva solo chiavi compatibili;
- conta quelle scartate;
- produce un avviso leggibile da OfflineBanner.

Tutte le funzioni data e i loro parametri

Apri i file uno alla volta. Questa è la mappa da tenere accanto.

`appointments.js`

- `fetchAppointmentsOnDay(memberId, dayISO)`: appuntamenti membro di quel giorno.
- `fetchAgendaOnDay(proId, dayISO)`: agenda pro del giorno.
- `fetchAppointmentsInRange(proId, fromISO, toISO)`: intervallo pro.

- `fetchAppointment(appointmentId)` : singola riga con profili.
- `createAppointment({ id, memberId, proId, kind, startsAt, endsAt, notes })` : RPC atomica.
- `fetchMemberAppointmentsInRange(memberId, fromISO, toISO)` : storico membro.
- `setAppointmentStatus({ appointmentId, expectedStatus, status })` : compare-and-set.

`dayISO` è `YYYY-MM-DD` ; i confini locali vengono convertiti in timestamp UTC per la query.

availability.js

- `fetchAvailability(proId)` ;
- `addAvailability({ id, proId, weekday, startsAt, endsAt })` ;
- `deleteAvailability({ availabilityId })` .

chat.js

- `ensureThread({ memberId, proId })` ;
- `fetchMemberThread(memberId, proId)` ;
- `fetchThread(threadId)` ;
- `fetchThreads(proId)` ;
- `fetchMessages(threadId)` ;
- `sendMessage({ id, threadId, senderId, body })` ;
- `markThreadRead({ threadId, readerId, readAt, throughCreatedAt })` .

checkin.js

- `generateBadgeCode()` : nessun parametro, restituisce 8 caratteri;
- `mintCheckinToken({ memberId, token })` ;
- `redeemCheckinToken(token)` .

clients.js

- `fetchClients(proId)` ;
- `fetchClient(clientId)` ;
- `setSubscriptionStatus({ memberId, status })` .

notifications.js

- `fetchNotifications(userId)` ;
- `fetchUnreadNotificationCount(userId)` ;
- `markNotificationRead({ id })` ;
- `markNotificationsRead({ url })` ;
- `deleteNotification({ id })` ;
- funzioni `...ByIds({ ids })` per array selezionato;
- funzioni `...All({ before })` per righe create prima del confine.

Il parametro `before` evita che una notifica arrivata dopo il click venga eliminata come se l'utente l'avesse già vista.

nutrition.js

- `fetchNutritionPlan(memberId);`
- `createNutritionPlan({ id, memberId, replacesPlanId, name, kcalTarget, proteinG, carbsG, fatG, notes, days });`.

profile.js

- `fetchProfessionals();`
- `chooseProfessional({ memberId, proId });`
- `uploadAvatar({ userId, blob });`
- `clearAvatar({ userId });`.

push.js

- `savePushSubscription({ userId, subscription })` estrae endpoint e chiavi browser;
- `deletePushSubscription(endpoint)`.

rewards.js

- `fetchRewards(memberId)`.

runs.js

- `fetchOpenRun(memberId);`
- `fetchRun(runId);`
- `fetchRunsSince(memberId, sinceISO);`
- `fetchRunLogs(runId);`
- `startRun({ id, sessionId, startedAt });`
- `pauseRun({ id, expectedPausedTotalMs, pausedAt });`
- `resumeRun({ id, expectedPausedAt, resumedAt });`
- `endRun({ id, endedAt, outcome });`
- `saveRunNote({ id, note });`.

workouts.js

- `fetchActivePlan(memberId);`
- `fetchSession(sessionId);`
- `fetchSessionExercise(sessionExerciseId);`
- `logSet({ id, runId, sessionExerciseId, setNumber, reps, weight, performedAt });`
- `fetchExerciseCatalogue();`
- `createPlan({ id, memberId, replacesPlanId, name, goal, level, weeks, sessions });`.

Regola per leggere qualunque funzione data

1. identifica parametri;
2. trova `.from` o `.rpc`;
3. leggi select/insert/update e filtri;

4. trova gestione `error` ;
5. osserva la forma restituita.

Devi saper dire

Il data layer espone funzioni con parametri del dominio. TanStack le associa a chiavi, persiste le operazioni compatibili e invalida le cache collegate. Gli aggiornamenti ottimistici conservano snapshot per rollback; ID stabili e vincoli rendono i retry idempotenti.

Lezione 07 — Tutto il codice workout

Segui questa lezione con due editor affiancati. Apri ogni file quando viene nominato.

Prima di aprire il codice: i livelli del workout

<code>workout_plan</code>	programma generale di un membro
<code>workout_session</code>	scheda ripetibile, per esempio Leg Day
<code>session_exercise</code>	esercizio nella scheda con serie/reps/peso/recupero prescritti
<code>workout_run</code>	singolo tentativo datato di eseguire la scheda
<code>set_log</code>	serie realmente registrata durante quel tentativo

“Squat” nel catalogo è un esercizio generale. “Squat, 3×10, 40 kg” dentro Leg Day è una prescrizione. L’allenamento di martedì è una run. Ognuna delle tre serie svolte è un set log. Questa separazione permette di ripetere Leg Day senza sovrascrivere la cronologia.

```
WorkoutPlanScreen → SessionDetailScreen → startRun  
→ LiveSessionScreen → ExerciseDetailScreen → logSet  
→ endRun → calcolo server → SessionSummaryScreen
```

1. Apri `src/features/workout/WorkoutPlanScreen.jsx`

Dati letti

- `useAuth()` fornisce `user` e profilo;
- `fetchActivePlan(user.id)` legge piano e sessioni;
- query delle run recenti serve a derivare lo stato settimanale;
- mutation notifiche segna letti gli avvisi che portano a questa pagina.

La `queryKey` contiene l’ID dell’utente per non mischiare piani.

Stati della pagina

- pending senza dati → `LoadingState`;
- error senza dati → `ErrorState`;
- nessun piano → `EmptyState` con scelta professionista o builder personale;
- piano esistente → informazioni e lista di `SessionCard`.

I return anticipati impediscono di usare `data` prima che esista. `undefined` significa che nessuna risposta utilizzabile è mai arrivata; una vecchia copia presente può invece essere mostrata durante un errore di aggiornamento.

Stato sessione

Per ogni sessione chiama le funzioni di `week.js` e `status.js`. Non legge ciecamente `workout_sessions.status`, perché una sessione è ripetibile ogni settimana.

2. Apri `src/features/workout/SessionDetailScreen.jsx`

useParams

```
const { sessionId } = useParams()
```

Estrae l'ID dall'URL. È una stringa.

Query

- `fetchSession(sessionId)`: sessione, esercizi e prescrizione;
- `fetchRunsSince(user.id, ...)`: storico necessario allo stato;
- `fetchOpenRun(user.id)`: eventuale run aperta;
- `fetchRunLogs(openRun.id)`: log se la run aperta appartiene a questa sessione.

Le query con `enabled: Boolean(id)` partono soltanto quando l'ID esiste.

Mutation start

`useMutation({ mutationKey: mutationKeys.startRun })` recupera il default registrato.

La chiamata passa:

- `id`: UUID generato ora;
- `memberId`: per aggiornare la cache corretta;
- `sessionId`: sessione selezionata;
- `startedAt`: timestamp ISO.

Se non ci sono esercizi, il pulsante non deve creare una run vuota.

`startedAt` nasce nel telefono al momento del gesto: se la scrittura aspetta la rete, il server riceve comunque l'ora reale di inizio. L'UUID `id` rende il replay la stessa run invece di crearne una seconda.

Run già aperta

Se la run è della stessa sessione, il pulsante riprende il live. Se appartiene a un'altra sessione, apre `OpenRunSheet`.

3. Apri `OpenRunSheet.jsx`

Props principali:

- `open`: visibilità;
- dati della run/sessione aperta;
- `onResume`: torna alla run;

- `onAbandonAndStart` : chiude `abandoned` e avvia la nuova;
- `onClose` : chiude il foglio;
- stati `pending/error` per disabilitare controlli.

È presentazione; l'ordine reale delle mutation resta nel chiamante e nello scope `TanStack`.

4. Apri `LiveSessionScreen.jsx`

Query

- run aperta del membro;
- dettaglio della sessione dell'URL;
- log della run;
- run recenti per reward/stato.

La schermata verifica che la run aperta corrisponda alla sessione dell'URL. Un URL manuale non deve mostrare come live una run diversa.

`useLiveSession`

Riceve run e membro e restituisce:

- `elapsed` calcolato;
- `paused`;
- funzioni `pause/resume`;
- stati delle mutation.

Il custom hook non disegna JSX. Riunisce dati e operazioni e restituisce un oggetto alla schermata. Quando aggiorna lo state `now`, React ricalcola la durata e ridisegna il numero.

Lista esercizi

Per ogni `session_exercise`, filtra i log con lo stesso ID e calcola quante serie valide risultano. La card porta a `/m/workout/exercise/:id`.

Chiusura

`EndRunSheet` raccoglie la scelta. La mutation end passa:

- `id` : run;
- `memberId` : cache;
- `endedAt` : timestamp;
- `outcome` : richiesta normale o `abandoned`.

Il server può trasformare la richiesta normale in `completed/partial` in base ai log.

La navigazione al summary può avvenire subito anche offline: l'ID è già noto e l'operazione resta persistita.

5. Apri `useLiveSession.js`

È un custom hook, non una schermata.

Parametri

Riceve la run e gli ID necessari. Le mutation pause/resume usano la stessa famiglia serializzata.

Timer

Uno state `now` viene aggiornato da un intervallo per provocare il render. La durata viene calcolata da `timer.js`; l'intervallo non è la fonte del tempo.

Pausa

Passa `expectedPausedTotalMs`: il valore che il client ha letto. Il server rifiuta se la run è cambiata diversamente.

Ripresa

Passa `expectedPausedAt`: identifica esattamente la pausa che vuole chiudere.

6. Apri `timer.js`

È logica pura.

Funzione durata

Input:

- `started_at` obbligatorio;
- `ended_at` oppure momento corrente;
- `paused_at` se pausa aperta;
- `paused_total_ms` già accumulato.

Calcolo:

```
fine effettiva - inizio - pause chiuse - pausa ancora aperta
```

Il risultato non scende sotto zero.

`formatElapsed(ms)`

Converte millisecondi in un testo con ore/minuti/secondi. Non modifica la run.

7. Apri `ExerciseDetailScreen.jsx`

È il file più importante da leggere lentamente.

Parametro URL

`sessionExerciseId` identifica **la prescrizione nella sessione**, non il catalogo generale.

Query

- dettaglio prescrizione;

- run aperta dell'utente;
- log di quella run;
- sessione completa, per conoscere ordine e prossimo esercizio.

Condizione di log

Il form è attivo soltanto se esiste una run aperta che appartiene alla stessa sessione. Consultare un esercizio fuori dalla run non autorizza a registrare set casuali.

Stato del form

- reps;
- weight;
- eventuali errori;
- dialog/timer;
- prossimo numero serie, derivato dai log esistenti.

Il peso può essere nullo per esercizi a corpo libero. Stringa vuota del campo viene tradotta in `null`, non in zero inventato.

Mutation log

Parametri:

- `id` : UUID nuovo;
- `runId` ;
- `sessionExerciseId` ;
- `setNumber` ;
- `reps` convertito in numero;
- `weight` numero o null;
- `performedAt` ISO;
- `memberId` per cache/invalidazione.

Ottimismo locale

`onMutate` :

1. annulla query log concorrenti;
2. salva la vecchia lista come `previous` ;
3. aggiunge una riga con gli stessi campi;
4. restituisce il contesto.

`onError` usa il contesto per rimettere `previous` . `onSettled` invalida e riconcilia.

La riga ottimistica è una previsione nella cache locale, non ancora prova del salvataggio PostgreSQL. Se la mutation è sospesa resta visibile; se fallisce, il rollback la rimuove e `OfflineBanner` avvisa l'utente.

Prossimo set/esercizio

Dopo il log calcola se esistono serie target mancanti. Se l'esercizio è completo, può suggerire il prossimo esercizio; se l'intera sessione è completa, torna al live per la chiusura.

Contenuto informativo

Mostra nome, gruppo muscolare, attrezzatura, istruzioni e note della prescrizione. Media vengono mostrate soltanto se realmente presenti.

8. Apri `RestTimer.jsx`

Prop

`seconds` : durata prescritta.

Costanti

- chiave `localStorage` `mute`;
- minimo 10;
- massimo 600.

State

- `custom` : override del membro oppure null;
- `target` : istante fine oppure null;
- `remaining` : secondi mostrati;
- `muted` : preferenza;
- `editing` : testo del campo manuale oppure null.

Ref

- `audio` : elemento audio reale;
- `rung` : impedisce due segnali allo zero.

Valori derivati

- `running` : target esiste;
- `base` : `custom`, altrimenti `prop seconds`;
- `shown` : `remaining` se attivo, altrimenti `base`.

Effetto intervallo

Ogni 250 ms calcola `ceil((target-Date.now())/1000)` . Quando arriva a zero suona una volta se non `muted`.

La funzione di cleanup chiama `clearInterval` .

Funzioni

- `toggleMute()` : cambia stato e `localStorage`;
- `start()` : sblocca audio dal gesto, imposta `remaining` e `target`;

- `stop()` : azzera target;
- `adjust(delta)` : aggiunge/toglie secondi e applica clamp;
- `commitEdit()` : converte input e lo limita all'intervallo.

Limite iOS

Il codice prova a usare `navigator.audioSession.type='transient'` dove disponibile per abbassare temporaneamente altra musica. Non può garantire suono in background/telefono bloccato.

9. Apri `EndRunSheet.jsx`

Props contengono apertura, numeri di completamento, callback normal/abandon/cancel e stato mutation. Mostra all'utente la conseguenza della chiusura; non calcola il valore autorevole.

10. Apri `SessionSummaryScreen.jsx`

Legge run, log e sessione. Usa `summary.js` per preparare:

- serie valide/totali;
- ripetizioni;
- volume;
- punti previsti/confermati;
- stato reward.

La nota usa `saveRunNote`. È opzionale e finisce in `workout_runs.note`.

`CongratsDialog` riceve apertura, esito/punti e callback di chiusura; è feedback, non calcolo.

11. Apri `status.js`

Funzioni pure:

- associano status a label/colore;
- calcolano progresso dell'esercizio dai log;
- considerano completa una prescrizione quando raggiunge target set;
- non lasciano che set extra superino il target.

12. Apri `summary.js`

`POINTS` contiene workout 30 e checkin 10.

Funzioni:

- raggruppano log per esercizio;
- calcolano serie valide;
- sommano reps;
- calcolano volume `reps × weight` se il peso esiste;
- stimano punti con la stessa formula server;
- calcolano progresso verso le soglie reward.

Il server resta l'autorità. Questa copia client serve a presentare subito risultati coerenti.

13. Apri `week.js`

- `mondayOf(dayISO)` : lunedì della settimana locale;
- `weekdayOf(dayISO)` : numero 0–6;
- `daysBefore(dayISO, n)` : sottrae giorni senza affidarsi a stringhe UTC sbagliate;
- `earnedOn(runs, dayISO)` : trova reward di quel giorno;
- `runStatusOf(runs, weekStartISO)` : stato corrente della sessione.

Regola: run aperta prevale; poi ultima completed/partial; abandoned non completa; altrimenti to do.

14. Apri `src/data/runs.js` e `workouts.js`

Ora collega le schermate alle funzioni viste nella lezione 06. Cerca ogni `.rpc` e pronuncia:

screen → mutation key → mutation default → funzione data → RPC → tabelle

Esempio:

```
ExerciseDetailScreen
→ logSet
→ log_workout_set_secure
→ set_logs
```

Devi saper dire

Il codice workout separa piano prescritto ed esecuzione. Le schermate leggono sessione, run e log con query; le scritture passano da mutation durabili e RPC. Gli ID nascono sul client, timer e pause usano timestamp, mentre percentuale, outcome e reward vengono ricostruiti dal database.

Lezione 08 — Builder workout e nutrizione

Prima di aprire il codice: builder e transazione

Un builder o wizard è un form lungo diviso in passaggi. La bozza vive nello stato React finché l'utente conferma. Non viene salvato mezzo piano a ogni schermata: l'ultima conferma invia l'intera struttura a una RPC.

La RPC usa una **transazione atomica**: piano, sessioni ed esercizi vengono creati tutti; se una parte non è valida, PostgreSQL annulla tutto. Non resta un piano incompleto.

`CreatePlanFlow` è il genitore che possiede la bozza. `PlanForm`, `SessionForm` e `PlanSummary` ricevono dati e callback. Quando un figlio chiama `onSubmit(data)`, consegna i dati al genitore: non scrive direttamente nel database.

Parte A — Builder workout

Apri `CreatePlanFlow.jsx`

Firma:

```
CreatePlanFlow({ memberId, replacesPlanId = null, onDone, onAbandon })
```

Props:

- `memberId`: destinatario del piano;
- `replacesPlanId`: piano precedente oppure null;
- `onDone`: callback dopo salvataggio;
- `onAbandon`: callback dopo conferma di abbandono.

State

- `step`: `'meta'`, `'session'` o `'summary'`;
- `meta`: oggetto nome/goal/level/weeks;
- `sessions`: array di bozze;
- `editingIndex`: indice della sessione in modifica o null;
- `abandoning`: dialog conferma aperto.

`step` stabilisce quale componente viene restituito. Questo è il significato concreto di “macchina a stati”: solo un insieme preciso di stringhe è ammesso e ogni azione porta a uno stato successivo.

Query catalogo

`fetchExerciseCatalogue` fornisce gli esercizi. Serve soltanto dallo step session, ma può essere preparata dal flow.

Mutation create

Usa `mutationKeys.createPlan`. Alla conferma genera l'ID del piano nel gestore del click, non durante il render.

Scelta del body

La variabile `body` riceve JSX diverso.

- `meta` → `PlanForm`;
- caricamento catalogo → `loading/error`;
- `session` → `SessionForm`;
- `summary` → `PlanSummary`.

È una piccola **macchina a stati**: soltanto uno step è attivo.

Aggiunta/modifica sessione

`editingIndex === null` significa nuova sessione. Altrimenti `map` sostituisce soltanto la sessione con quell'indice. L'ID già presente viene mantenuto.

`map` crea un nuovo array senza alterare quello precedente. React vede il nuovo riferimento e renderizza la bozza aggiornata. Conservare l'ID evita che una semplice modifica venga trattata come un'entità nuova.

Conferma

Payload:

- nuovo `id`;
- `memberId`;
- `replacesPlanId`;
- proprietà di `meta` tramite `...meta`;
- `sessions` trasformate da `buildSessionPayloads`.

`...meta` è spread: copia le proprietà dell'oggetto dentro il nuovo oggetto.

```
meta = { name: 'Summer', weeks: 4 }  
{ id: 'abc', ...meta }  
// { id: 'abc', name: 'Summer', weeks: 4 }
```

Annullamento

Il dialog è sempre esplicito, perché la bozza vive solo nel componente e verrebbe persa.

Apri `PlanForm.jsx`

Firma:

```
PlanForm({ onSubmit, initial = null, submitLabel = 'Continue' })
```

- `onSubmit` : riceve l'oggetto finale;
- `initial` : riempie il form in modifica;
- `submitLabel` : testo pulsante.

State: `name` , `goal` , `level` , `weeks` . `LEVELS` è una costante array. Al submit converte weeks in Number.

I campi hanno `required` , label, value e `onChange` . `event.target.value` è il testo corrente.

Apri `SessionForm.jsx`

Props:

- `catalogue` : esercizi disponibili;
- `onSubmit` ;
- `initial` ;
- `submitLabel` .

State: nome sessione, esercizi scelti e valori temporanei dei campi.

Ogni esercizio draft contiene:

- `exercise/ID`;
- `targetSets` ;
- `targetReps` ;
- `targetWeight` ;
- `restSeconds` ;
- `notes` .

Il form impedisce invio senza nome o esercizi e limita numeri tramite attributi input e conversioni.

Apri `PlanSummary.jsx`

Props includono `meta` , `sessions` , `pending` , `paused` , `error` e callback edit/add/delete/confirm.

Il componente non possiede la bozza: mostra e richiama funzioni del genitore. Questo si chiama **lifting state up**: lo stato comune è nel flow.

Apri `contracts.js`

```
buildSessionExercisePayloads(exercises, createId = createUuid):
```

- riceve array bozza;
- usa un generatore ID sostituibile nei test;
- `map` crea un nuovo oggetto per voce;
- converte camelCase UI in snake_case database;
- assegna `position: index + 1` ;
- mette `null` nel peso mancante e 90 nel recupero mancante.

`buildSessionPayloads` fa lo stesso per le sessioni e chiama la funzione `esercizi`.

Parte B — Nutrizione

Apri `NutritionPlanEditorScreen.jsx`

`clientId` viene dall'URL. Query:

- profilo cliente;
- piano nutrizionale più recente.

Stati possibili:

- nessun piano → flow nuovo;
- piano presente → mostra target, giorni e pasti;
- `replacing=true` → flow con `replacesPlanId`.

L'abbonamento disabilita la creazione, ma la lettura del piano rimane.

Apri `CreateNutritionPlanFlow.jsx`

Firma uguale concettualmente al workout. State:

- `step`: meta/day/summary;
- `meta`;
- `days`;
- `editingIndex`;
- dialog annullamento.

`takenWeekdays` è un `Set` costruito dai giorni diversi da quello in modifica. Viene passato a `DayForm` per disabilitare weekday già assegnati.

Apri `NutritionPlanForm.jsx`

Props: `onSubmit`, `initial`, `submitLabel`.

State:

- `name`;
- `kcal`;
- `protein`;
- `carbs`;
- `fat`;
- `notes`.

La funzione `toNumberOrNull` restituisce null per campo vuoto e numero altrimenti. Evita che “non specificato” diventi zero.

Apri `DayForm.jsx`

Firma:


```
DayForm({ onSubmit, initial = null, submitLabel = 'Save day', takenWeekdays = new Set() })
```

State principale:

- `name` ;
- `weekdays` : Set selezionato;
- `meals` : array.

`WEEKDAYS` converte valori `Date.getDay` in ordine visivo lunedì-domenica.

`MealBuilder({ onAdd })`

Componente interno per creare un pasto. State: nome, ora, kcal, macro, alternative, elementi alimentari e campi temporanei dell'item.

`onAdd(meal)` passa il nuovo oggetto a DayForm. L'ID stabile del pasto verrà mantenuto/generato dal contratto.

Ogni food item contiene normalmente nome e quantità. Non è una riga SQL separata.

Il giorno non può essere salvato senza weekday o pasti.

Apri `NutritionPlanSummary.jsx`

Come PlanSummary workout: riceve dati e callback, non possiede la fonte della bozza. Mostra target, giorni, weekdays e pasti e abilita confirm solo con contenuto valido.

Apri `nutrition/contracts.js`

- `buildMealItemPayloads(items)` : normalizza elementi JSON;
- `buildMealPayloads(meals, createId)` : ID, nome, orario, posizione, macro, alternative e items;
- `buildDayPayloads(days, createId)` : ID, nome, posizione, weekdays e pasti annidati.

Gli ID generati prima della richiesta rendono la creazione ripetibile senza inventare nuove righe.

Apri `MemberNutritionScreen.jsx`

State `selected` : data scelta.

Query: `fetchNutritionPlan(user.id)` .

```
days.find((day) => day.weekdays.includes(weekdayOf(selected)))
```

Traduzione: trova il primo tipo di giorno il cui array weekdays contiene il numero della data selezionata.

La pagina mostra target del piano, WeekStrip e card pasti. Ogni card porta a `/m/nutrition/meal/:mealId` .

Apri `MealDetailScreen.jsx`

`mealId` viene dall'URL. Invece di una query separata, riusa il piano e cerca:

days → tutti i meals → meal con ID uguale

Se manca, mostra “Meal not found”: potrebbe appartenere a un piano sostituito.

`Macro({ label, grams })` è un componente interno. `grams` null produce un trattino/assenza coerente, non zero inventato.

Devi saper dire

I due builder sono macchine a stati che tengono una bozza locale. Form figli ricevono valori/callback, mentre il flow possiede lo stato. I contracts normalizzano ID, posizioni e nomi di campo. La conferma invia un JSON annidato a una RPC atomica; annullare prima non lascia righe parziali.

Lezione 09 — Clienti, abbonamento, calendario e appuntamenti

Prima di aprire il codice: relazioni e stati

Un membro ha `assigned_pro_id`, l'UUID del professionista scelto. Da questo collegamento derivano roster, autorizzazione sui piani, chat e prenotazioni.

La **disponibilità** è una regola ricorrente, per esempio "lunedì 09:00–12:00". L'**appuntamento** è un intervallo preciso con data, partecipanti, tipo e stato.

- `pending` : richiesta del membro non ancora confermata;
- `confirmed` : accettata o creata dal professionista;
- `cancelled` : annullata;
- `done` : svolta.

Il frontend prepara la richiesta. La RPC decide lo stato iniziale usando il vero `auth.uid()` e controlla relazione, ruolo, date e abbonamento.

Apri `features/clients/ClientsScreen.jsx`

State `search` conserva il testo. Query `fetchClients(user.id)` legge il roster.

```
const term = search.trim().toLowerCase()
```

- `trim` : toglie spazi estremi;
- `toLowerCase` : confronto senza maiuscole.

`filter` mantiene i clienti il cui `full_name` contiene `term`. È locale perché il roster è piccolo e così non parte una richiesta a ogni carattere.

`setSearch(event.target.value)` viene chiamata a ogni modifica del campo. Il nuovo render ricalcola `term` e la lista; non modifica il database.

La pagina distingue elenco vuoto da ricerca senza risultati.

Apri `ClientDetailScreen.jsx`

`clientId` arriva da `useParams`. Query:

- cliente;
- piano workout;
- piano nutrizione.

`clientId` è soltanto una stringa UUID nell'URL, non il cliente completo. Le tre query usano quell'ID per leggere tabelle diverse e possono partire in parallelo.

La mutation membership chiama `setSubscriptionStatus`.

Il return costruisce il dossier e link relativi allo stesso clientId. Lo stato dell'abbonamento viene calcolato da `subscriptionStateOf`.

Apri `clients/subscription.js`

È logica pura.

- `daysBetween(fromISO, toISO)` : crea date UTC dalle parti per evitare errori DST;
- `subscriptionStateOf(profile, todayISO)` : restituisce oggetto label/colore;
- `isSubscriptionActive(profile, todayISO)` : booleano usato dalla UI;
- `membershipAction(state)` : propone suspend o reactivate.

Ordine stato:

1. suspended;
2. expired esplicito;
3. data passata;
4. entro 30 giorni → Close to Expiring;
5. active.

Apri `ClientWorkoutScreen.jsx`

Legge cliente e piano. Se non esiste, mostra CreatePlanFlow. Se esiste, mostra dati e sessioni.

`SessionAccordion({ session, statusLabel })` è interno. State `open` controlla espansione. La query dettaglio ha `enabled: open`, quindi esercizi vengono caricati solo alla prima apertura utile.

`replacing` cambia la pagina dal piano esistente al flow di sostituzione.

Apri `ClientAppointmentsScreen.jsx`

Legge cliente e appuntamenti in un intervallo. Usa AppointmentCard con link al dettaglio del calendario.

Apri `trainer/BrowseTrainersScreen.jsx`

Query professionisti e mutation `chooseProfessional`. Le card mostrano specialità e bio. La scelta aggiorna profilo e cache/evento Auth.

Apri `trainer/MyTrainerScreen.jsx`

Legge i professionisti e trova quello il cui ID è `profile.assigned_pro_id`. Mostra contatti e link. Se manca assegnazione, propone browse.

Apri `trainer/MemberAppointmentsScreen.jsx`

State controllano intervallo e apertura BookingSheet. Query membro; mutation notifiche segna lette quelle destinate a questa pagina.

Apri `trainer/BookingSheet.jsx`

Props includono apertura/chiusura e profilo. Query disponibilità del pro assegnato.

State:

- data;
- tipo appuntamento;
- slot/orario;
- note.

`slotToISO(dayISO, timeHHMM, minutes)` produce inizio/fine ISO partendo da giorno locale.

`createAppointment` riceve UUID, coppia, tipo, timestamp e note. Dal membro lo stato server è pending.

La funzione riceve giorno `YYYY-MM-DD`, ora `HH:MM` e durata. Crea una `Date` locale, calcola la fine e usa `toISOString()` per inviare istanti assoluti al database.

Apri `calendar/CalendarScreen.jsx`

State `selected` è il giorno; un secondo state/valore rappresenta il mese visibile. `month.js` genera celle e intervallo. Query `fetchAppointmentsInRange` legge il mese.

Gli appuntamenti vengono raggruppati per giorno locale per creare markers e lista del giorno selezionato.

Apri `calendar/month.js`

Funzioni pure costruiscono:

- primo/ultimo giorno;
- griglia completa a settimane;
- etichette/serie di sette giorni.

Usano componenti `Date` locali per non spostare il giorno a causa di UTC.

Apri `AvailabilityScreen.jsx`

Query delle proprie fasce. Mutation add/delete.

Per aggiungere passa ID, proId, weekday, startsAt ed endsAt. Il database impone fine maggiore dell'inizio.

Apri `NewAppointmentSheet.jsx`

Il professionista sceglie uno dei propri clienti. La stessa RPC `createAppointment` riconosce che il chiamante è il pro e crea stato `confirmed`.

Apri `AppointmentDetailScreen.jsx`

`appointmentId` arriva dall'URL. Query singola. Mutation di stato passa:

- `appointmentId`;
- `expectedStatus` : valore letto;
- `status` : nuovo valore.

Se il record è cambiato nel frattempo, il server rifiuta il compare-and-set.

Apri `agenda/ProfessionalHomeScreen.jsx`

Calcola `todayISO()`, legge `fetchAgendaOnDay(user.id, day)` e mostra `AppointmentCard` ordinate.

Devi saper dire

Il collegamento centrale è `assigned_pro_id`. Le pagine cliente usano `clientId` e `RLS`. Le disponibilità sono ricorrenti, gli appuntamenti sono timestamp assoluti. Il membro crea `pending`, il professionista `confirmed`; gli update di stato includono `expectedStatus` per evitare sovrascritture concorrenti.

Lezione 10 — Chat e aggiornamenti in tempo reale

Obiettivo della lezione

Alla fine devi saper raccontare questo percorso:

1. il professionista apre l'elenco delle conversazioni;
2. sceglie un cliente e l'URL contiene l'identificativo della conversazione;
3. la pagina legge i vecchi messaggi dal database;
4. Supabase Realtime consegna i nuovi messaggi senza ricaricare la pagina;
5. l'invio è ottimistico: il messaggio appare subito e, se manca Internet, resta in attesa;
6. quando il destinatario apre la chat, i messaggi e le notifiche vengono segnati come letti.

Non serve memorizzare ogni parentesi. Devi capire quale responsabilità ha ogni file.

1. Apri l'elenco delle conversazioni

In VS Code premi `Ctrl+P`, scrivi:

```
src/features/chat/ThreadListScreen.jsx
```

Le importazioni

- `useState` conserva ricerca e filtro “solo non letti”.
- `useQuery` esegue una lettura e ne conserva risultato, caricamento ed errore.
- `fetchThreads` contiene la vera richiesta a Supabase.
- `queryKeys` assegna un nome stabile a quel dato nella cache.
- `useAuth` fornisce l'utente autenticato.

La funzione principale

```
export default function ThreadListScreen() {
```

- `function` definisce una funzione.
- `ThreadListScreen` è il suo nome.
- le parentesi sono vuote perché il router non le passa proprietà direttamente;
- `export default` permette al router di importare questa schermata.

```
const { user } = useAuth()
```

`useAuth()` restituisce un oggetto. Le graffe estraggono soltanto la proprietà `user`. Da `user.id` si ricava chi è il professionista collegato.

```
const [search, setSearch] = useState('')
const [unreadOnly, setUnreadOnly] = useState(false)
```

Ogni `useState` restituisce una coppia:

- il primo elemento è il valore attuale;
- il secondo è la funzione che lo cambia;
- `''` significa testo inizialmente vuoto;
- `false` significa filtro inizialmente spento.

```
const threads = useQuery({
  queryKey: queryKeys.threads(user.id),
  queryFn: () => fetchThreads(user.id),
})
```

Il parametro di `useQuery` è un oggetto di configurazione:

- `queryKey`: nome del dato in cache, diverso per ciascun utente;
- `queryFn`: funzione da eseguire per ottenere il dato;
- `() => ...`: funzione freccia senza parametri. Non viene eseguita durante la configurazione: React Query la chiamerà quando serve.

`threads` non è l'array puro. È un oggetto con `data`, `isPending`, `isError`, `error` e `refetch`.

Il filtro usa:

```
const term = search.trim().toLowerCase()
```

`trim()` toglie gli spazi alle estremità; `toLowerCase()` rende il confronto indipendente dalle maiuscole.

`filter(...)` conserva solo le conversazioni che corrispondono al nome e, se richiesto, hanno `unreadCount > 0`. `reduce(...)` somma i non letti di tutte le conversazioni.

La parte dopo `return` è JSX: descrive casella di ricerca, filtro e lista. Non salva nulla nel database.

2. Apri la richiesta che carica le chat

`Ctrl+P` → `src/data/chat.js`.

Questo file separa la schermata dalla banca dati. La schermata decide **quando** leggere o scrivere; questo file stabilisce **come** farlo.

ensureThread({ memberId, proId })

Il parametro è un solo oggetto, destrutturato in due proprietà:

- `memberId` : UUID del membro;
- `proId` : UUID del professionista.

Chiama:

```
supabase.rpc('ensure_assigned_thread', {  
  p_member_id: memberId,  
  p_pro_id: proId,  
})
```

`rpc` significa Remote Procedure Call: invece di scrivere direttamente nella tabella, chiede al database di eseguire una funzione protetta. I nomi `p_member_id` e `p_pro_id` devono coincidere con i parametri SQL. La funzione restituisce la conversazione già esistente oppure la crea. Il vincolo unico sulla coppia membro-professionista evita duplicati.

fetchMemberThread(memberId, proId)

Legge `threads`, restringe con `.eq(...)` sia il membro sia il professionista e usa `.maybeSingle()` perché il risultato lecito è zero oppure una riga.

fetchThread(threadId)

Serve al professionista, che ha già l'ID nell'URL. La select include anche i profili collegati, così la pagina può mostrare nome e avatar dell'altra persona.

fetchThreads(proId)

Legge le conversazioni del professionista. Poi associa a ogni conversazione:

- il profilo del membro;
- l'ultimo messaggio;
- il numero di messaggi ricevuti ma non letti.

La forma restituita alla schermata è più comoda della forma grezza del database.

fetchMessages(threadId)

Legge le colonne elencate in `MESSAGE_COLUMNS`, filtra per conversazione e ordina `created_at` in modo crescente: dal messaggio più vecchio al più recente.

sendMessage({ id, threadId, senderId, body })

- `id` : UUID creato nel telefono prima dell'invio;
- `threadId` : conversazione di destinazione;
- `senderId` : autore;

- `body` : testo.

L'ID creato dal client rende l'operazione ripetibile in sicurezza: se una richiesta offline viene ritentata, la chiave primaria impedisce la creazione di una seconda copia.

```
markThreadRead({ threadId, readerId, readAt, throughCreatedAt })
```

Aggiorna `read_at` soltanto per:

- la conversazione indicata;
- messaggi scritti dall'altra persona;
- messaggi ancora non letti;
- messaggi creati entro `throughCreatedAt`.

Il limite temporale evita di marcare per errore un messaggio arrivato dopo che la pagina aveva calcolato quali messaggi erano visibili.

Ogni funzione segue lo stesso schema:

```
const { data, error } = await ...
if (error) throw error
return data
```

`await` aspetta la risposta; `throw` consegna l'errore a React Query; `return` consegna i dati.

3. Apri la singola conversazione

Ctrl+P → `src/features/chat/ThreadScreen.jsx`.

Parametro nell'URL

```
const { threadId: threadIdParam } = useParams()
```

`useParams()` legge i segmenti variabili dell'URL. Il colon qui rinomina la proprietà: la proprietà `threadId` viene conservata nella variabile locale `threadIdParam`.

Una pagina per entrambi i ruoli

```
const isMember = profile?.role === 'member'
```

- `?.` evita un errore se `profile` non è ancora disponibile;
- `===` confronta valore e tipo;
- il risultato è booleano.

Il membro non ha l'ID della conversazione nell'URL: la cerca con `fetchMemberThread(user.id, proId)`. Il professionista arriva da `/p/chat/:threadId`, quindi esegue `fetchThread(threadIdParam)`.

```
enabled: isMember && Boolean(proId)
```

La query parte soltanto se entrambe le condizioni sono vere. Non avrebbe senso cercare una conversazione se il membro non ha ancora scelto un professionista.

```
const threadId = isMember ? (memberThread.data?.id ?? null) : threadIdParam
```

È un ternario: se è membro usa l'ID trovato nel database; altrimenti usa quello dell'URL. `?? null` sostituisce solo `null` o `undefined` con `null` esplicito.

Perché ci sono più `useEffect`

Ogni effetto gestisce un effetto esterno diverso:

1. marca come letti i nuovi messaggi ricevuti;
2. marca come lette le notifiche relative alla chat;
3. scorre fino all'ultimo messaggio;
4. crea la conversazione al primo utilizzo, se non esiste.

`useRef` conserva un valore tra i render senza causare un nuovo render. `markedUpTo.current` evita di inviare più volte la stessa conferma di lettura. `messagesEnd.current` contiene il riferimento all'elemento HTML in fondo alla lista.

La forma generale è:

```
useEffect(() => {  
  // operazione  
}, [dipendenzaA, dipendenzaB])
```

React riesegue l'effetto quando cambia una dipendenza. Un `return` interno all'effetto, se restituisce una funzione, è la pulizia da eseguire quando l'effetto termina.

Stati prima del JSX principale

I vari `if (...) return ...` distinguono:

- membro senza professionista;
- caricamento;
- errore;
- creazione della prima conversazione fallita;
- conversazione professionale inesistente.

Questo evita di eseguire più sotto codice che presume l'esistenza dei dati.

Raggruppamento e fumetti

`groupByDay(...)` divide i messaggi in Today, Yesterday o data. Nel `map`, `next` e `previous` permettono di sapere se più messaggi consecutivi appartengono allo stesso autore. `tail` dice a `MessageBubble` se deve disegnare la coda conclusiva.

Invio

`MessageComposer` riceve tre proprietà:

- `paused` : l'invio è sospeso perché manca la rete;
- `error` : eventuale errore;
- `onSend` : funzione da chiamare con il testo.

La funzione crea un UUID e chiama la mutation `sendMessage`. Il compositore viene mostrato solo quando `threadId` esiste: un messaggio con `thread_id: null` violerebbe il database.

4. Apri il componente di scrittura

Ctrl+P → `src/features/chat/MessageComposer.jsx`.

```
export default function MessageComposer({ onSend, paused, error })
```

Le tre proprietà vengono destrutturate direttamente dal parametro.

`body` è il testo controllato. “Input controllato” significa che il valore visualizzato è `body` e ogni modifica chiama `setBody(...)`.

`submit(event)` :

1. usa `event.preventDefault()` per impedire al form di ricaricare la pagina;
2. non fa nulla se il testo ripulito è vuoto;
3. chiama `onSend(body.trim())` ;
4. svuota il campo.

`paused` mostra che il messaggio è in coda; `error` comunica un invio fallito.

5. Apri il singolo fumetto

Ctrl+P → `src/features/chat/MessageBubble.jsx`.

Parametri:

- `message` : oggetto della riga `messages` ;
- `mine` : vero se `message.sender_id === user.id` ;
- `tail = true` : valore predefinito, usato per la forma degli angoli.

Da `mine` dipendono allineamento, colore e posizione. L'ora deriva da `message.created_at`. Per un messaggio proprio, due spunte indicano `read_at` presente; una spunta indica consegnato ma non ancora letto. È presentazione di uno stato già salvato nel database.

6. Apri il collegamento Realtime

`Ctrl+P` → `src/features/chat/useThreadMessages.js`.

La funzione è un **custom hook**, cioè una funzione che combina altri hook:

```
export function useThreadMessages(threadId)
```

Riceve l'ID, restituisce l'oggetto query dei messaggi.

Prima esegue la query normale. `refetchInterval: 60_000` significa nuova lettura ogni 60.000 millisecondi, cioè un minuto, come recupero se il collegamento live si interrompe.

Poi crea un canale:

```
supabase
  .channel(`messages:${threadId}`)
  .on('postgres_changes', configurazione, callback)
  .subscribe()
```

- il backtick crea una stringa inserendo `threadId`;
- `postgres_changes` chiede eventi del database;
- `INSERT` ascolta nuovi messaggi;
- `UPDATE` ascolta conferme di lettura;
- `filter` limita alla conversazione corrente;
- `payload.new` è la nuova versione della riga.

All' `INSERT`, `setQueryData` aggiunge il messaggio alla cache solo se quell'ID non esiste già. Il controllo elimina il duplicato tra aggiornamento ottimistico, risposta server e Realtime.

All' `UPDATE`, `map(...)` sostituisce soltanto la riga con lo stesso ID, così compare la seconda spunta.

La pulizia:

```
return () => {
  supabase.removeChannel(channel)
}
```

stacca il canale quando si cambia conversazione o si lascia la pagina. Senza questa pulizia si accumulerebbero ascoltatori duplicati.

7. La frase pronta per l'esame

La chat separa interfaccia, accesso ai dati e sincronizzazione. Le schermate gestiscono ricerca, stati e navigazione; `data/chat.js` interroga Supabase; `useThreadMessages` mantiene la cache aggiornata tramite Realtime. L'invio usa un UUID creato sul client e una mutation ottimistica, quindi appare subito e può essere riprodotto senza duplicati dopo un periodo offline. RLS e funzioni protette controllano che possano accedere soltanto il membro e il professionista della conversazione.

Controllo personale

Se sai rispondere a queste domande, la lezione è acquisita:

1. Perché il membro e il professionista ricavano `threadId` in modo diverso?
2. Perché `MessageComposer` non compare prima che esista `threadId`?
3. Cosa fa `payload.new`?
4. Perché si controlla l'ID prima di aggiungere un messaggio alla cache?
5. Qual è la differenza tra leggere i messaggi e ascoltare Realtime?

Lezione 11 — Profilo, badge, scanner, premi, notifiche e progressi

Questa lezione collega funzioni che sembrano separate ma condividono identità, abbonamento e cronologia dell'utente.

1. Profilo: la pagina che indirizza alle funzioni personali

`Ctrl+P` → `src/features/profile/ProfileScreen.jsx`.

`Row({ icon, label, to })`

È un piccolo componente riutilizzato per ogni voce.

- `icon` : componente grafico da mostrare;
- `label` : testo leggibile;
- `to` : URL verso cui navigare.

Il componente non contiene dati propri: riceve tutto dal genitore. Questo si chiama componente “presentazionale”.

`ProfileScreen()`

`useAuth()` fornisce `user` e `profile`. Non sono la stessa cosa:

- `user` viene da Supabase Auth e identifica la sessione;
- `profile` viene dalla tabella pubblica `profiles` e contiene nome, ruolo, avatar, coach e abbonamento.

```
const isMember = profile?.role === 'member'
```

Le voci Badge, Subscription e Rewards vengono mostrate al membro. Il professionista non ha un abbonamento da esibire all'ingresso.

2. Avatar: dal file del telefono al profilo

Apri affiancati con `Ctrl+\`:

```
src/features/profile/AvatarPicker.jsx  
src/data/profile.js
```

Parametri di `AvatarPicker`

```
AvatarPicker({ userId, avatarUrl, fullName })
```

- `userId` : decide il percorso del file nello storage;
- `avatarUrl` : immagine corrente;
- `fullName` : iniziali di riserva.

`fileRef = useRef(null)` punta all'input file nascosto. Premendo il pulsante, il codice richiama programmaticamente quell'input.

I limiti

```
const MAX_EDGE = 512
const MAX_INPUT_BYTES = 12 * 1024 * 1024
```

L'immagine in ingresso non può superare 12 MiB. `shrink(file)` :

1. decodifica l'immagine con `createImageBitmap` ;
2. calcola `scale` , mai superiore a 1, quindi non ingrandisce immagini piccole;
3. imposta larghezza e altezza mantenendo le proporzioni;
4. disegna in un `canvas` ;
5. produce un JPEG compresso da caricare.

`onPick(event)` legge `event.target.files?.[0]` . Controlla tipo e dimensione, riduce il file e avvia `upload.mutateAsync({ userId, blob })` . `mutateAsync` restituisce una Promise che si può attendere con `await` .

`uploadAvatar({ userId, blob })`

In `data/profile.js` il bucket è `avatars` ; il percorso è:

```
<uuid utente>/avatar.jpg
```

`upsert: true` sostituisce la vecchia immagine nello stesso percorso. Dopo il caricamento si ottiene l'URL pubblico e si aggiunge `?v=<timestamp>` : cambiando URL si evita che il browser continui a mostrare la copia precedente in cache. Infine si aggiorna `profiles.avatar_url` .

`clearAvatar({ userId })` prima azzerava il campo nel profilo e poi rimuove il file. Le policy dello storage permettono a ciascun utente di modificare soltanto la propria cartella.

3. Badge temporaneo del membro

`Ctrl+P` → `src/features/profile/BadgeScreen.jsx` .

Il badge non è un QR permanente. La pagina genera un token temporaneo e il database lo collega al membro.

`clock(seconds)`

- `Math.floor(seconds / 60)` calcola i minuti interi;
- `seconds % 60` è il resto, quindi i secondi;
- `padStart(2, '0')` trasforma 5 in `05`.

Stati e riferimenti

- `badge` : token, QR ed istante di scadenza;
- `error` : errore da mostrare;
- `secondsLeft` : conto alla rovescia;
- `mintingRef` : impedisce due creazioni simultanee;
- `aliveRef` : evita di aggiornare lo stato dopo l'uscita dalla pagina.

`mint()` crea un codice casuale con `generateBadgeCode()`, lo salva con `mintCheckinToken({ memberId: user.id, token })`, e trasforma il token in una data URL con la libreria `qrcode`.

Gli effetti:

1. impostano e puliscono `aliveRef`;
2. creano il primo badge;
3. aggiornano il conto alla rovescia una volta al secondo;
4. alla scadenza generano un badge nuovo.

Il QR contiene solo il token casuale, non nome, email o stato dell'abbonamento. Il dato autorevole resta sul server.

Apri `src/data/checkin.js`.

`generateBadgeCode()` usa `crypto.getRandomValues`, non `Math.random`, perché serve casualità adatta a un token. L'alfabeto omette caratteri facilmente confondibili. `mintCheckinToken` inserisce nella tabella `checkin_tokens`; `redeemCheckinToken(token)` chiama la RPC protetta del database.

4. Scanner del professionista

`Ctrl+P` → `src/features/checkin/ScannerScreen.jsx`.

Riferimenti

- `videoRef` : elemento video che mostra la fotocamera;
- `canvasRef` : tela invisibile su cui viene copiato un fotogramma;
- `lastToken` : ultimo token letto, per non riscansionarlo cinque volte al secondo;
- `cooldownTimer` : consente un nuovo tentativo dopo tre secondi se la rete fallisce.

```
redeem = useCallback(async (token) => ...)
```

`useCallback` conserva la stessa funzione tra i render; è importante perché l'effetto della fotocamera dipende da `redeem`.

La funzione:

1. ignora token vuoto o già elaborato;
2. imposta lo stato “verifica in corso”;
3. attende `redeemCheckinToken(token)`;
4. salva risultato accettato o rifiutato;
5. in caso di errore tecnico riabilita il token dopo 3 secondi;
6. chiude sempre lo stato di caricamento nel `finally`.

Effetto fotocamera

`navigator.mediaDevices.getUserMedia({ video: { facingMode: 'environment' } })` chiede la fotocamera posteriore. Ogni 200 ms:

1. copia il video nel canvas;
2. legge i pixel con `getImageData`;
3. passa i byte a `jsQR`;
4. se trova un QR, passa `code.data` a `redeem`.

La funzione restituita dall'effetto ferma intervallo, timeout e tracce video. È essenziale: altrimenti la spia della fotocamera resterebbe accesa dopo aver lasciato la pagina.

Il modulo `src/features/checkin/scanResult.js` traduce gli stati tecnici del server (`accepted`, `expired`, `used`, abbonamento inattivo e così via) in titolo, messaggio e booleano `accepted` per la UI.

Il modulo permette anche l'inserimento manuale. Il testo viene ripulito con `trim()`, `toUpperCase()` e `replace(/\s+/g, '')`.

Regola reale del database

La versione attuale della RPC è nella patch 026. Verifica token, scadenza, uso precedente, ruolo di chi scansiona e abbonamento. Se tutto è valido crea il check-in e assegna 10 punti al massimo una volta per giornata italiana.

5. Premi

`Ctrl+P` → `src/features/rewards/RewardsScreen.jsx`.

`CATALOGUE` è un array locale, non una tabella:

- shake a 1000 punti;
- sessione PT a 2500;
- mese gratuito a 6000.

La tabella `rewards`, invece, registra i punti realmente guadagnati.

```
const total = data.reduce((sum, reward) => sum + reward.points, 0)
```

`reduce` parte da `0`; per ogni premio aggiunge `reward.points` all'accumulatore `sum`.

`rewardProgress(total, CATALOGUE)` calcola soglia successiva, punti mancanti e percentuale.

`groupByMonth(data, limit)` raggruppa la cronologia; inizialmente ne mostra al massimo 10.

I punti non sono scelti liberamente dal browser:

- allenamento: massimo 30, proporzionali ai set prescritti completati;
- check-in: 10, una volta al giorno;
- abbonamento inattivo: nessun nuovo punto.

Il server calcola e inserisce i punti, impedendo che l'utente invii un numero inventato.

6. Notifiche nell'app e push

Apri:

```
src/features/notifications/NotificationsScreen.jsx
```

Stati:

- `filter`: `all` oppure `unread`;
- `selecting`: modalità selezione multipla;
- `selectedIds`: un `Set`, cioè una collezione senza duplicati;
- `confirming`: operazione distruttiva da confermare;
- `menuAnchor`: elemento a cui ancorare il menu.

La query legge al massimo le 50 notifiche più recenti. Le sei mutation gestiscono: segna una come letta, elimina una, segna selezionate, elimina selezionate, segna tutte, elimina tutte.

Per aggiornare un `Set` React richiede una nuova istanza:

```
setSelectedIds((current) => {  
  const next = new Set(current)  
  // modifica next  
  return next  
})
```

Modificare direttamente `current` potrebbe non produrre un nuovo render.

`groupByDay(...)` divide la lista per data. `TypeIcon` sceglie icona e colore dal tipo; `NotificationText` stampa ora, titolo e corpo. Il colore non è l'unico segnale: restano icona ed etichetta.

Apri `src/data/notifications.js`. Le scritture chiamano RPC protette: il browser non ha il permesso di aggiornare direttamente la tabella `notifications`. I parametri `p_id`, `p_ids`, `p_url` e `p_before` limitano esattamente quali righe gestire.

Push del dispositivo

Apri `src/features/profile/NotificationSwitch.jsx` e `pushSubscription.js`.

`pushSupported()` controlla la presenza di Service Worker, PushManager e Notification. `pushConfigured()` verifica la chiave pubblica VAPID. `currentSubscription()` legge l'iscrizione attuale.

`enablePush(userId)`:

1. chiede permesso all'utente;
2. attende che il Service Worker sia pronto;
3. crea o recupera una sottoscrizione Push;
4. salva endpoint e chiavi in Supabase.

`disablePush()` cancella prima la riga server e poi annulla la sottoscrizione del browser.

7. Progressi del cliente

Apri affiancati:

```
src/features/progress/ClientProgressScreen.jsx
src/features/progress/progress.js
```

La schermata ricava `clientId` dall'URL e carica:

- profilo del cliente;
- piano attivo;
- tentativi di allenamento recenti;
- eventuale tentativo ancora aperto.

`workoutWeekSummary(runs, totalSessions, todayISO)` è una funzione pura: riceve dati, restituisce un risultato e non modifica database o UI.

Passaggi:

1. individua lunedì e domenica della settimana corrente;
2. ordina i tentativi dal più recente;
3. conserva il tentativo più recente per ogni sessione;
4. considera per il riepilogo solo `completed` e `partial`;
5. conta separatamente completati, parziali e da fare;
6. la percentuale usa soltanto i completati.

Ripetere due volte la stessa sessione non riempie due posti dell'obiettivo settimanale.

`runDurationMs(run)` calcola:

ora di fine - ora di inizio - tempo totale in pausa

Per un tentativo ancora aperto restituisce `null`, perché non esiste ancora una durata finale.

Apri `WorkoutRunDetailScreen.jsx`. `ExerciseResult({ item, logs })` confronta set prescritti e set validamente registrati. Mostra quelli completati e quelli mancanti. La pagina riunisce run, sessione, esercizi e log attraverso i rispettivi ID.

8. Risposte pronte

Il QR contiene lo stato dell'abbonamento? No. Contiene un token temporaneo; il server legge membro e abbonamento quando il professionista lo riscatta.

Il catalogo premi è nel database? No. Le soglie mostrate sono costanti del frontend; il database conserva gli eventi di guadagno punti.

Perché la durata esclude le pause? Perché `paused_total_ms` viene sottratto dal tempo trascorso tra apertura e chiusura del tentativo.

Un allenamento parziale completa l'obiettivo settimanale? No. Resta visibile al professionista, ma soltanto `completed` incrementa la parte completata.

Qual è la differenza tra notifica in-app e push? La notifica in-app è una riga persistente nel database. Il push è il messaggio inviato al sistema operativo anche quando la pagina non è aperta.

Lezione 12 — Il database, spiegato da zero

Prima idea: che cos'è il database?

Il frontend React è ciò che vedi e tocchi. Il database PostgreSQL è la memoria condivisa e autorevole dell'applicazione. Se chiudi il browser, una variabile React scompare; una riga salvata nel database rimane e può essere letta da un altro dispositivo autorizzato.

Supabase fornisce:

- PostgreSQL, cioè il database relazionale;
- Auth, cioè registrazione, accesso e sessione;
- API automatiche per leggere le tabelle;
- Realtime per ascoltare modifiche;
- Storage per l'avatar;
- Edge Functions per il push.

Parole indispensabili

- **tabella**: insieme di righe dello stesso tipo, simile a un foglio strutturato;
- **riga**: un elemento, per esempio un profilo;
- **colonna**: una sua proprietà, per esempio `full_name`;
- **tipo**: genere di valore ammesso, come testo, numero, data o UUID;
- **UUID**: identificativo lungo e praticamente unico;
- **primary key**: colonna che identifica senza ambiguità una riga;
- **foreign key**: riferimento alla primary key di un'altra tabella;
- **constraint**: regola che il database fa rispettare;
- **index**: struttura che rende più veloce una ricerca;
- **null**: valore assente, diverso da testo vuoto e da zero;
- **transaction**: insieme di operazioni che riescono tutte oppure vengono annullate tutte;
- **RPC**: funzione SQL richiamata dall'app.

1. Apri lo schema

In VS Code: `Ctrl+P` → `supabase/schema.sql`.

Questo è lo schema iniziale. Non rappresenta da solo il database finale: dopo vengono applicati `policies.sql` e le patch numerate. In particolare `notifications` viene aggiunta dalla patch 020.

Le righe che iniziano con `--` sono commenti e PostgreSQL non le esegue. Il punto e virgola `;` conclude un comando SQL.

Gli enum, righe iniziali

```
create type user_role as enum ('member', 'professional');
```

Un enum accetta soltanto i valori elencati. Impedisce, per esempio, di salvare per errore `membro`, `admin` o `trainer` nella colonna ruolo.

- `user_role` : membro o professionista;
- `pro_specialty` : personal trainer, nutrizionista o entrambi;
- `session_status` : vecchio stato della sessione, mantenuto per compatibilità ma non più usato come fonte reale;
- `appointment_kind` : allenamento, consulto protocollo o nutrizione;
- `appointment_status` : in attesa, confermato, cancellato o concluso;
- `subscription_status` : attivo, scaduto o sospeso;
- `run_outcome` : allenamento completato, parziale o abbandonato.

2. profiles : chi usa l'app

Vai alla riga con `create table profiles` usando `Ctrl+F`.

```
id uuid primary key references auth.users on delete cascade
```

Lo stesso UUID identifica l'account in Supabase Auth e il profilo pubblico. `on delete cascade` significa che eliminando l'account vengono eliminate automaticamente le righe dipendenti.

Colonne:

- `role` : membro o professionista;
- `specialty` : specializzazione, assente per i membri;
- `full_name` : obbligatorio (`not null`);
- `avatar_url`, `bio` : facoltativi;
- `assigned_pro_id` : profilo del professionista scelto dal membro; se quel professionista viene rimosso diventa `null`;
- `subscription_status`, `subscription_until` : stato e scadenza dell'abbonamento;
- `created_at` : istante di creazione, impostato automaticamente con `now()`.

Il `check` impone questa equivalenza: professionista ↔ specializzazione presente. Un membro con specializzazione o un professionista senza specializzazione viene rifiutato.

L'indice su `assigned_pro_id` accelera l'elenco “tutti i miei clienti”.

Il trigger in fondo allo schema, `on_auth_user_created`, esegue `handle_new_user()` dopo la creazione di un account Auth e inserisce automaticamente il profilo con ruolo membro.

3. Allenamenti: dalla ricetta all'esecuzione

Queste tabelle formano una catena. Leggila così:

```
profiles
├─ workout_plans
│   └─ workout_sessions
│       └─ session_exercises → exercises
│           └─ set_logs
└─ workout_runs
```

exercises

È il catalogo condiviso. Ogni esercizio ha nome unico, gruppo muscolare, attrezzatura, istruzioni e possibili URL di video e immagine. Il catalogo non dice quante ripetizioni fare: quello dipende dalla prescrizione nel piano.

workout_plans

Un piano appartiene a `member_id` ed è scritto da `author_id`. Contiene nome, obiettivo, livello, durata in settimane e scadenza.

`replaces_plan_id` collega un nuovo piano al precedente. Il vecchio non viene cancellato: rimane la cronologia. L'indice unico impedisce che due piani diversi dichiarino di sostituire lo stesso predecessore.

workout_sessions

Sono le giornate o schede dentro il piano, per esempio "Leg Day". `position` ne definisce l'ordine. `unique(plan_id, position)` impedisce due sessioni nella stessa posizione.

La colonna `status` è legacy. Lo stato mostrato oggi viene derivato dai `workout_runs` della settimana corrente, non scritto in questa colonna.

session_exercises

È una **tabella di associazione arricchita**: collega una sessione a un esercizio del catalogo e aggiunge la prescrizione concreta.

- `target_sets` : serie richieste;
- `target_reps` : ripetizioni richieste;
- `target_weight` : peso suggerito, facoltativo;
- `rest_seconds` : recupero;
- `notes` : indicazioni;
- `position` : ordine.

Il vincolo vieta posizioni, serie e ripetizioni non positive, recuperi negativi e pesi negativi.

workout_runs

Una sessione può essere ripetuta: ogni tentativo produce una nuova riga.

- `started_at` , `ended_at` : inizio e fine;
- `paused_at` : inizio della pausa corrente;
- `paused_total_ms` : somma delle pause già concluse;
- `outcome` : completed, partial o abandoned;
- `pct` : percentuale di serie prescritte validamente registrate;
- `note` : nota opzionale finale.

L'indice parziale `workout_runs_one_open_per_member` vale soltanto dove `ended_at is null` : ogni membro può avere al massimo un allenamento aperto.

set_logs

Ogni riga è una serie realmente eseguita: esercizio prescritto, run, membro, numero della serie, ripetizioni, peso e istante.

L'ID è fornito dal client perché il salvataggio può restare offline e venire ripetuto. L'unicità (`run_id`, `session_exercise_id`, `set_number`) impedisce di registrare due volte la stessa serie nello stesso tentativo.

Questa separazione è fondamentale:

- `session_exercises` dice **cosa si dovrebbe fare**;
- `set_logs` dice **cosa è stato fatto**;
- `workout_runs` riassume **quel tentativo**.

4. Nutrizione

Catena:

```
profiles
├─ nutrition_plans
│   └─ nutrition_days
│       └─ meals
```

nutrition_plans

Contiene membro, autore, nome, obiettivi di kcal/proteine/carboidrati/grassi, note e collegamento all'eventuale piano sostituito.

nutrition_days

Ogni riga è un tipo di giornata, non necessariamente una sola data: per esempio “Training day” applicato a lunedì, mercoledì e venerdì. `weekdays` è un array di numeri da 0 a 6, nello stesso ordine di `Date.getDay()` JavaScript: 0 domenica, 6 sabato.

meals

Un pasto appartiene a un day type. Contiene nome, orario testuale, posizione, valori nutrizionali, alternative e `items` in formato `jsonb`.

`jsonb` permette di salvare una lista strutturata di alimenti dentro la riga. È usato qui perché gli elementi del pasto sono letti e salvati come un blocco; le entità principali restano invece tabelle relazionali.

5. Agenda e prenotazioni

availability

Descrive la disponibilità settimanale ricorrente del professionista:

- giorno da 0 a 6;
- ora locale di inizio e fine;
- `ends_at > starts_at` impedisce intervalli invertiti.

Non è ancora un appuntamento: è la finestra entro cui si può prenotare.

appointments

Collega membro e professionista e salva tipo, stato, inizio, fine e note. Gli istanti sono `timestampz`, quindi rappresentano un momento preciso e possono essere convertiti correttamente nel fuso locale.

Regole applicative importanti:

- una richiesta del membro nasce `pending`;
 - una prenotazione creata dal professionista nasce `confirmed`;
 - la RPC controlla ruolo, assegnazione, abbonamento, disponibilità e conflitti;
 - i cambi di stato usano confronto con lo stato atteso, evitando che due azioni contemporanee si sovrascrivano.
-

6. Chat

threads

Una conversazione collega esattamente un membro e un professionista. `unique(member_id, pro_id)` garantisce una sola conversazione per coppia.

messages

Ogni messaggio contiene conversazione, autore, testo, istante di lettura e creazione. `read_at = null` significa non ancora letto. L'indice su conversazione e data rende veloce il caricamento ordinato.

Il database e RLS verificano che il mittente appartenga alla conversazione.

7. Accessi e premi

checkin_tokens

Token monouso del QR:

- appartiene a un membro;
- scade dopo 60 secondi per default;
- `used_at` mostra se è già stato consumato;
- `token` è unico.

checkins

Registra membro, professionista che ha scansionato e istante. Non si limita a dire “QR valido”: crea una cronologia reale degli ingressi.

rewards

Ogni riga è un guadagno punti, non un premio già riscattato. `code` identifica univocamente l'origine per quel membro.

- un allenamento può collegarsi a `run_id` e `workout_session_id`;
- `reward_day` consente massimo un premio per la stessa sessione nella stessa giornata;
- il check-in usa un codice basato sulla giornata di Roma;
- le foreign key `restrict` impediscono di cancellare run o sessione ancora citati da una ricompensa.

body_metrics

Prevede peso e nota registrati da un professionista. La coppia (`member_id`, `measured_on`) è unica. È presente nel backend ma non è un flusso principale esposto nell'interfaccia corrente: all'esame va detto chiaramente, senza presentarlo come una funzione completa della UI.

8. Push e notifiche

push_subscriptions

Una riga identifica un browser/dispositivo autorizzato a ricevere push:

- `endpoint` : indirizzo assegnato dal servizio push;
- `p256dh` e `auth` : chiavi crittografiche della sottoscrizione;
- un utente può avere più dispositivi;
- lo stesso endpoint è unico.

app_config

Contiene configurazione privata usata dal database per chiamare la Edge Function: URL e segreto. Il browser non deve leggerla.

notifications

Non cercarla in `schema.sql` : apri `supabase/patches/020-notifications.sql` .

Colonne: utente, tipo, titolo, corpo, URL, istante di lettura e creazione. Il trigger che genera una notifica salva prima questa riga; se il push non è configurato, la notifica in-app continua comunque a esistere.

Tipi generati: messaggio, appuntamento, nuovo piano workout e nuovo piano nutrizionale.

9. Cosa significano le parole SQL nel file

Esempio:

```
member_id uuid not null references profiles(id) on delete cascade
```

- `member_id` : nome colonna;
- `uuid` : tipo;
- `not null` : obbligatorio;
- `references profiles(id)` : deve indicare un profilo esistente;
- `on delete cascade` : eliminando quel profilo, elimina anche questa riga.

Altri termini:

- `default` : valore usato se non ne viene fornito uno;
 - `unique` : vieta duplicati;
 - `check` : espressione che deve risultare vera;
 - `numeric(6,2)` : massimo sei cifre totali, due dopo la virgola;
 - `timestampz` : data e ora con riferimento temporale assoluto;
 - `date` : solo giorno;
 - `time` : solo ora;
 - `desc` : ordine decrescente;
 - `where ... is not null` su un indice: indice che vale solo per quelle righe.
-

10. Come una query JavaScript corrisponde a SQL

Apri `src/data/appointments.js` accanto allo schema.

```
supabase
  .from('appointments')
  .select('id, kind, status, starts_at')
  .eq('member_id', memberId)
  .gte('starts_at', from)
  .lt('starts_at', to)
  .order('starts_at')
```

Significa concettualmente:

```
select id, kind, status, starts_at
from appointments
where member_id = ...
    and starts_at >= ...
    and starts_at < ...
order by starts_at;
```

- `.eq` : uguale;
- `.gte` : maggiore o uguale;
- `.lt` : minore;
- `.is('ended_at', null) : IS NULL ;`
- `.in('id', ids) : uno dei valori;`
- `.single()` : deve arrivare una riga;
- `.maybeSingle()` : zero o una riga sono entrambi normali.

Una select annidata come `pro:profiles!appointments_pro_id_fkey (...)` chiede a PostgREST di seguire quella specifica foreign key e restituire il profilo sotto il nome `pro` . Serve specificare il vincolo perché `appointments` punta a `profiles` due volte.

11. Non confondere schema iniziale e database finale

Ordine di installazione:

1. `schema.sql` crea la base;
2. `policies.sql` attiva i permessi per riga;
3. patch 001–026 evolvono struttura e sicurezza;
4. `seed.sql` carica i dati demo;
5. `verify.sql` verifica struttura e invarianti;
6. i probe Node controllano RLS passando dall'API pubblica.

Le patch non sono 26 database diversi: sono la storia ordinata che conduce allo stato finale. Quando una funzione viene riscritta, vale l'ultima versione applicata.

Versioni finali più importanti:

- piani workout, chiusura run, nutrizione e appuntamenti: patch 023;
 - riscatto check-in: patch 026;
 - notifica appuntamento: patch 021.
-

Risposta completa da esame

TrainHub usa PostgreSQL tramite Supabase. Auth gestisce gli account, mentre `profiles` aggiunge ruolo e dati applicativi. Il modello separa prescrizione ed esecuzione: piani, sessioni ed esercizi descrivono il workout; run e set log registrano ogni tentativo. Nutrizione usa piano, tipi di giornata e pasti. Appuntamenti collegano disponibilità, membro e professionista; thread e messaggi gestiscono la chat. Token e check-in verificano l'accesso, rewards conserva i punti, notifications e push_subscriptions supportano gli avvisi. Le foreign key mantengono i collegamenti, i constraint impediscono dati impossibili e RLS/RPC proteggono ogni operazione.

Esercizio davanti a VS Code

Scegli una riga `set_logs`. Indica ad alta voce:

1. qual è la primary key;
2. quali sono le foreign key;
3. quali valori non possono essere null;
4. quali vincoli vietano valori impossibili;
5. come risalire da quel set al nome dell'esercizio e al membro.

Soluzione del punto 5:

```
set_logs
  → session_exercises
  → exercises (nome)

set_logs
  → profiles tramite member_id (membro)
```

Lezione 13 — Sicurezza: chi può fare cosa e perché

Il problema da capire

Nascondere un pulsante non protegge i dati. Un utente potrebbe aprire gli strumenti del browser e inviare a mano una richiesta. Perciò TrainHub applica due livelli:

1. il frontend mostra soltanto le azioni adatte al ruolo;
2. il database controlla comunque l'identità e rifiuta richieste non autorizzate.

La seconda è la vera barriera di sicurezza.

1. Identità: `auth.uid()`

Quando l'utente è autenticato, Supabase invia un token firmato con ogni richiesta. PostgreSQL rende disponibile l'UUID del chiamante tramite:

```
auth.uid()
```

Non è un ID scelto in un campo del form. Deriva dalla sessione verificata. Un parametro come `p_member_id`, invece, arriva dalla richiesta e deve essere confrontato con `auth.uid()` o con relazioni autorizzate.

2. Apri le policy

`Ctrl+P` → `supabase/policies.sql`.

Attivare RLS

```
alter table profiles enable row level security;
```

RLS significa Row Level Security, sicurezza a livello di riga. Una policy può consentire allo stesso utente di vedere alcune righe e non altre della stessa tabella.

Due parole diverse:

- `using (...)`: quali righe esistenti posso leggere, modificare o eliminare;
- `with check (...)`: quali valori nuovi posso inserire o lasciare dopo un aggiornamento.

`is_professional()`

La funzione restituisce vero se esiste un profilo con:

```
id = auth.uid() and role = 'professional'
```

`owns_member(target uuid)`

Il parametro `target` è l'UUID del membro di cui si stanno leggendo dati. Restituisce vero se:

- `target` è l'utente stesso; oppure
- il profilo `target` ha `assigned_pro_id = auth.uid()`.

Questa funzione rende riutilizzabile la regola “membro sui propri dati, coach sui propri clienti”. Il nome non significa proprietà legale: significa relazione autorizzata nell'app.

3. Leggi le policy tabella per tabella

Profili

- ciascuno legge il proprio profilo;
- ogni autenticato può leggere i professionisti, perché deve poterli scegliere;
- il professionista legge i profili assegnati a lui;
- l'utente aggiorna la propria riga, ma i privilegi di colonna impediscono di cambiare ruolo, specialità e abbonamento.

La condizione `auth.uid() is not null` nella policy dei professionisti è necessaria: senza un riferimento al chiamante, anche una persona non autenticata con la chiave pubblica potrebbe leggere quei profili.

Esercizi

Ogni autenticato legge il catalogo. Solo i professionisti possono scriverlo.

Workout e nutrizione

`workout_plans`, `run`, `set` e `nutrition_plans` usano `owns_member(member_id)`. Per tabelle figlie, la policy segue le foreign key fino al piano e poi al membro.

Esempio: per leggere `session_exercises`, il database controlla che la sessione appartenga a un piano il cui membro è accessibile al chiamante.

Disponibilità

Ogni autenticato la legge, perché il membro deve vedere gli slot. Soltanto il professionista può scrivere righe con il proprio `pro_id`.

Appuntamenti

La lettura è permessa soltanto se il chiamante è `member_id` oppure `pro_id` di quella riga. Le scritture finali passano per RPC sicure.

Chat

Per thread e messaggi devono essere vere entrambe le idee:

- il chiamante è uno dei due partecipanti;
- quel membro è ancora assegnato a quel professionista.

Per inserire un messaggio deve valere anche `sender_id = auth.uid()`: non si può fingere di essere l'altra persona.

Per aggiornare un messaggio, il chiamante deve essere il destinatario, non il mittente, e `read_at` deve diventare non nullo. La patch 008 revoca l'aggiornamento generale e concede solo la colonna `read_at`, così non si può riscrivere `body`.

Premi, check-in e misure

- rewards: membro e coach assegnato leggono;
- check-in: membro, coach assegnato o professionista che ha effettuato la scansione leggono;
- token: il membro crea e legge i propri, ma solo la RPC può consumarli;
- body metrics: membro e coach assegnato leggono; la scrittura professionale passa da funzione protetta.

Push

L'utente gestisce sottoscrizioni con il proprio `user_id`. `app_config` non ha policy e perde anche i grant: dal browser è inaccessibile.

4. GRANT e RLS sono due cancelli

Immagina due porte in serie:

```
richiesta → privilegio GRANT → policy RLS → riga
```

Un `GRANT SELECT` dice che il ruolo può tentare una lettura della tabella. RLS decide quali righe. Se manca il grant, PostgreSQL rifiuta prima ancora di consultare la policy.

Per le tabelle sensibili TrainHub revoca `INSERT`, `UPDATE`, `DELETE` al ruolo dell'app. Nessuna policy troppo permissiva può allora consentire una scrittura diretta; il browser deve chiamare una RPC specifica.

5. Perché usare RPC sicure

Apri `supabase/patches/015-essential-security.sql` e cerca `create_appointment_secure`.

La firma:

```
create or replace function public.create_appointment_secure(  
  p_id uuid,  
  p_member_id uuid,
```

```

p_pro_id uuid,
p_kind public.appointment_kind,
p_starts_at timestamptz,
p_ends_at timestamptz,
p_notes text
) returns setof public.appointments

```

Spiegazione:

- `create or replace` : crea oppure aggiorna la funzione;
- `public.` : schema esplicito;
- `p_...` : convenzione per parametri;
- `returns setof public.appointments` : restituisce righe con la stessa forma della tabella;
- `plpgsql` : linguaggio procedurale PostgreSQL.

La funzione:

1. legge il chiamante da `auth.uid()` ;
2. verifica date valide;
3. verifica che il membro sia assegnato al professionista;
4. verifica il ruolo del professionista;
5. se chi chiama è il membro imposta `pending` ;
6. se chi chiama è il professionista imposta `confirmed` ;
7. altrimenti rifiuta;
8. inserisce usando l'ID fornito;
9. tratta un replay identico come successo;
10. rifiuta il riuso dello stesso ID con contenuto diverso.

Nella versione finale della patch 023 controlla anche che un membro che prenota abbia abbonamento attivo.

Compare-and-set dello stato

`set_appointment_status_secure(appointmentId, expectedStatus, status)` riceve lo stato che la schermata aveva letto. Aggiorna soltanto se il database è ancora in quello stato. Se nel frattempo un altro dispositivo lo ha cambiato, restituisce conflitto invece di sovrascrivere una decisione più recente.

Transizioni consentite:

- `pending` → `confirmed` o `cancelled`;
- `confirmed` → `done` o `cancelled`;
- `cancelled` → `confirmed`.

6. `security definer` : potente e pericoloso se usato male

Una funzione normale usa i privilegi di chi la chiama. Una funzione `security definer` usa i privilegi del proprietario della funzione e può superare RLS. Serve per operazioni atomiche, ma per questo deve controllare ogni parametro al proprio interno.

```
security definer
set search_path = ''
```

`search_path` è l'elenco di schemi in cui PostgreSQL cerca nomi non qualificati. Lasciarlo manipolabile può permettere a un attaccante di far trovare una tabella o funzione temporanea omonima. TrainHub lo azzerava e scrive nomi completi come `public.profiles`.

La sequenza sicura è:

1. identificare `auth.uid()`;
2. convalidare ruolo e relazioni;
3. convalidare contenuto;
4. eseguire in un'unica transazione;
5. concedere `EXECUTE` agli autenticati;
6. revocarlo a pubblico e anonimo.

7. Operazioni workout protette

Cerca questi nomi nella patch 015; ricorda che alcune versioni vengono poi sostituite:

- `create_workout_plan_secure` : crea piano, sessioni ed esercizi insieme;
- `_insert_session_bundle` : helper interno, non chiamabile dal browser;
- `start_workout_run_secure` : apre un tentativo soltanto sulla sessione del membro autenticato;
- `pause_workout_run_secure` : mette in pausa verificando il contatore atteso;
- `resume_workout_run_secure` : calcola la pausa e la aggiunge al totale;
- `log_workout_set_secure` : registra un set legato alla run, al membro e all'esercizio corretti;
- `close_workout_run_secure` : calcola percentuale, esito e punti;
- `save_workout_run_note_secure` : aggiunge la nota a una run già chiusa.

Apri poi `supabase/patches/023-subscription-enforcement.sql` e cerca `close_workout_run_secure`.

Calcolo verificabile

- `v_target` : somma delle serie prescritte;
- `v_done` : per ogni esercizio, minimo tra log validi e serie prescritte;
- `v_pct` : `floor(100 × completate / prescritte)`;
- esito: `abandoned` se dichiarato abbandonato, `completed` se tutte le serie sono valide, altrimenti `partial`;
- punti: zero per `abandoned`, altrimenti `floor(30 × completate / prescritte)`;
- una riga reward viene inserita solo con abbonamento attivo;
- il vincolo giornaliero impedisce di guadagnare di nuovo ripetendo la stessa sessione nello stesso giorno.

Il `LEAST(logged, target_sets)` impedisce che serie extra portino percentuale o punti oltre il massimo.

8. Piani e abbonamento

Nella patch 023:

```
has_active_subscription(p_member_id)
```

restituisce vero se:

- lo stato non è suspended o expired;
- la scadenza è assente oppure non precedente a oggi.

La funzione viene applicata a:

- creazione del piano workout;
- creazione del piano nutrizionale;
- prenotazione iniziata dal membro;
- assegnazione punti alla chiusura workout.

Un professionista può prenotare per un membro inattivo, utile per una sessione di rientro, ma quel membro non può prenotare da sé.

`set_subscription_status_secure` permette al professionista assegnato di simulare attivazione o sospensione. Nell'app didattica rappresenta una conferma del sistema palestra; in produzione sarebbe normalmente un sistema di fatturazione, non il trainer, a controllarlo.

9. Check-in sicuro

Apri `supabase/patches/026-checkin-reward-day-frame.sql` e cerca `redeem_checkin_token`.

Parametri: solo `p_token text`. Membro e stato non arrivano dal browser: vengono ricavati dalla riga token.

La funzione finale:

1. verifica che il chiamante sia professionista;
2. trova e blocca membro e token per evitare corse tra due scanner;
3. distingue token inesistente, scaduto e già usato;
4. calcola abbonamento sospeso, scaduto o attivo;
5. se non attivo non consuma il token;
6. se attivo imposta `used_at` e crea `checkins`;
7. prova ad aggiungere 10 punti una sola volta nel giorno `Europe/Rome`;
8. restituisce uno stato comprensibile alla schermata.

L'uso del giorno di Roma risolve gli errori attorno alla mezzanotte che si avrebbero usando il giorno UTC.

10. Trigger e notifiche

Un trigger è una funzione eseguita automaticamente quando una tabella cambia.

Esempi:

- nuovo utente Auth → crea `profiles` ;
- nuovo messaggio → notifica l'altra persona;
- appuntamento confermato/cancellato/concluso → notifica il membro;
- nuovo piano → notifica il membro;
- nuovo evento premi → i punti vengono derivati dal server.

Il vantaggio è che la notifica non dipende dal ricordarsi di inviarla in ogni schermata: segue il cambiamento autorevole nel database.

11. Come verificare davvero la sicurezza

Apri `supabase/INSTALL.md` .

- `verify.sql` controlla schema, funzioni, grant, vincoli e dati, ma viene eseguito dal dashboard privilegiato, quindi non può provare RLS dal punto di vista dell'utente;
- `node supabase/probe-rls.mjs` prova cosa può leggere un anonimo attraverso l'API;
- `node supabase/probe-security.mjs` accede con ruoli demo e controlla le letture consentite e negate.

È importante distinguere i test: un controllo eseguito come amministratore può vedere tutto anche se una policy utente è sbagliata.

12. Mappa delle patch da ricordare

Non memorizzare 26 file riga per riga. Ricorda gli snodi:

- 001: chiude lettura anonima dei profili;
- 006: ripristina i grant necessari prima di RLS;
- 007–008: Realtime messaggi e update limitato a `read_at` ;
- 009: token QR;
- 010–012: push e trigger;
- 013–014: workout run e percentuale;
- 015: RPC sicure e vincoli principali;
- 016: conteggio serie corretto;
- 017–018: piano self-authored e piano multi-sessione;
- 020–021: centro notifiche e gestione multipla;
- 022: day types nutrizionali;
- 023: abbonamento realmente applicato;
- 024 e 026: punti check-in e giornata italiana;

- 025: bucket avatar.

Le patch 002 e 005 sono storia superata e non si applicano in una nuova installazione.

Risposta da esame

Il controllo grafico del ruolo migliora l'esperienza, ma la sicurezza reale è nel database. Supabase ricava l'utente da `auth.uid()`. I GRANT stabiliscono quali operazioni un ruolo può tentare e RLS filtra le righe. Le scritture sensibili non sono concesse direttamente: passano da RPC `security definer` che azzerano il `search_path`, verificano ruolo, assegnazione, abbonamento e integrità dei parametri, poi eseguono l'operazione atomicamente. UUID client e controlli dello stato rendono sicura anche la ripetizione dopo un periodo offline.

Domande di controllo

1. Perché nascondere un pulsante non basta?
2. Differenza tra GRANT e RLS?
3. Differenza tra `using` e `with check`?
4. Perché una RPC `security definer` deve controllare `auth.uid()`?
5. Perché `set search_path = ''`?
6. Chi può creare un piano nutrizionale?
7. Chi può creare un piano workout?
8. Perché un set extra non aumenta i punti oltre il massimo?

Lezione 14 — PWA, modalità offline, tema e avvio del progetto

1. Come avviare TrainHub sul computer

Apri in VS Code la cartella principale `TrainHub`, poi apri il terminale integrato con `Ctrl+``.

Prerequisiti:

- Node.js 20.19 o superiore, oppure 22.12 o superiore;
- dipendenze installate con `npm install`;
- file `.env.local` compilato;
- database Supabase già installato.

Il file `.env.local` deriva da `.env.example`:

```
VITE_SUPABASE_URL=...  
VITE_SUPABASE_ANON_KEY=...  
VITE_VAPID_PUBLIC_KEY=...
```

- URL e anon key collegano il browser al progetto Supabase;
- la chiave VAPID pubblica serve per iscriversi alle notifiche push;
- il prefisso `VITE_` rende il valore disponibile al codice frontend;
- proprio per questo tali valori finiscono nel bundle e sono pubblici: qui non si deve mai mettere la service role key o un segreto.

Comandi:

```
npm run dev  
npm run lint  
npm run build  
npm run preview
```

- `dev`: server rapido di sviluppo, normalmente `http://localhost:5173`;
- `lint`: controlli statici di sintassi e regole React;
- `build`: produce la versione ottimizzata nella cartella `dist/`;
- `preview`: serve localmente il contenuto di `dist/`.

Il Service Worker è disabilitato in `dev`. Installazione, offline e push vanno provati con `build` seguito da `preview`.

2. Apri `package.json`

`Ctrl+P` → `package.json`.

È il manifesto del progetto JavaScript.

```
"type": "module"
```

Permette la sintassi `import / export` dei moduli ES.

`scripts` associa un nome corto a un comando. Quando scrivi `npm run build`, npm esegue `vite build`.

Dipendenze principali

- `react`, `react-dom`: componenti e rendering nel browser;
- `react-router`: URL e navigazione tra schermate;
- `@mui/material`, icone, Emotion: componenti grafici e CSS-in-JS;
- `@tanstack/react-query`: cache, caricamenti, errori e mutation;
- `@supabase/supabase-js`: Auth, database, Realtime e Storage;
- `idb-keyval` e `persist`: salvataggio cache in IndexedDB;
- `jsqr`: lettura QR dalla fotocamera;
- `qrcode`: generazione del QR del membro.

Dipendenze di sviluppo:

- `vite`: server e build;
- plugin React: trasformazione JSX e refresh;
- `eslint`: analisi statica;
- `vite-plugin-pwa` e Workbox: manifest e Service Worker.

Il frontend è JavaScript con file `.js` e `.jsx`. L'unico file TypeScript principale è la Edge Function `supabase/functions/notify/index.ts`, eseguita su Deno, non nel browser.

3. Che cos'è una PWA?

PWA significa Progressive Web App: è un'app web che, con browser compatibile e HTTPS, può essere installata, avviata in una finestra indipendente, usare una cache offline e ricevere push.

Tre pezzi:

1. **manifest**: nome, icone, colori, orientamento e modalità standalone;
 2. **Service Worker**: programma in background controllato dal browser;
 3. **HTTPS**: requisito di sicurezza, eccetto `localhost` durante lo sviluppo.
-

4. Apri la configurazione PWA

Ctrl+P → vite.config.js .

Struttura generale

```
export default defineConfig({ ... })
```

Esporta l'oggetto di configurazione letto da Vite.

plugins contiene:

- react() : abilita React/JSX;
- VitePWA({...}) : genera manifest e costruisce il Service Worker.

Parametri PWA

- registerType: 'autoUpdate' : una nuova versione del worker prende il controllo automaticamente;
- strategies: 'injectManifest' : si usa il Service Worker scritto dal progetto, e Vite inserisce l'elenco dei file costruiti;
- srcDir: 'src', filename: 'sw.js' : file sorgente;
- includeAssets : icone e favicon da includere;
- manifest : metadati installabili;
- devOptions.enabled: false : nessun Service Worker durante npm run dev .

Nel manifest:

- display: 'standalone' toglie l'interfaccia tipica del browser;
- orientation: 'portrait' indica orientamento verticale;
- start_url è la pagina di avvio;
- scope limita le URL controllate;
- l'icona maskable tollera ritagli diversi sui sistemi operativi.

globPatterns precarica JavaScript, CSS, HTML e i sottoinsiemi latini del font Inter. Non scarica alfabeti non usati.

Host e telefono

allowedHosts consente gli host ngrok dichiarati. preview.host: true rende il server raggiungibile dalla rete locale. Questo non sostituisce HTTPS: per installare e usare push su telefono si utilizza un tunnel HTTPS.

5. Apri il Service Worker

Ctrl+P → src/sw.js .

Questo file viene eseguito dal browser in un contesto separato dalla pagina. Non può renderizzare React.

Aggiornamento e cache shell

```
self.skipWaiting()
clientsClaim()
```

Il worker nuovo non aspetta la chiusura di tutte le schede e prende il controllo dei client.

```
cleanupOutdatedCaches()
precacheAndRoute(self.__WB_MANIFEST)
```

Workbox elimina cache di build obsolete e precachea i file che Vite inserisce al posto di `self.__WB_MANIFEST`.

```
registerRoute(new NavigationRoute(createHandlerBoundToURL('index.html')))
```

TrainHub è una Single Page Application: URL differenti devono ricevere lo stesso `index.html`, poi React Router sceglie la schermata. Servire la shell dalla precache permette un avvio offline.

Cosa non viene messo nella cache del Service Worker

Non viene aggiunta una cache runtime per le risposte REST di Supabase. I dati applicativi sono già persistiti da TanStack Query in IndexedDB. Una seconda cache sarebbe più difficile da invalidare e potrebbe conservare dati del precedente utente dopo il logout. Le immagini restano alla normale cache HTTP.

Evento push

```
self.addEventListener('push', (event) => { ... })
```

Quando arriva un push, il worker prova a leggere JSON con titolo, corpo e URL. Se il payload non è valido usa valori sicuri. `event.waitUntil(...)` dice al browser di tenere vivo il worker fino a quando la notifica è stata mostrata.

Click sulla notifica

L'URL ricevuto viene risolto rispetto all'origine TrainHub. Se l'origine non coincide, viene ignorato: evita che una notifica con il logo dell'app apra un sito esterno.

Il worker cerca una finestra già aperta:

- se esiste, la naviga all'URL e la porta in primo piano;
- se non esiste, apre una nuova finestra.

6. Cache dati e offline

Apri `src/lib/queryClient.js`.

Query

- `gcTime: ONE_WEEK` : una query inattiva resta conservabile per una settimana;
- `staleTime: 30 secondi` : entro 30 secondi il dato è considerato fresco;
- `retry` : ritenta meno di due volte solo se `navigator.onLine` è vero;
- `refetchOnWindowFocus: false` : tornare alla scheda non ricarica automaticamente tutto;
- `networkMode: 'offlineFirst'` : mostra prima la cache e tenta poi la rete.

Mutation

- `networkMode: 'offlineFirst'` : una scrittura senza rete può essere sospesa;
- `retry: 3` : errori transitori vengono ritentati.

Persistenza

`createAsyncStoragePersister` collega la cache a IndexedDB tramite `idb-keyval`.

- `getItem` : legge;
- `setItem` : salva;
- `removeItem` : elimina;
- `key: 'trainhub-query-cache'` : nome del contenitore;
- `throttleTime: 1000` : al massimo un salvataggio al secondo.

Quindi esistono due cache differenti:

- Service Worker: file dell'app, per poter avviare la shell;
- IndexedDB/React Query: dati e mutation sospese.

Migrazione della cache

Apri `src/lib/cacheMigration.js`.

Una mutation in attesa salvata ieri contiene parametri per il codice di ieri. Se una nuova versione cambia quel contratto, riprodurla potrebbe essere pericoloso.

`QUERY_CACHE_BUSTER` identifica il formato corrente. `migratePersistedClient(persisted)` :

1. non cambia una cache già corrente;
2. legge le mutation persistite;
3. conserva soltanto le vecchie operazioni esplicitamente compatibili;
4. scarta le altre;
5. restituisce conteggio e cache migrata.

Le compatibili elencate sono disponibilità, invio messaggio e scelta professionista. Per gli scarti, l'app mostra un avviso chiedendo all'utente di ripetere l'azione.

7. Tema e token Figma

Apri in tre colonne:

```
doc/assets/variables.tokens.json
src/theme/tokens.js
src/theme/index.js
```

tokens.js

Importa il JSON esportato da Figma e chiama `resolveTokens(raw)`. Il JSON è la fonte unica; non vengono ricopiati colori schermata per schermata.

Apri `src/theme/resolveTokens.js`.

`flatten(node, prefix, out)` visita ricorsivamente l'albero annidato e produce chiavi come `Action.Primary`. “Ricorsiva” significa che una funzione richiama se stessa per scendere nei gruppi.

`resolveTokens(raw)` risolve anche alias come `{Color.Coral}`:

- `flat`: mappa piatta;
- `deref(key)`: segue un riferimento fino al valore reale;
- `resolving`: Set delle chiavi attualmente visitate;
- se una chiave torna su se stessa, segnala un alias circolare;
- se un riferimento non esiste, genera errore.

theme/index.js

`createTheme` trasforma i token in convenzioni MUI:

- palette primaria, sfondi, testi e stati;
- colori personalizzati per tipi di appuntamento;
- raggio base;
- font Inter e gerarchia h1/h2/h3/body;
- stile comune di card, bottoni, dialog e chip;
- focus da tastiera visibile;
- asterisco rosso per campi obbligatori.

`responsiveFontSizes(theme)` adatta la scala tipografica alle dimensioni dello schermo.

La proprietà `sx` dei componenti legge questi valori. Per esempio `bgcolor: 'primary.main'` non contiene un hex: MUI lo risolve dalla palette.

8. Edge Function push e TypeScript

`Ctrl+P` → `supabase/functions/notify/index.ts`.

Questo file gira sui server Supabase con Deno. Estensione `.ts` significa TypeScript: è JavaScript con informazioni di tipo.

Esempi:

- `Deno.env.get('...')`: legge una variabile server; `!` dice a TypeScript che qui ci si aspetta che esista;

- `result.reason as { statusCode?: number }`: tratta il valore come oggetto con eventuale status code per poterlo leggere in sicurezza durante il controllo dei tipi.

Flusso:

1. configura `web-push` con le chiavi VAPID private del server;
2. crea un client Supabase amministrativo con service role;
3. riceve una richiesta HTTP dal database;
4. confronta `x-notify-secret` con `NOTIFY_SECRET`, altrimenti risponde 403;
5. legge `user_id`, titolo, corpo e URL;
6. legge tutte le sottoscrizioni di quell'utente;
7. invia in parallelo con `Promise.allSettled`;
8. elimina endpoint 404/410, ormai morti;
9. registra gli altri errori;
10. restituisce quanti invii sono riusciti, falliti o eliminati.

La funzione viene distribuita senza verifica JWT perché il chiamante è PostgreSQL e non possiede una sessione utente. La protezione equivalente è il segreto condiviso.

9. Controlli del progetto

Esegui dal terminale:

```
npm run lint
npm run build
```

Servono entrambi: ESLint controlla regole del codice, mentre Vite risolve importazioni e crea davvero il bundle.

Le funzioni pure hanno 13 self-check con `node:assert`. Sono elencati nel README e coprono formattazione, settimana, raggruppamento, token, timer, stato workout, riepilogo, abbonamento, calendario, progressi, push, contratti nutrizionali e premi.

Un file `.selfcheck.js` importa la funzione pura, le passa esempi noti e usa `assert` per dichiarare il risultato atteso. Se non stampa errori e termina con codice 0, è passato.

Database:

- `supabase/verify.sql`: controlli amministrativi di schema e dati;
- `probe-rls.mjs`: letture anonime via API;
- `probe-security.mjs`: letture dei ruoli demo via API.

10. Test offline manuale

1. `npm run build`;

2. `npm run preview`;
3. apri DevTools → Network → Offline;
4. naviga su pagine già visitate;
5. registra un set, invia un messaggio o prenota;
6. riattiva Online;
7. osserva la mutation riprendere e la cache aggiornarsi.

Non tutte le azioni possono essere completate veramente senza rete. Per esempio una prima chat non può inviare finché non esiste un `threadId`; la UI lo comunica invece di accodare dati invalidi.

Su iPhone il push richiede iOS 16.4+, Safari e installazione tramite “Aggiungi a Home”. Una scheda browser normale non riceve Web Push come PWA installata.

Risposta da esame

TrainHub è una PWA costruita con Vite. Il manifest ne definisce installazione e aspetto; un Service Worker precachea la shell e gestisce push e click sulle notifiche. I dati Supabase non vengono duplicati nella cache del worker: TanStack Query li persiste in IndexedDB e conserva le mutation offline, con contratti idempotenti e migrazione versionata. Il tema MUI deriva dai token esportati da Figma, così colori e tipografia sono centralizzati. La build viene verificata con lint, build, self-check e controlli SQL/RLS.

Lezione 15 — Ripasso guidato per l'esame

Questa lezione non introduce nuovo codice. Ti insegna a ricostruire una risposta usando i file veri, senza recitare parole tecniche incomprese.

1. Presentazione del progetto in 60 secondi

TrainHub è una Progressive Web App mobile che collega membri di una palestra e professionisti. Il membro può scegliere il professionista, consultare e svolgere il piano workout, vedere il piano nutrizionale, prenotare, chattare, mostrare un badge QR e controllare punti e abbonamento. Il professionista gestisce clienti, piani, agenda, chat, progressi e scansione degli accessi. Il frontend è scritto in JavaScript e JSX con React, React Router, Material UI e TanStack Query. Supabase gestisce autenticazione, PostgreSQL, sicurezza per riga, Realtime, Storage ed Edge Function push. L'app è installabile e mantiene shell, dati già letti e scritture compatibili anche offline.

Se una parola non è chiara, torna alla lezione in cui viene aperto il file che la usa.

2. Mappa completa: da un click al database

Quando l'utente preme un pulsante, ricostruisci sempre cinque livelli:

```
componente della schermata
  → event handler
  → mutation React Query
  → funzione in src/data
  → tabella o RPC Supabase
  → invalidazione/cache
  → nuovo render
```

Esempio “registra una serie”:

1. `ExerciseDetailScreen.jsx` raccoglie reps e peso;
2. il submit chiama una mutation;
3. `mutations.js` applica subito l'aggiornamento ottimistico;
4. `data/runs.js` chiama `log_workout_set_secure`;
5. SQL controlla utente, run, esercizio e numero serie;
6. React Query conferma o annulla lo stato;
7. la schermata ricalcola il conteggio.

All'esame, questa struttura vale più di un elenco di librerie.

3. I file che devi saper aprire senza esitazione

Avvio e struttura

- `index.html` : contenitore HTML e punto `root` ;
- `src/main.jsx` : monta React e registra PWA;
- `src/App.jsx` : provider di tema, autenticazione, cache e router;
- `src/routes/index.jsx` : URL e schermate;
- `src/layouts/AppLayout.jsx` : struttura comune e protezione dei ruoli.

Dati

- `src/lib/supabase.js` : client API;
- `src/lib/queryClient.js` : cache e offline;
- `src/data/mutations.js` : comportamento comune delle scritture;
- `src/data/*.js` : query e RPC divise per dominio.

Funzioni principali

- workout: `src/features/workout/` ;
- nutrizione: `src/features/nutrition/` ;
- clienti e calendario: cartelle omonime;
- chat: `ThreadScreen.jsx` e `useThreadMessages.js` ;
- badge/scanner: `BadgeScreen.jsx` , `ScannerScreen.jsx` ;
- notifiche: `NotificationsScreen.jsx` ;
- progressi: `ClientProgressScreen.jsx` , `progress.js` .

Backend

- `supabase/schema.sql` : tabelle iniziali;
- `supabase/policies.sql` : RLS;
- `supabase/patches/` : evoluzione e RPC finali;
- `supabase/seed.sql` : dati demo;
- `supabase/verify.sql` : verifiche;
- `supabase/functions/notify/index.ts` : invio Web Push.

4. Domande tecniche con risposta comprensibile

Che linguaggio usa?

Il frontend usa JavaScript moderno. I file `.jsx` aggiungono JSX, sintassi simile all'HTML che React trasforma in elementi. SQL definisce database, vincoli e funzioni PostgreSQL. La sola Edge Function usa TypeScript su Deno.

Che cos'è React?

Una libreria per costruire l'interfaccia con componenti. Ogni componente è una funzione che riceve proprietà e restituisce JSX. Quando stato o dati cambiano, React richiama la funzione e aggiorna solo il DOM necessario.

Che cos'è un hook?

Una funzione React che dà una capacità a un componente: `useState` memoria locale, `useEffect` effetti esterni, `useRef` riferimento stabile, `useContext` dato condiviso. Gli hook vanno chiamati sempre nello stesso ordine, non dentro condizioni casuali.

Props e state?

Le props arrivano dal genitore e sono input del componente. Lo state appartiene all'istanza del componente e cambia tramite il setter. Nessuno dei due è automaticamente persistente nel database.

Che cos'è React Router?

Associa URL a componenti senza ricaricare tutta la pagina. Le rotte figlie vengono mostrate in `Outlet`. `useParams` legge segmenti come `clientId`; `navigate` cambia URL; i redirect correggono percorsi o ruoli.

Che cos'è Material UI?

Una libreria di componenti accessibili e stilizzabili. `sx` applica stile usando valori del tema. TrainHub centralizza colori, tipografia, card e bottoni nel tema derivato dai token Figma.

Che cos'è React Query?

Gestisce dati server, non semplicemente variabili locali. `useQuery` legge e conserva dati; `useMutation` esegue scritture. Le query hanno chiavi; le mutation aggiornano o invalidano le query interessate. La cache viene salvata in IndexedDB.

Che cos'è Supabase?

È il backend: Auth, database PostgreSQL, API, Realtime, Storage ed Edge Functions. Il client pubblico non è amministrativo: RLS e grant limitano ciò che può fare.

Perché esistono `src/data` e `src/features`?

`features` contiene comportamento e UI di una funzione utente. `data` contiene le richieste Supabase. La separazione rende la schermata leggibile e permette di cambiare query senza mescolare dettagli grafici.

Perché UUID creati nel client?

Una scrittura può partire offline ed essere ripetuta. Riutilizzando lo stesso ID, il server riconosce lo stesso tentativo invece di creare duplicati. La RPC verifica che un replay con quell'ID abbia contenuto identico.

Come funziona offline?

Il Service Worker conserva i file dell'app. React Query conserva dati e mutation in IndexedDB. Le letture mostrano la copia disponibile; le scritture idempotenti si sospendono e riprendono alla riconnessione. Le mutation incompatibili dopo un aggiornamento vengono scartate con avviso.

Come funziona la sicurezza?

Il token di sessione determina `auth.uid()`. I grant sono il primo cancello, RLS seleziona righe accessibili e RPC protette gestiscono le scritture complesse. Il database ricontrolla ruolo, relazione membro-coach, abbonamento, vincoli e stato atteso.

5. Tutti i flussi da provare nell'app e seguire nel codice

Per ciascun flusso, esegui nell'app e poi apri i file indicati.

Accesso

1. apri Login;
2. inserisci credenziali;
3. osserva redirect per ruolo;
4. leggi `LoginScreen.jsx` → `AuthProvider.jsx` → `AppLayout.jsx`.

Scelta professionista

1. membro apre Browse Trainers;
2. seleziona il professionista;
3. leggi `BrowseTrainersScreen.jsx` → `data/profile.js`;
4. nota l'aggiornamento di `assigned_pro_id` e del contesto Auth.

Piano workout

1. apri piano e sessione;
2. avvia workout;
3. registra set, pausa/riprendi, concludi;
4. apri nell'ordine `WorkoutPlanScreen`, `SessionDetailScreen`, `OpenRunSheet`, `LiveSessionScreen`, `ExerciseDetailScreen`, `EndRunSheet`, `SessionSummaryScreen`;
5. affianca `useLiveSession.js`, `data/runs.js`, patch 023.

Creazione piano

1. professionista apre un cliente e crea piano;
2. `CreatePlanFlow` governa i passi;
3. `PlanForm` raccoglie metadati;
4. `SessionForm` crea sessioni ed esercizi;
5. `PlanSummary` conferma;
6. `create_workout_plan_secure` salva tutto atomicamente.

Nutrizione

Segui `NutritionPlanEditorScreen` → `CreateNutritionPlanFlow` → `NutritionPlanForm` → `DayForm` → `NutritionPlanSummary` → `data/nutrition.js` → RPC patch 023.

Appuntamento

1. disponibilità del professionista;
2. il membro sceglie slot in `BookingSheet` ;
3. `createAppointment` chiama RPC;
4. membro crea pending, professionista crea confirmed;
5. `AppointmentDetailScreen` usa cambio stato con valore atteso.

Chat

`ThreadListScreen` → `ThreadScreen` → `MessageComposer` → `data/chat.js` ; in parallelo `useThreadMessages` ascolta INSERT e UPDATE Realtime.

Badge e accesso

`BadgeScreen` genera token → `checkin_tokens` → QR → `ScannerScreen` → `redeem_checkin_token` → `checkins` e possibile reward.

Notifiche

Una modifica DB attiva un trigger → `notify_user` inserisce `notifications` → eventualmente chiama Edge Function → Service Worker mostra il push → click apre URL → schermata marca la notifica letta.

6. Errori concettuali da evitare all'esame

- “React salva i dati”: no, React gestisce UI; Supabase salva.
- “JSX è HTML”: gli somiglia, ma è sintassi JavaScript trasformata.
- “Il router apre nuovi file HTML”: no, sceglie componenti dentro la stessa SPA.
- “La anon key è segreta”: no, è pubblica; la sicurezza deve stare in RLS/RPC.
- “La service role è nel frontend”: assolutamente no; è solo nella Edge Function server.
- “Offline significa che ogni operazione può completarsi”: no; shell e dati possono essere disponibili, le scritture compatibili attendono la rete.
- “workout_sessions.status dice lo stato attuale”: no, è legacy; lo stato deriva dai run della settimana.
- “Un run partial completa il target”: no, viene mostrato ma non conta come completed.
- “I punti sono decisi dal client”: no, il database li calcola.
- “Il QR contiene i dati personali”: no, contiene token breve e monouso.
- “Il professionista vede tutti i membri”: no, RLS limita agli assegnati.
- “Il piano nuovo cancella il vecchio”: no, lo collega con `replaces_plan_id`.

7. Simulazione orale: 20 domande

Prova prima senza leggere; poi confronta con le lezioni.

1. Da `index.html` , come si arriva a una schermata?
2. Cosa fanno i provider in `App.jsx` ?

3. Differenza tra componente, funzione normale e custom hook?
 4. Differenza tra props e parametri dell'URL?
 5. Quando useresti `useState` , `useEffect` e `useRef` ?
 6. Come distingue l'app membro e professionista?
 7. Come impedisce di aprire una rotta del ruolo sbagliato?
 8. Come viene letto un piano workout?
 9. Differenza tra prescrizione, run e set log?
 10. Come viene calcolata la durata effettiva?
 11. Come vengono calcolati esito, percentuale e punti?
 12. Perché l'ID di alcune scritture nasce nel browser?
 13. Cosa fa l'aggiornamento ottimistico?
 14. Cosa accade a una mutation durante l'assenza di rete?
 15. Come arrivano i messaggi live?
 16. Come funziona una prenotazione?
 17. Come funziona il badge QR?
 18. Differenza tra notifica persistente e push?
 19. Cosa sono GRANT, RLS e RPC?
 20. Come verifichi che il progetto funzioni e sia sicuro?
-

8. Metodo di studio consigliato

Non leggere tutto passivamente.

Per ogni lezione:

1. apri esattamente il file indicato;
2. trova nel codice ogni riga citata;
3. metti un breakpoint mentale: “quali valori esistono qui?”;
4. riscrivi su carta input → elaborazione → output;
5. usa `Ctrl+click` su un'importazione per seguire il collegamento;
6. torna indietro con `Alt+Left` ;
7. spiega ad alta voce senza leggere;
8. prova il flusso nell'app;
9. rispondi alle domande finali.

Se una riga è complessa, dividila:

```
const threadId = isMember ? (memberThread.data?.id ?? null) : threadIdParam
```

Domande:

1. qual è la condizione? `isMember` ;

2. valore se vero? ID letto dalla query o null;
3. valore se falso? parametro URL;
4. tipo finale? stringa UUID oppure null.

Questo metodo funziona per quasi tutto il progetto.

9. Percorso minimo se hai poco tempo

Studia nell'ordine:

1. lezioni 01 e 02: sintassi e avvio;
2. lezioni 04–06: router, autenticazione, dati/offline;
3. lezione 07: flusso più complesso, workout;
4. lezione 12: modello dati;
5. lezione 13: sicurezza;
6. questa lezione e simulazione orale;
7. poi completa 08–11 e 14.

Non saltare la lezione 01 anche se sembra lenta: se non distingui una funzione chiamata da una funzione passata come callback, ogni file successivo sembrerà incomprensibile.

Ultima risposta: “qual è la scelta architetturale principale?”

L'app separa responsabilità: React compone l'interfaccia, Router sceglie la schermata, React Query governa dati server e offline, i moduli `data` traducono le azioni in API Supabase, PostgreSQL mantiene relazioni e invarianti e le RPC/RLS applicano la sicurezza. Questa separazione permette di seguire ogni funzione dal gesto dell'utente fino alla riga del database e ritorno.

Appendice — Mappa di tutti i file dell'applicazione

Usa questa pagina quando il docente indica un file a caso. Cerca il nome con `Ctrl+F`, poi aprilo con `Ctrl+P`. La colonna “lezione” indica dove viene spiegato nel flusso.

Radice e configurazione

File	Contenuto reale	Lezione
<code>index.html</code>	documento HTML iniziale, metadati PWA, contenitore <code>root</code> , caricamento di <code>main.jsx</code>	02
<code>package.json</code>	nome progetto, script npm, dipendenze runtime e sviluppo	14
<code>package-lock.json</code>	versioni risolte dell'albero npm; viene generato da npm, non contiene logica dell'app	14
<code>vite.config.js</code>	React plugin, PWA, manifest, precache e host di preview	14
<code>eslint.config.js</code>	regole statiche JavaScript/React e ambienti browser/worker/Node	14
<code>pwa-assets.config.js</code>	configurazione che genera le varie icone PWA dall'immagine sorgente	14
<code>.env.example</code>	nomi delle variabili ambiente richieste, senza credenziali vere	14
<code>README.md</code>	installazione, database, push, avvio e verifiche operative	14
<code>CLAUDE.md</code>	regole di collaborazione e manutenzione del repository, non codice runtime	consultazione

`public/` contiene favicon e icone copiate così come sono nella build. `src/assets/hero.png` è un'immagine importata dal frontend. I PNG in `doc/assets/` sono riferimenti Figma/documentazione: non sono ciascuno una classe o una funzione.

Entrata, layout e navigazione

File	Responsabilità	Lezione
<code>src/main.jsx</code>	monta React dentro <code>#root</code> , usa <code>StrictMode</code> e registra il Service Worker	02

File	Responsabilità	Lezione
<code>src/App.jsx</code>	compone AuthProvider, tema MUI, cache persistente, router e ripresa mutation	02
<code>src/routes/index.jsx</code>	importa le schermate in lazy loading e associa ogni URL al componente	04
<code>src/routes/navItems.js</code>	definisce le voci della barra inferiore per membro e professionista	04
<code>src/layouts/PublicLayout.jsx</code>	cornice delle pagine pubbliche come login e recupero password	04
<code>src/layouts/AppLayout.jsx</code>	guardia autenticazione/ruolo, header, banner, contenuto, live bar e bottom nav	04

Componenti condivisi

File	Props o responsabilità
<code>AppointmentCard.jsx</code>	riceve appuntamento, persona correlata e navigazione; mostra tipo, orario e stato
<code>BottomNav.jsx</code>	riceve le voci del ruolo e usa l'URL attuale per evidenziare quella attiva
<code>BrandLogo.jsx</code>	<code>width</code> , <code>withWordmark</code> , <code>ariaLabel</code> ; disegna logo SVG adattato al tema
<code>ClientCard.jsx</code>	<code>client</code> , eventuale <code>to /azione</code> ; riepilogo di un membro
<code>ConfirmDialog.jsx</code>	<code>open</code> , titolo, descrizione, testo pulsanti, stato busy, <code>onConfirm</code> , <code>onClose</code>
<code>LiveSessionBar.jsx</code>	run aperta e callback/navigazione; mini-player persistente dell'allenamento
<code>MembershipBanner.jsx</code>	profilo membro; mostra abbonamento sospeso, scaduto o vicino alla scadenza
<code>MonthGrid.jsx</code>	mese, giorno selezionato, indicatori e callback; griglia calendario
<code>OfflineBanner.jsx</code>	ascolta online/offline, mutation sospese e avvisi di migrazione cache
<code>PageHeader.jsx</code>	titolo, sottotitolo, ritorno, elemento leading e azioni di pagina
<code>PasswordField.jsx</code>	input password con stato interno per mostrare/nascondere; inoltra le normali props TextField
<code>RouteErrorScreen.jsx</code>	pagina di errore del router con retry/ritorno sicuro
<code>ScreenState.jsx</code>	<code>LoadingState</code> , <code>EmptyState</code> , <code>ErrorState</code> : stati standard delle query
<code>SessionCard.jsx</code>	sessione, stato, progressi e azione per aprire/avviare
<code>TopHeader.jsx</code>	utente, notifiche e avatar nella barra superiore

File	Props o responsabilità
WeekStrip.jsx	giorni della settimana, selezione e callback

Sono spiegati soprattutto nella lezione 03; i componenti calendario/workout ricompaiono nelle lezioni 07 e 09.

Accesso ai dati

File	Dominio
src/data/appointments.js	letture agenda/range, creazione e cambio stato appuntamenti
src/data/availability.js	lettura, aggiunta ed eliminazione disponibilità
src/data/chat.js	thread, messaggi, invio e conferma di lettura
src/data/checkin.js	codice badge, token e riscatto
src/data/clients.js	roster, singolo cliente e abbonamento
src/data/mutations.js	associa mutation key a funzioni, ottimismo, rollback e invalidazioni
src/data/notifications.js	lista/conteggio e RPC di gestione notifiche
src/data/nutrition.js	piano corrente e creazione atomica
src/data/profile.js	professionisti, assegnazione e avatar Storage
src/data/push.js	salvataggio/eliminazione sottoscrizione push
src/data/rewards.js	cronologia punti del membro
src/data/runs.js	apertura, pausa, ripresa, chiusura, note e log della run
src/data/workouts.js	piano, sessione, esercizio, catalogo, set e creazione piano

Tutte le firme e i parametri sono elencati nella lezione 06; i significati di dominio sono approfonditi nelle lezioni 07–11.

Librerie interne

File	Funzioni/oggetti	
src/lib/supabase.js	costruisce il client con URL/key ambiente e opzioni Auth	05
src/lib/queryClient.js	queryClient , persister , durata e configurazione offline	06, 14
src/lib/queryKeys.js	chiavi uniche e prefissi delle query	06
src/lib/mutationKeys.js	nomi stabili delle mutation	06
src/lib/cacheMigration.js	versione e migrazione delle mutation persistite	06, 14

File	Funzioni/oggetti	
<code>src/lib/format.js</code>	date, ore, durata e formati mostrati all'utente	varie
<code>src/lib/week.js</code>	calcoli della settimana locale e intervalli	07
<code>src/lib/dayGroups.js</code>	raggruppa elementi cronologici in Today/Yesterday/data	10, 11
<code>src/lib/uuid.js</code>	crea UUID tramite API crittografica del browser	06, 07

Ogni file con lo stesso nome e suffisso `.selfcheck.js` è un test minimale della corrispondente logica pura. Non viene importato dall'app in produzione: si esegue con Node e fallisce se un `assert` non coincide.

Autenticazione

File	Responsabilità
<code>AuthContext.js</code>	crea il contenitore React condiviso per sessione e profilo
<code>AuthProvider.jsx</code>	legge sessione, ascolta cambi Auth, carica profilo, espone signOut
<code>useAuth.js</code>	custom hook che legge il context e segnala uso fuori dal provider
<code>LoginScreen.jsx</code>	email/password, errore, sign-in e redirect per ruolo
<code>ForgotPasswordScreen.jsx</code>	richiesta email di recupero con risposta non rivelatrice
<code>ResetPasswordScreen.jsx</code>	recupera sessione PKCE e imposta nuova password

Tutto nella lezione 05.

Home, clienti, trainer e agenda

File	Responsabilità
<code>MemberHomeScreen.jsx</code>	riepilogo giornaliero del membro: attività, workout, abbonamento e scorciatoie
<code>ProfessionalHomeScreen.jsx</code>	agenda del giorno del professionista
<code>ClientsScreen.jsx</code>	elenco e ricerca dei clienti assegnati
<code>ClientDetailScreen.jsx</code>	dossier cliente e accesso a workout, nutrizione, progressi, chat, appuntamenti
<code>ClientWorkoutScreen.jsx</code>	piano cliente, sessioni lazy e sostituzione/creazione
<code>ClientAppointmentsScreen.jsx</code>	appuntamenti di uno specifico cliente
<code>subscription.js</code>	stato derivato, attivo/non attivo e azione disponibile
<code>BrowseTrainersScreen.jsx</code>	elenco professionisti e scelta

File	Responsabilità
<code>MyTrainerScreen.jsx</code>	professionista assegnato e collegamenti a chat/appuntamenti
<code>MemberAppointmentsScreen.jsx</code>	calendario appuntamenti del membro e apertura prenotazione
<code>BookingSheet.jsx</code>	scelta giorno, tipo, slot e nota da parte del membro

Lezioni 09 e 11 per abbonamento.

Calendario

File	Responsabilità
<code>CalendarScreen.jsx</code>	mese professionale, selezione giorno e appuntamenti
<code>month.js</code>	funzioni pure per griglia e intervallo del mese
<code>AvailabilityScreen.jsx</code>	gestione delle fasce settimanali del professionista
<code>NewAppointmentSheet.jsx</code>	creazione professionale per uno dei propri clienti
<code>AppointmentDetailScreen.jsx</code>	dettaglio e transizioni di stato protette

`month.selfcheck.js` prova i casi limite della griglia. Lezione 09.

Workout membro e creazione piano

File	Responsabilità
<code>WorkoutPlanScreen.jsx</code>	piano attivo e stato settimanale delle sessioni
<code>SessionDetailScreen.jsx</code>	prescrizione completa e apertura del foglio Start/Resume
<code>OpenRunSheet.jsx</code>	decide avvio, ripresa o abbandono della run già aperta
<code>LiveSessionScreen.jsx</code>	shell dell'allenamento attivo e timer
<code>useLiveSession.js</code>	carica run/sessione/log e orchestra pausa/ripresa/fine
<code>ExerciseDetailScreen.jsx</code>	registra serie, ripetizioni e peso per l'esercizio corrente
<code>RestTimer.jsx</code>	conto alla rovescia recupero e segnale sonoro
<code>EndRunSheet.jsx</code>	scelta di terminare o abbandonare
<code>SessionSummaryScreen.jsx</code>	percentuale/esito/punti e nota al professionista
<code>CongratsDialog.jsx</code>	dialog celebrativo quando un risultato lo prevede
<code>timer.js</code>	tempo attivo meno pause
<code>status.js</code>	stato settimanale derivato per una sessione
<code>summary.js</code>	percentuali, punti e prossima soglia premio

File	Responsabilità
<code>beep.js</code>	genera breve suono tramite Web Audio senza file esterno
<code>contracts.js</code>	normalizza e valida la forma del piano e delle sessioni
<code>NewPlanScreen.jsx</code>	pagina contenitore del wizard nuovo piano
<code>CreatePlanFlow.jsx</code>	macchina a passi del wizard
<code>PlanForm.jsx</code>	nome, obiettivo, livello, settimane
<code>SessionForm.jsx</code>	nome sessione ed esercizi prescritti
<code>PlanSummary.jsx</code>	riepilogo prima del salvataggio atomico

Lezioni 07 e 08. I relativi `.selfcheck.js` coprono timer, stato, riepilogo e contratti.

Nutrizione

File	Responsabilità
<code>MemberNutritionScreen.jsx</code>	sceglie il day type del giorno e mostra piano/pasti
<code>MealDetailScreen.jsx</code>	dettaglio alimenti, macro e alternative del pasto
<code>NutritionPlanEditorScreen.jsx</code>	contenitore professionale per creazione/sostituzione
<code>CreateNutritionPlanFlow.jsx</code>	macchina a passi per piano, giorni, pasti e riepilogo
<code>NutritionPlanForm.jsx</code>	metadati e obiettivi nutrizionali
<code>DayForm.jsx</code>	day type, weekdays e MealBuilder interno
<code>NutritionPlanSummary.jsx</code>	conferma del piano completo
<code>contracts.js</code>	normalizzazione/validazione della struttura JSON

Lezione 08; `contracts.selfcheck.js` verifica i contratti puri.

Chat

File	Responsabilità
<code>ThreadListScreen.jsx</code>	inbox professionale, ricerca e non letti
<code>ThreadScreen.jsx</code>	conversazione dei due ruoli, letture, invio e stati
<code>ClientChatRedirectScreen.jsx</code>	da un cliente crea/ritrova il thread e reindirizza all'URL corretto
<code>MessageComposer.jsx</code>	campo controllato e callback <code>onSend</code>
<code>MessageBubble.jsx</code>	forma, ora e ricevuta di lettura del messaggio
<code>useThreadMessages.js</code>	query messaggi e canale Realtime INSERT/UPDATE

Lezione 10.

Profilo, check-in, premi, notifiche e progressi

File	Responsabilità
<code>ProfileScreen.jsx</code>	menu personale differenziato per ruolo
<code>AvatarPicker.jsx</code>	scelta, riduzione, upload e rimozione avatar
<code>BadgeScreen.jsx</code>	token temporaneo, QR e conto alla rovescia
<code>SubscriptionScreen.jsx</code>	stato e servizi inclusi nell'abbonamento membro
<code>SettingsScreen.jsx</code>	cambio password e uscita
<code>NotificationSwitch.jsx</code>	interruttore push con stati supportato/configurato/permesso
<code>pushSubscription.js</code>	conversione VAPID e subscribe/unsubscribe
<code>ScannerScreen.jsx</code>	fotocamera, jsQR, deduplica e inserimento manuale
<code>scanResult.js</code>	traduce gli esiti server per la UI
<code>RewardsScreen.jsx</code>	totale, catalogo soglie e cronologia punti
<code>grouping.js</code>	raggruppa i reward per mese con limite
<code>NotificationsScreen.jsx</code>	lista, filtri, selezione, lettura ed eliminazione
<code>ClientProgressScreen.jsx</code>	riepilogo settimana e run recenti del cliente
<code>WorkoutRunDetailScreen.jsx</code>	dettaglio run e confronto set fatti/prescritti
<code>progress.js</code>	aggregazioni pure e durata senza pause

Lezione 11. I corrispondenti self-check provano i moduli puri.

Tema e PWA

File	Responsabilità
<code>src/theme/tokens.js</code>	importa il JSON Figma e ne esporta i token risolti
<code>src/theme/resolveTokens.js</code>	appiattisce albero e risolve alias/cicli
<code>src/theme/index.js</code>	crea palette, tipografia e override MUI
<code>src/sw.js</code>	precache shell, fallback SPA, push e click
<code>doc/assets/variables.tokens.json</code>	export dei design token, fonte cromatica

Lezione 14; `resolveTokens.selfcheck.js` verifica alias e casi di errore.

Supabase

File	Responsabilità
<code>schema.sql</code>	enum, tabelle, indici, vincoli e trigger iniziale profilo
<code>policies.sql</code>	RLS iniziale e helper di autorizzazione
<code>seed.sql</code>	ricostruisce dati demo coerenti e ancorati alla data corrente
<code>verify.sql</code>	controlli SQL che devono tutti risultare PASS
<code>probe-rls.mjs</code>	verifica dall'API ciò che può leggere un anonimo
<code>probe-security.mjs</code>	verifica letture dei ruoli demo
<code>INSTALL.md</code>	ordine completo di installazione e gestione demo
<code>functions/notify/index.ts</code>	Edge Function server per Web Push

Lezioni 12–14.

Tutte le patch, in una riga ciascuna

Patch	Scopo
001	chiude la lettura anonima dei profili professionali
002	vecchio riallineamento agenda, superato dal seed
003	deriva i punti reward sul server
004	aggiunge body metrics
005	vecchi clienti demo, superato dal seed
006	ripristina i grant necessari
007	pubblica messages su Realtime
008	limita UPDATE messaggi alla colonna <code>read_at</code>
009	token e RPC iniziale di check-in
010	configurazione, trigger ed Edge Function push
011	fissa il <code>search_path</code> delle funzioni <code>definer</code>
012	notifica anche l'appuntamento creato dal professionista
013	introduce workout runs e collegamento dei set
014	aggiunge percentuale run
015	grande hardening: RPC sensibili, revoke, vincoli e trigger
016	limita i set contati a quelli prescritti

Patch	Scopo
017	consente piano workout creato dal membro stesso
018	crea piano e più sessioni in un'unica chiamata
019	espande catalogo esercizi
020	aggiunge centro notifiche persistente
021	azioni multiple sulle notifiche e correzione fuso appuntamenti
022	day types nutrizionali e nuova RPC
023	applica davvero lo stato dell'abbonamento
024	assegna punti al check-in
025	bucket e policy avatar
026	rende giornaliero il premio check-in nel fuso di Roma

File di documentazione

- `doc/context.md`, `trainhub.md`, `live_train_session.md`, `nutrition_plan.md` : requisiti e decisioni funzionali;
- `docs/superpowers/specs/` : motivazione delle fasi;
- `docs/superpowers/plans/` : passi di implementazione;
- `docs/ONBOARDING-AGENT.md` : orientamento tecnico;
- `docs/ux-refinement-nielsen.md` : revisione euristica UX;
- `doc/technical_report_examples/` : esempi esterni per la relazione, non codice TrainHub.

Questi file spiegano il perché storico; se una descrizione contraddice il codice e le patch finali, per il comportamento attuale valgono codice, schema finale e verifiche.